**COMPUTER SYSTEMS ENGINEERING PROGRAM**

**ISS 496 — Senior Project Report**

# Real-Time Vehicle Monitoring and Diagnostics System


**By:**

Yassine Mtibaa  Mohamed Baccouche  Mahdi Hachicha

Hana Gdoura  Khadija Ben Hamida  Selima Grissa

Fares Smaoui

**Academic Supervisor:**

Dr. Mortadha Dahmani

**Institution Supervisor:**

Achraf *[Last Name]*

BAKO Motors


Tunis, 2025 – 2026

# ABSTRACT

BAKO Motors Monitoring System is an embedded IoT solution developed as part of the ISS 496 senior project at MedTech Mediterranean Institute of Technology. The project addresses critical gaps in BAKO Motors' electric vehicle platform, including the absence of real-time sensor monitoring beyond the battery, no vehicle-wide data acquisition interface, and insufficient diagnostic documentation. The system is built around an ESP32 microcontroller interfaced with a CAN bus (via MCP2515/TJA1051T), a suite of 12 external sensors (current, voltage, temperature, handbrake, GNSS), and a custom Python-based sensor emulator for testing. A real-time web dashboard streams live vehicle telemetry via WebSocket, while a cloud pipeline transmits data over GPRS (SIM800L) to a SQLite database hosted on a VPS. A custom PCB was designed in KiCad to house all components. At the midway stage, the project has achieved 90% completion in embedded development and 80% in system design and dashboard visualisation, with future work focused on GNSS tracking, GSM/GPRS connectivity, and full in-vehicle PCB installation.

**Keywords:** Electric Vehicle, CAN Bus, ESP32, BMS, IoT, Real-Time Monitoring, Telemetry, PCB Design, BAKO Motor.

# ACKNOWLEDGEMENTS

The project team wishes to express sincere gratitude to all those who made this work possible.

We are deeply grateful to our academic supervisor, **Dr. Mortadha Dahmani**, for his continuous guidance, technical advice, and institutional support throughout the project. His provision of VPS infrastructure and dedicated office hours were instrumental in enabling the cloud connectivity features of this system.

We thank **BAKO Motors** and our institution supervisor, **Achraf**, for granting access to their electric motorcycle prototype and for sharing domain knowledge about the vehicle's CAN bus architecture. The weekly on-vehicle testing sessions held on the MedTech campus were critical for validating the monitoring system under real operating conditions.

We acknowledge the **MedTech Mediterranean Institute of Technology FA-BLAB** for providing unrestricted access to soldering stations, bench power supplies, logic analysers, and PCB rework equipment throughout the project lifecycle. This support significantly reduced prototype development costs and timelines.

Finally, the team thanks all peers and faculty members who provided feedback during progress reviews, and the open-source communities behind the ESP32 Arduino framework, KiCad, FastAPI, and the many libraries that form the backbone of this system.

# TABLE OF CONTENTS

# LIST OF TABLES

# LIST OF FIGURES

# 1 EXECUTIVE SUMMARY

## 1.1 Project Overview

The BAKO Motors Monitoring System is a full-stack embedded IoT platform developed to provide real-time diagnostics and telemetry for BAKO Motors' electric vehicle prototype. The system integrates an ESP32-based custom hardware board with a CAN bus interface, a twelve-sensor network, a live WebSocket web dashboard, and a cloud database pipeline through GSM/GPRS connectivity.

The project was designed and developed by a seven-member engineering student team at MedTech Mediterranean Institute of Technology (SMU) as part of the ISS 496 senior project (one academic semester), in direct collaboration with BAKO Motors as the industrial partner. The system is intended to serve both as a diagnostic tool for engineers and as a foundation for future predictive maintenance capabilities.

## 1.2 Client Brief & Objectives

BAKO Motors identified several critical gaps in their existing electric vehicle prototype:

- the CAN bus communication was limited to the battery management system only;
- no real-time data access was available to engineers, and no monitoring interface existed for vehicle diagnostics;
- sensors were exclusively placed on the battery pack with no coverage of the motor, MPPT controller, or power distribution unit;
- insufficient system documentation was available for maintenance and development teams.

The primary objectives of this project were therefore: to extend CAN bus monitoring across the full vehicle by adding external sensors; to develop a real-time web-based dashboard accessible over Wi-Fi and LTE; to design and fabricate a custom PCB integrating all monitoring electronics; to implement a cloud telemetry pipeline for remote data storage and analysis; and to produce a Python-based software emulator enabling full system testing without physical hardware.

## 1.3 Key Milestones & Timeline

The project was structured across a one-semester academic timeline following an iterative Agile development approach. The first phase covered system design, component selection, and feasibility study, culminating in a validated bill of materials and system architecture synoptic. The

second phase focused on hardware development, firmware implementation, dashboard development, and cloud pipeline integration.

Key milestones achieved at the midway stage include: completion of the system architecture and synoptic design; full PCB schematic and layout in KiCad (Revision RECTIFICATION_2, April 2026); implementation of CAN bus communication with the BMS via MCP2515; DS18B20 temperature sensor integration for motor, MPPT, and DC/DC converter monitoring; current and voltage measurement across all energy nodes; a Python-based sensor emulator with seven automated test scenarios; a real-time WebSocket dashboard with data export functionality; and a cloud telemetry pipeline from ESP32 through SIM800L to a SQLite database on a VPS.

Remaining work at the midway stage includes: GNSS position tracking integration, full SIM800L GSM/GPRS connectivity, fault detection and SMS alert system, in-vehicle PCB installation and field validation, and completion of testing and quality assurance procedures.

## 1.4 Document Conventions

This report follows the MedTech ISS 496 reporting standard and all electronic references use the IEEE citation style as specified in the MedTech Internship Guide. Component references throughout the document follow KiCad designator conventions, where U denotes integrated circuits, J denotes connectors, R denotes resistors, C denotes capacitors, D denotes diodes, and F denotes fuses.

Voltage levels are expressed in volts (V) or millivolts (mV). Temperature values are expressed in degrees Celsius (°C). Current values are expressed in amperes (A). CAN frame identifiers are expressed in hexadecimal notation with the `0x` prefix. All PCB dimensions are given in millimetres (mm) and electrical trace widths in micrometres (µm) unless otherwise stated.

# 2 INTRODUCTION

## 2.1 Background

The global electric vehicle industry is undergoing rapid expansion, driven by environmental awareness, rising fuel costs, and legislative pressure to decarbonise transportation. In emerging markets such as Tunisia, small electric vehicle manufacturers and startups are beginning to establish themselves, facing unique challenges related to limited infrastructure, local market constraints, and the need for affordable yet reliable technology.

BAKO Motors is a Tunisian company founded in 2021 that designs and manufactures solar-powered electric micro-vehicles for urban mobility and last-mile delivery. Their vehicles incorporate a lithium-ion battery pack with a dedicated BMS, a solar panel with MPPT controller, a three-phase motor, and a DC/DC power distribution unit. Despite the technical sophistication of their drivetrain, BAKO Motors lacked a comprehensive monitoring and diagnostics system capable of observing all these subsystems in real time.

## 2.2 Problem Statement

Prior to this project, the BAKO Motors vehicle monitoring infrastructure suffered from five critical deficiencies. First, the CAN bus was limited exclusively to the battery BMS. Second, no real-time data access mechanism existed. Third, no monitoring interface or dashboard had been developed. Fourth, sensor placement was restricted to the battery pack, leaving the motor, DC/DC converter, and solar system entirely unmeasured. Fifth, insufficient system documentation was available for maintenance and development.

These deficiencies translate directly into operational risks: undetected overheating at the motor or DC/DC converter can cause component failure; lack of solar performance data prevents MPPT optimisation; and the absence of remote diagnostics limits field support capability. This project was commissioned to solve all five problems through a unified embedded IoT monitoring solution.

## 2.3 Scope of Work & Deliverables

The scope covers the full hardware and software stack: component selection and justification; custom PCB schematic and layout in KiCad; ESP32 firmware for CAN bus, sensor acquisition, and data transmission; a Python sensor emulator; a real-time WebSocket dashboard with data export; and a cloud telemetry pipeline using LTE, FastAPI, and SQLite.

Primary deliverables include: a validated bill of materials (BOM); a complete PCB design

package (schematic, layout, and Gerber files); documented firmware source code; a functional live dashboard; a live cloud platform with database schema; a Python emulator covering seven test scenarios; and this technical report. The scope explicitly excludes motor control algorithms, BMS cell balancing logic, and mechanical vehicle chassis integration.

## 2.4 Report Structure

This report is organised into twenty-two chapters following the MedTech ISS 496 structure. Chapters 1 through 4 cover the executive summary, introduction, company context, and research phase respectively. Chapter 5 describes the full systems architecture. Chapters 6 through 11 detail the preparation, circuit design, mechanical design, software implementation, cloud connectivity, and implementation phases. Chapters 12 through 22 address Scrum and Agile management, bill of materials, resources, limitations, blockages, risk management, quality assurance, results, recommendations, and conclusions. A full reference list is provided at the end of the document.

# 3 COMPANY CONTEXT

## 3.1 Description of the Company

Bako Motors is a Tunisian company founded in 2021, specialising in the design and manufacturing of solar-powered electric vehicles dedicated to micro-mobility and last-mile delivery applications. The company focuses on providing sustainable and accessible transportation solutions adapted to urban environments and emerging markets.

Bako Motors develops compact electric vehicles that integrate renewable energy, particularly solar power, to enhance energy efficiency and reduce dependence on conventional charging infrastructure. Its product portfolio includes small urban vehicles and utility vans designed for both personal mobility and logistics purposes.

Through its activities, Bako Motors contributes to the transition toward cleaner transportation by promoting environmentally responsible solutions and reducing the carbon footprint of mobility systems. The company operates within a growing ecosystem focused on sustainable development and green energy adoption.

## 3.2 Mission and Objectives

The mission of Bako Motors is to develop sustainable mobility solutions by powering electric vehicles with renewable energy sources. The company aims to reduce environmental impact while offering innovative and practical transportation alternatives adapted to modern urban needs.

The company's objectives can be summarised as follows:

- ensuring high levels of customer satisfaction by improving product reliability and maintaining responsive and efficient support services;
- maintaining strict compliance with legal and regulatory requirements, while anticipating future standards;
- promoting continuous improvement through investment in technology, training, and collaboration;
- defining and monitoring clear quality objectives to enhance organisational performance;
- providing a safe and healthy working environment through proper equipment, procedures, and continuous awareness.

Within the scope of this project, these objectives are directly reflected in the development of an IoT-based real-time monitoring and diagnostics system, which contributes to improving vehi-

cle reliability, enabling predictive maintenance, and supporting sustainable and efficient vehicle operation.

## 3.3 Industry Structure

Bako Motors operates within the electric mobility and clean transportation industry, which is part of the broader automotive and sustainable energy sectors. This industry is currently experiencing rapid growth due to increasing environmental concerns, rising fuel costs, and global regulatory pressure to reduce carbon emissions.

In recent years, the electric vehicle (EV) market has evolved from a niche segment into a key pillar of the automotive industry. It is driven by advancements in battery technology, renewable energy integration, and the development of intelligent transportation systems. Within this context, companies focusing on micro-mobility and small urban electric vehicles are gaining importance due to the growing demand for efficient and affordable city transportation solutions.

In Tunisia and similar emerging markets, the EV industry is still in a developing phase. However, it presents significant potential due to urbanisation, increasing energy awareness, and governmental interest in sustainable development. Companies such as Bako Motors contribute to shaping this emerging ecosystem by introducing locally adapted solutions based on renewable energy and electric mobility.

The industry is characterised by strong technological convergence between automotive engineering, embedded systems, energy management, and IoT-based connectivity, which aligns directly with the objectives of this project.

## 3.4 Market Structure

The market structure for electric mobility solutions can be described as an emerging and moderately competitive market, particularly in developing regions such as Tunisia. Bako Motors operates in a niche segment focused on solar-powered electric micro-vehicles, targeting urban mobility and last-mile delivery solutions rather than mass-market passenger vehicles.

At the global level, the EV market is dominated by large automotive companies and technology-driven firms. In local and regional markets, smaller companies like Bako Motors compete by offering cost-effective, energy-efficient, and locally adapted solutions. Overall, the market structure is evolving toward increased electrification, digitalisation, and connectivity, creating opportunities for IoT-based solutions such as real-time vehicle monitoring and predictive diagnostics systems.

# 4 RESEARCH PHASE

## 4.1 State of the Art — On-Board Diagnostics

### OBD-II Standard and PIDs

On-Board Diagnostics version 2 (OBD-II) is a mandatory diagnostic interface standard for passenger vehicles sold in the United States since 1996, adopted by the European Union under the EOBD directive and widely applied globally. The standard defines a 16-pin diagnostic connector (SAE J1962), a set of communication protocols (ISO 9141-2, ISO 14230 KWP2000, SAE J1850, ISO 15765-4 CAN), and a standardised set of **Parameter IDs (PIDs)** that any compliant scan tool can query from the vehicle's Electronic Control Units (ECUs).

PIDs are structured request-response pairs: the scan tool sends a request frame containing a service ID and a PID number; the ECU responds with the requested parameter. For example, PID 0x0D requests vehicle speed, PID 0x0C requests engine RPM, and PID 0x05 requests coolant temperature. Service 0x03 retrieves stored Diagnostic Trouble Codes (DTCs), and service 0x01 returns live powertrain data. The OBD-II framework is therefore a *request-driven* diagnostic architecture: the scan tool controls what data is visible.

### ECU Architecture and Diagnostic Logic

A modern vehicle contains multiple ECUs interconnected on one or more CAN buses. Each ECU manages a specific vehicle domain: the Engine Control Module (ECM) governs fuel injection and ignition; the Transmission Control Module (TCM) controls gear selection; the Battery Management System (BMS) supervises the high-voltage battery pack in electric vehicles; and the Body Control Module (BCM) manages lighting, wipers, and comfort systems.

Each ECU independently monitors its sensors, applies control algorithms, and stores fault events as DTCs when a measured parameter falls outside calibrated limits. The diagnostic gateway ECU arbitrates access between external scan tools (connected via the OBD-II port) and the internal vehicle network, translating standardised OBD-II PID requests into proprietary CAN messages addressed to the relevant ECU.

### Limitations of OBD-II for EV Monitoring

The OBD-II standard was originally designed around internal combustion engines and covers parameters such as RPM, coolant temperature, and fuel trim that have no direct equivalent in electric vehicles. While the ISO 15765-4 CAN transport layer used for OBD-II communication is di-

rectly compatible with EV CAN buses, the PID set defined by the standard does not cover electric-vehicle-specific data: battery cell voltages, per-cell temperature distribution, state of charge, MPPT solar input, or motor winding temperature.

Electric vehicle manufacturers therefore transmit this EV-specific telemetry using proprietary CAN frame formats defined by their own DBC files. The **SAE J1939** standard, originally developed for heavy-duty commercial vehicles, is increasingly adopted by EV battery management systems (including the O'CELL IFS60.8 used in this project) as it provides a structured addressing scheme (Source Address, Function Code, Destination Address) and a well-defined 29-bit extended CAN ID format.

### Positioning of the BAKO ISS System

The BAKO ISS monitoring system operates in a complementary role to OBD-II rather than as a replacement. It communicates directly on the vehicle's J1939 CAN bus to capture high-resolution BMS telemetry (19 individual cell voltages, 4 temperature probes, SOC, pack current) that is invisible to any generic OBD-II scan tool. This approach — passive CAN listening combined with active sensor polling — gives the system access to data that reflects the true health of the electric powertrain rather than the OBD-II subset that regulatory compliance requires.

## 4.2 Market & Competitor Analysis

### Market Context

The solar-powered electric motorcycle market is growing steadily across North Africa and Sub-Saharan Africa, where fuel costs are high and solar irradiance is excellent. BAKO Motors positions itself in this space by offering electric motorcycles paired with solar charging (MPPT-based), making real-time monitoring of energy flow, battery health, and thermal conditions a critical product differentiator.

### Competitor Landscape

Existing monitoring solutions fall into three categories:

- **Generic EV dashboards** (e.g., those embedded in Chinese EV controllers) offer basic SOC display but no solar integration or CAN bus decoding.
- **Off-the-shelf MPPT monitors** track solar data but have no motor or battery CAN interface.
- **Professional BMS displays** (e.g., DALY BMS app, JK BMS Bluetooth) cover battery data via CAN

or UART but lack MPPT voltage/current measurement and thermal mapping around the motor.

### BAKO Motors' Differentiation

This monitoring system consolidates CAN bus battery data (via MCP2515), multi-point temperature monitoring, MPPT output current measurement, and multi-source voltage measurement (solar panel, MPPT output, battery) into a single unified interface on an ESP32-based platform at a fraction of the cost of comparable commercial systems.

## 4.3 Feasibility Study

### Technical Feasibility

The ESP32 microcontroller is well-suited as the central processing unit. It supports SPI (for MCP2515 CAN interface), 1-Wire (for DS18B20 temperature sensors), ADC inputs (for voltage and current measurement), and Wi-Fi/Bluetooth for wireless data transmission or dashboard display. All four monitoring functions can be handled within a single cooperative main loop on the ESP32, as CAN frames and sensor polling intervals are compatible with sequential scheduling.

The MCP2515 CAN controller communicates with the battery management system over standard CAN 2.0B at 250 kbps or 500 kbps, which are the most common rates in EV battery packs. Decoding cell voltages, current, and SOC from CAN frames is feasible once the BMS CAN DBC (database map) is known or reverse-engineered.

DS18B20 sensors are well-proven for embedded thermal monitoring, supporting up to 127 sensors on a single 1-Wire bus — more than sufficient for motor and MPPT placement. Current measurement via a shunt resistor or ACS712/ACS758 Hall-effect sensor on the MPPT output line is straightforward and accurate to within $\pm 1$–$2\%$.

Voltage divider networks are used to scale down MPPT output (typically 24–48 V), solar panel voltage (up to 50 V), and battery voltage into the 0–3.3 V safe range for the ESP32 ADC.

### Economic Feasibility

The full BOM is low-cost in the Tunisian market: ESP32-DEVKITC (~30 TND), MCP2515 SPI module (~5 TND each), DS18B20 temperature sensors (~3 TND each), ACS712 current sensor (~5 TND), passive components and connectors (~17 TND total). Total prototype component cost was approximately 60 TND, with PCB fabrication adding 180 TND, making the total hardware cost approximately 240 TND — highly viable for both prototype and small-scale production in the Tunisian context.

## Operational Feasibility

The system can operate continuously from the motorcycle's battery supply, with a regulated 3.3 V/5 V step-down. A watchdog timer ensures recovery from any software hang. Data can be displayed locally via an LCD/OLED or transmitted remotely via a cellular module (SIM800L GPRS) for fleet-level monitoring.

## Key Risks

CAN frame decoding depends on the specific BMS protocol used — if proprietary, reverse engineering will be required. ADC accuracy on the ESP32 is limited (∼11-bit effective), so calibration routines are essential for precise voltage and current readings.

## 4.4 Standards & Regulatory Review

The following standards are relevant to the BAKO Motors Monitoring System:

**Table 1:** Applicable Standards and Regulatory Framework

| **Domain** | Standard | Relevance |
|---|---|---|
| **CAN Bus Communication** | ISO 11898-1/2 | CAN 2.0B protocol used with MCP2515 |
| **Battery Data (BMS)** | CiA 447 / proprietary DBC | CAN frame structure for EV battery packs |
| **Temperature Sensing** | DS18B20 datasheet | Accuracy class and operating range |
| **Electrical Safety** | IEC 62368-1 | General electronics safety for the monitoring unit |
| **EMC** | CISPR 25 | Emissions from switching regulators and CAN lines |
| **GSM Module (SIM800L)** | ETSI TS 51.010 | Radio conformance for GSM/GPRS transmission |
| **RoHS** | EU 2011/65/EU | Lead-free component compliance |
| **Solar / MPPT** | IEC 62109-1 | Safety of power converters in PV systems |

## Key Design Implications

CAN bus lines require 120 Ω termination resistors at both ends of the bus. DS18B20 sensors require a 4.7 kΩ pull-up on the data line. Voltage dividers must be calculated to ensure no ADC input ever exceeds 3.3 V under any fault condition. The SIM800L requires compliance with local telecom authority regulations (ATI — Agence Tunisienne des Infrastructures) for GSM transmission.

## 4.5 Technology Stack Selection

## Microcontroller Selection: Why the ESP32?

The choice of microcontroller was the single most consequential design decision in the project, as it determines the available peripherals, firmware ecosystem, cost, and long-term maintainability. Three candidate platforms were evaluated before the ESP32 was selected.

**Table 2:** Microcontroller Platform Comparison

| **Criterion** | ESP32 | Arduino Mega 2560 | STM32F4 (Nucleo) |
|---|---|---|---|
| **CPU speed** | 240 MHz dual-core | 16 MHz single-core | 168 MHz single-core |
| **Wi-Fi / BT** | Built-in (802.11 + BLE 4.2) | None | None (external module) |
| **UART count** | 3 hardware UARTs | 4 hardware UARTs | 3 hardware UARTs |
| **SPI buses** | 4 (VSPI, HSPI, …) | 1 | Multiple |
| **ADC** | 12-bit, 18 channels | 10-bit, 16 channels | 12-bit, 24 channels |
| **RAM** | 520 kB SRAM | 8 kB SRAM | 192 kB SRAM |
| **Flash** | 4 MB (external) | 256 kB | 1 MB |
| **Arduino-compat** | Full (arduino-esp32) | Native | Partial |
| **Unit cost (TND)** | ~30 | ~60 | ~80 |
| **Local stock** | Available | Available | Rare |

The **Arduino Mega 2560** was rejected primarily because of its 8-bit, 16 MHz processor and 8 kB SRAM, which are insufficient for simultaneous CAN frame decoding, JSON serialisation (which alone requires a 1 kB buffer), and AT command management for the cellular module. It also lacks built-in Wi-Fi and Bluetooth, which are required for the local dashboard connection.

The **STM32F4 Nucleo** offers adequate processing power and ADC precision, but its Arduino compatibility layer is incomplete for the `mcp_can` and `DallasTemperature` libraries used in this project. Its higher cost and near-zero availability in the Tunisian local market were additional disqualifying factors.

The **ESP32** was selected because it uniquely satisfies all requirements simultaneously: sufficient processing speed for the cooperative multi-sensor loop, built-in Wi-Fi and BLE for local dashboard connectivity, three independent hardware UARTs (one for SIM800L, one for debugging, one spare), SPI for the MCP2515, and ADC channels for the analogue sensors. Its mature `arduino-esp32` framework gives access to every required library. At approximately 30 TND from local Tunisian suppliers, it is also the most cost-effective option.

## Key Module Selection Rationale

**Core Microcontroller:** ESP32 (Espressif) — dual-core 240 MHz, built-in Wi-Fi + Bluetooth, multiple UARTs, SPI, I2C, 1-Wire support, 12-bit ADC. Selected for its versatility, community support, and low cost.

**CAN Interface:** MCP2515 (Microchip) + TJA1050 CAN transceiver — SPI-connected to ESP32, supports CAN 2.0B up to 1 Mbps. Handles all battery communication frames.

**Temperature Sensing:** DS18B20 — digital 1-Wire sensor, $\pm 0.5$°C accuracy, operating range $-55$°C to $+125$°C. Multiple units on a single GPIO line for DC/DC and MPPT zones.

**Current Measurement:** ACS712 (30 A variant) — Hall-effect current sensor, analog output to ESP32 ADC, non-invasive measurement of MPPT input/output current.

**Voltage Measurement:** Resistive voltage divider networks scaled to ESP32 ADC range (0–3.3 V) for three voltage measurement points: solar panel, MPPT output, battery.

**GSM Connectivity:** SIM800L (SIMCOM) — UART-connected to ESP32 for remote SMS alerts and GPRS data push. Enables remote fleet monitoring.

**Firmware Framework:** Arduino-ESP32 or ESP-IDF with a cooperative main loop for sensor polling, CAN frame processing, display update, and cellular communication.

**Development Tools:** VS Code + PlatformIO, KiCad for PCB, MATLAB/Python for calibration and data analysis.

## 4.6 Reference Architecture Review

The BAKO Motors Monitoring System architecture is organised around the ESP32 as the central hub, with four dedicated sensing and communication subsystems feeding into it, as illustrated in Figure 1.

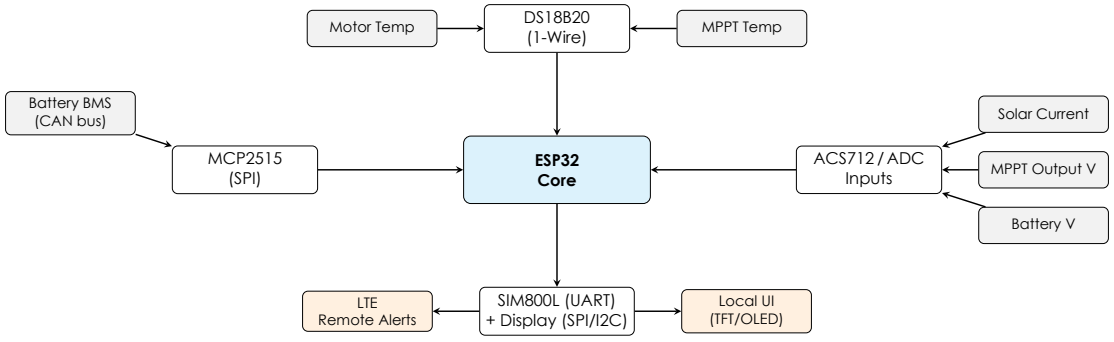



**Figure 1:** BAKO Motors Monitoring System — Reference Architecture

## Reference Designs Reviewed

Three existing reference designs informed this architecture. The ESP32 CAN bus logger (open-source, GitHub) provided the MCP2515-to-ESP32 SPI wiring pattern and interrupt-driven frame reception approach. The MPPT solar charge controller monitoring projects (EPever RS485 loggers) informed the multi-voltage measurement strategy and calibration methodology. The SIM800L ESP32 integration guides (SIMCOM official and community) established the hardware flow control wiring and AT command firmware structure used for cellular communication.

## Key Architecture Decisions

All sensor reads are performed sequentially within a cooperative main loop at defined polling intervals — CAN frames at 10 ms, temperature at 1 s, voltage/current at 500 ms — feeding a shared data structure that the publish and alert logic reads from. This single-threaded design ensures no inter-task synchronization is needed. Alert thresholds (temperature, voltage out-of-range, overcurrent) are evaluated centrally after each polling cycle and trigger cellular SMS dispatch if breached.

# 5 SYSTEMS ARCHITECTURE

## 5.1 System Overview Diagram

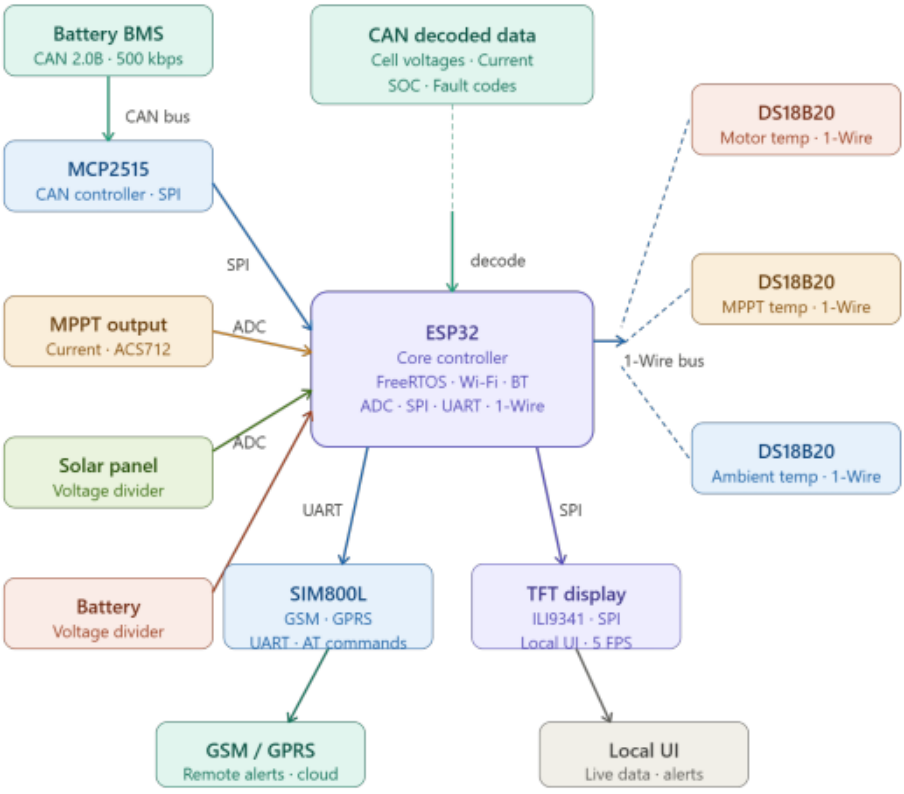



**Figure 2:** ISS system architecture — ESP32 core controller interfacing with the O'CELL BMS via MCP2515 CAN, three DS18B20 temperature sensors (1-Wire), ACS712 current sensors, voltage dividers, SIM800L GSM/GPRS cellular module, and local TFT display.

The system is organised around a central ESP32 hub with five primary data flows: CAN bus communication with the BMS via MCP2515 SPI; DS18B20 digital temperature sensing on motor, MPPT, and DC/DC; ACS712/ACS758 analogue current measurement on solar panel and MPPT output; resistive divider voltage measurement at four nodes; and NPN handbrake detection. Data transmits over two parallel channels: a local Wi-Fi WebSocket server for dashboard access and a cellular LTE/GPRS link to a cloud VPS via FastAPI into SQLite.

## 5.2 Detailed System Block Diagram

Figure 3 presents a detailed hardware and software block diagram of the full ISS system, from the vehicle sensors and BMS through the ISS PCB internals to the cloud backend and browser dashboard.

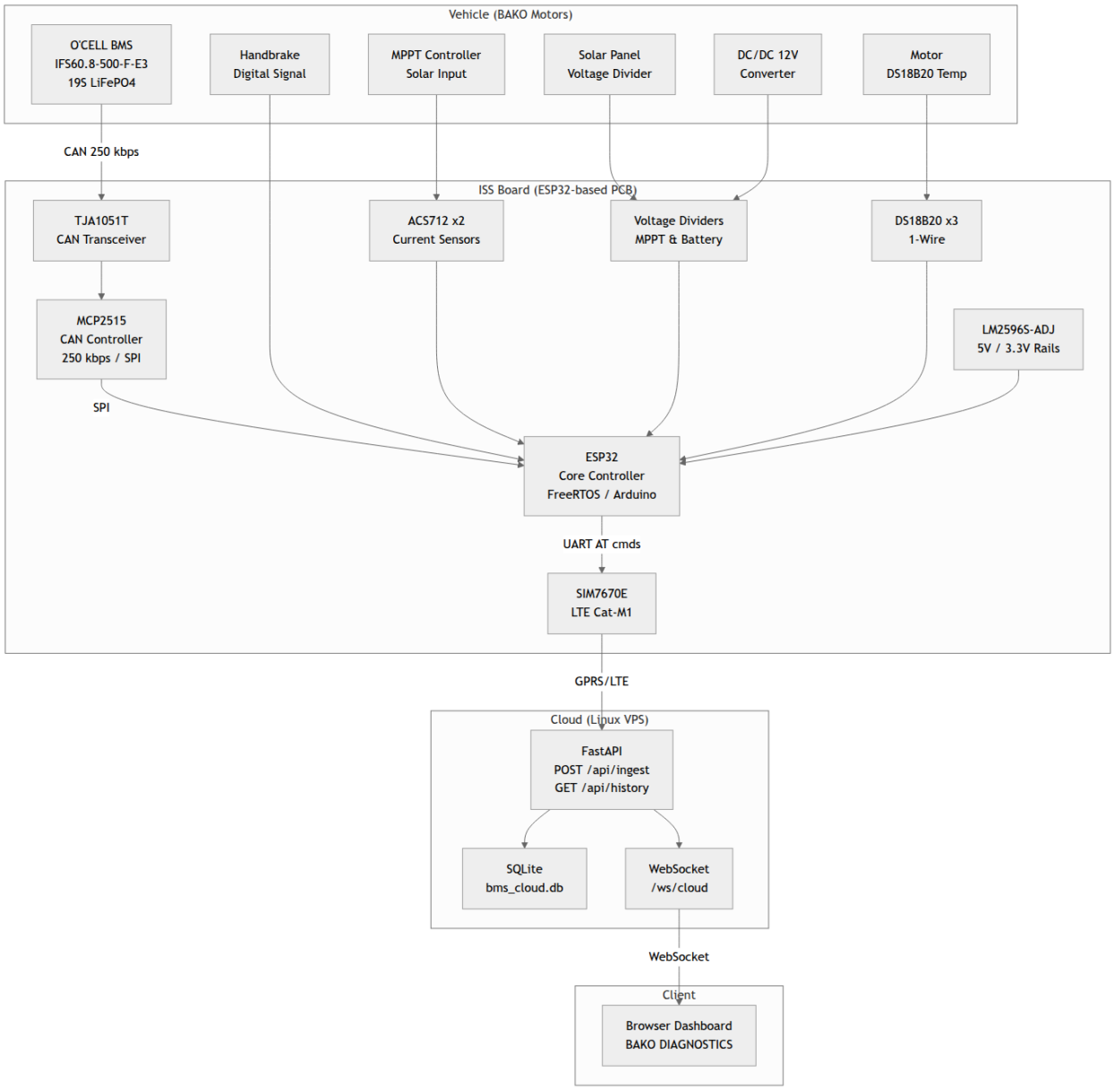



**Figure 3:** Detailed ISS system block diagram showing all hardware subsystems (vehicle layer), ISS board internals (ESP32 peripherals and power regulation), cloud pipeline (FastAPI, SQLite, WebSocket), and browser client.

## 5.3 Electrical & Power Systems

The electrical and power subsystem is responsible for energy distribution, voltage regulation, and real-time monitoring of the vehicle's electrical parameters. The system integrates the battery pack, solar charging subsystem, MPPT controller, DC/DC converter, and embedded monitoring electronics.

The battery pack acts as the main energy source of the system [1] and is supervised by the Battery Management System (BMS) [2]. Communication with the BMS is established through the CAN Bus protocol [3] using the MCP2515 CAN controller. The monitoring unit acquires important battery parameters such as pack voltage, current, state of charge, cell voltages, battery temperatures, and fault status information.

A solar energy subsystem is integrated to support power generation. The solar panel is connected to an MPPT (Maximum Power Point Tracking) controller, which optimises energy extraction efficiency. Current and voltage measurements are performed before and after the MPPT stage in order to evaluate charging performance and monitor power flow within the system.

The system also includes a DC/DC converter responsible for regulating voltage levels for low-power electronic components. Both input and output voltages of the converter are monitored to ensure stable operation and proper voltage regulation.

Electrical measurements are performed using ACS712 and ACS758 current sensors together with voltage sensing modules based on analogue voltage division. In addition, DS18B20 digital temperature sensors are installed on critical components such as the MPPT heatsink and DC/DC converter to supervise thermal behaviour and prevent overheating conditions.

The ESP32 microcontroller serves as the central processing and acquisition unit. It periodically collects sensor measurements and transmits the data through communication interfaces including CAN Bus, Wi-Fi, Bluetooth BLE, and GSM/GPRS connectivity.

## 5.4 Electronic Control Units (ECUs)

The monitoring platform integrates several Electronic Control Units (ECUs) responsible for data acquisition, communication, and system supervision. These units ensure reliable real-time monitoring of the vehicle's electrical and thermal parameters.

The main control unit of the system is the ESP32 microcontroller. It collects sensor measurements, decodes CAN Bus messages, processes data, and transmits information to the monitoring dashboard. Its integrated Wi-Fi and Bluetooth capabilities make it suitable for embedded IoT applications.

The Battery Management System (BMS) supervises the battery pack by monitoring parameters such as voltage, current, temperature, state of charge (SOC), and fault conditions. Com-

munication between the BMS and the ESP32 is achieved through the CAN Bus network using the MCP2515 CAN controller module.

Additional sensors are used to monitor the MPPT controller, DC/DC converter, solar panel, and motor temperature [4]. These measurements are periodically processed and transmitted by the ESP32.

During development, the ESP32 was also used as a firmware simulation platform to generate simulated sensor data and CAN frames. This allowed testing the dashboard and communication interfaces without continuously connecting the system to the vehicle, simplifying debugging and software validation.

Overall, the ECU architecture provides efficient monitoring, communication, and diagnostic capabilities for the vehicle monitoring platform.

## 5.5 Embedded Software Architecture

The embedded software running on the ESP32 microcontroller is structured as a **single-threaded, cooperative loop** built on the Arduino framework. This architecture was chosen for its simplicity and determinism: all BMS-related tasks execute sequentially on one core, eliminating the need for inter-task synchronization.

The software is divided into five logical layers, shown in Figure 4:

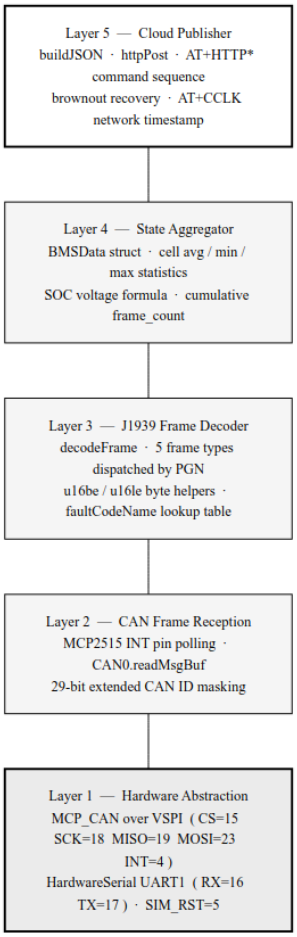



**Figure 4:** ESP32 embedded software layer diagram

**Hardware Abstraction Layer (HAL)** Provides direct access to the MCP2515 via SPI (`mcp_can` library) and to the SIM800L via HardwareSerial UART1. Pin assignments and bus parameters (CS, INT, baud rate, crystal frequency) are defined as constants and isolated from the application logic.

**CAN Frame Reception** The MCP2515 asserts the `INT` line (GPIO 4) when a frame is placed in its receive buffer. The main loop polls this line and calls `CAN0.readMsgBuf()` to drain all pending frames. Extended CAN IDs are masked from the 29-bit field before processing.

**J1939 Decoder** The `decodeFrame()` function dispatches each received frame by inspecting the `FUNC` byte of the CAN ID (bits 23–16). It handles five frame types (cell voltages, temperatures, BMS Basic 1 & 2, charging request) and writes decoded values into the `BMSData` struct.

**State Aggregator** The `BMSData` struct accumulates all decoded fields and maintains derived statistics (cell average, min, max, voltage-based SOC). The struct is updated in place and always holds the most recent complete state of the battery.

**Cloud Publisher** Every 5 seconds, `buildJSON()` serializes `BMSData` into a JSON document using the ArduinoJson library, and `httpPost()` delivers it to the VPS via the SIM800L AT+HTTP* command set.

### 5.5.1 Key Design Decisions

**No RTOS.** A FreeRTOS task structure was considered but rejected because the single-responsibility loop handles all timing requirements naturally: CAN frames arrive at 100–500 ms intervals, and the publish cycle runs every 5 seconds. Adding preemptive scheduling would have introduced stack management and mutex overhead without any functional benefit.

**CAN drain during HTTP wait.** The `AT+HTTPACTION` response can take up to 30 seconds over a cellular link. To prevent the MCP2515 receive buffer (6 frames deep) from overflowing during this window, the HTTP wait loop polls the CAN `INT` pin and drains frames continuously, calling `decodeFrame()` in-place. This ensures the `BMSData` struct is fully up-to-date when the next publish cycle begins.

**HardwareSerial over SoftwareSerial.** SoftwareSerial operates via bit-banging and is sensitive to interrupt latency. The SIM800L produces multi-line AT response bursts that SoftwareSerial occasionally drops bytes from at 9600 baud on a loaded core. HardwareSerial UART1 uses dedicated silicon and a hardware FIFO, making it robust under all operating conditions.

## 5.6 Cloud & Connectivity Architecture

The cloud and connectivity architecture connects the on-vehicle ESP32 to a remote browser with minimal intermediaries. The full pipeline is shown in Figure 5.

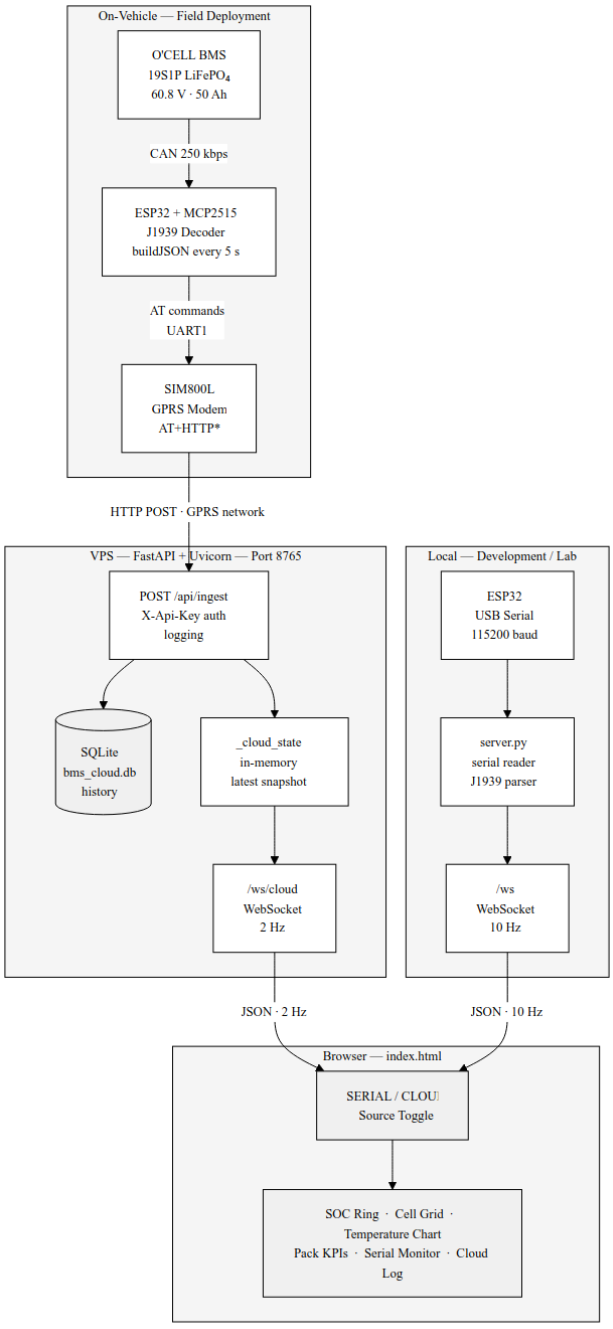



**Figure 5:** End-to-end cloud connectivity architecture

### 5.6.1 Component Roles

**ESP32 + MCP2515 (On-vehicle edge node)** Reads and decodes SAE J1939 CAN frames from the BMS at up to 10 frames per second, aggregates them into a snapshot, and transmits one JSON document to the VPS every 5 seconds.

**SIM800L GSM/GPRS Module** Provides wide-area cellular connectivity using the carrier GPRS network. Communicates with the ESP32 via AT commands over UART. Handles TCP/IP, HTTP, and DNS entirely on-chip using the AT+HTTP* service, so the ESP32 only needs to manage the AT command dialogue.

**VPS Backend (FastAPI + Uvicorn)** Acts as the single ingestion point and real-time relay for all cloud data. Receives POST requests from the ESP32, stores them to SQLite for persistence, and maintains an in-memory state that is pushed to connected browser clients via WebSocket at 2 Hz. The server also handles the SERIAL mode WebSocket, allowing the same dashboard to be used with a direct USB connection.

**SQLite Database** Provides persistent local storage for all received snapshots. Enables historical data retrieval via the `/api/history` endpoint without any external database service.

**Browser Dashboard** A single-page web application that renders live battery data received over WebSocket. Supports SERIAL mode (direct USB, 10 Hz) and CLOUD mode (VPS relay, 2 Hz). Auto-reconnects on network interruption.

### 5.6.2 Data Flow Summary

1. O'CELL BMS → CAN bus (250 kbps, SAE J1939)
2. ESP32 + MCP2515 → decode all frames → `BMSData` struct
3. ESP32 → `buildJSON()` → JSON document
4. ESP32 → SIM800L AT+HTTPACTION=1 → `POST /api/ingest` (HTTP, LTE)
5. VPS server → validate API key → SQLite insert + in-memory update
6. VPS server → `/ws/cloud` WebSocket push every 2 s
7. Browser → render panels (SOC, cells, temperatures, faults, cloud log)

### 5.6.3 Dual-Mode Operation

A key architectural feature is that the dashboard supports two independent data sources via the same frontend interface. In **SERIAL mode**, the backend reads raw CAN log lines from the ESP32 USB serial port, decodes them through its own Python J1939 parser (`BMSState.decode()`), and pushes the result at 10 Hz. In **CLOUD mode**, the backend serves the latest snapshot received from the ESP32 over GPRS.

This dual-mode design provides two practical benefits:

- During development and testing, the system can be operated entirely over USB without requiring a SIM card or GPRS connection.
- In the field, the dashboard can be opened from any web browser pointed at the VPS IP address, with no local software to install.

## 5.7 3D Mechanical Architecture

The 3D mechanical architecture of the ISS board is described in Chapter 7 (Mechanical Design). The two-layer PCB is dimensioned to fit within a standard DIN-rail enclosure, with all external connectors (CAN bus, sensor headers, power input) placed along the board perimeter for clean harness routing. A 3D render of the board is shown in Figure 13.

## 5.8 Safety & Redundancy Design

### Hardware Protection

Reverse polarity protection is implemented at the main power input using diode D6, which prevents damage to all downstream circuitry in the event of an incorrect supply connection. Overcurrent protection is provided by fuses F1, F2, and F3, placed on the main power rail and the sensor supply lines respectively, limiting fault currents before they can damage components. Zener diodes D3, D4, and D7 provide ESD and overvoltage clamping on the CAN bus lines, protecting the MCP2515 and TJA1051T transceiver from voltage spikes induced by the vehicle's electrical environment.

### Software Redundancy

A hardware watchdog timer ensures automatic recovery from firmware hangs by resetting the ESP32 microcontroller without requiring manual intervention. The MCP2515 CAN error counter is monitored continuously; if the error count exceeds the bus-off threshold — which can occur after a transient disconnection or excessive noise on the CAN bus — the driver automatically reinitialises the controller and resumes normal frame reception. All ADC input readings are subject to sanity-check bounds validation before being stored in the telemetry structure, discarding implausible values caused by transient noise or sensor disconnection. Threshold violation alerts for over-temperature, over-voltage, and over-current conditions are transmitted via SMS through the SIM800L module, enabling remote notification of critical fault events without requiring an active internet connection.

# 6 PREPARATION PHASE

## 6.1 Project Governance & Team Structure

The project is supervised academically by **Dr. Mortadha Dahmani** and conducted in direct collaboration with **BAKO Motors** as the industrial partner. The development team consists of seven members: Mahdi Hachicha, Mohamed Baccouche, Yassine Mtibaa, Hana Gdoura, Selima Grissa, Fares Smaoui, and Khadija Ben Hamida. Tasks are distributed across five functional areas: hardware design and PCB, embedded firmware, dashboard and cloud back-end, documentation and reporting, and integration testing. Version control is managed through Git with a shared private repository; task tracking and sprint planning follow a product backlog structure aligned with the Agile methodology described in Chapter 13.

## 6.2 Development Environment Setup

The embedded firmware is developed using **VS Code** with the **PlatformIO** [5] extension, targeting the ESP32 microcontroller with the Arduino-ESP32 framework. This toolchain provides dependency management, cross-compilation, and integrated serial monitoring in a single environment. PCB schematic capture and layout are performed in **KiCad 10.0** [6]–[8]. The back-end server and Python sensor emulator are developed in **Python 3.11**. The cloud database runs on **SQLite** hosted on a Linux VPS, managed directly via the FastAPI server. All source code, PCB design files, and documentation are maintained in a private Git repository shared across the full team [9]. Component datasheets were retrieved from manufacturer websites and `alldatasheet.com` [10]; tutorial resources were drawn from YouTube [11] and Google Scholar [12]. Technical writing assistance used Claude AI [13].

## 6.3 Simulation & Prototyping Environment

A Python-based sensor emulator was developed to simulate the complete vehicle sensor network, generating realistic values for 12 sensors with smooth, continuous transitions between states. The emulator cycles through seven test scenarios every 30 seconds:

- **Normal Driving** — typical operational parameters across all sensors.
- **Low Battery** — state of charge below 20%, reduced motor performance.
- **Charging Mode** — MPPT active, battery current positive.
- **High Solar Input** — peak solar irradiance, maximum MPPT throughput.

- **Night Mode** — no solar input, battery discharging only.
- **Handbrake Fault** — handbrake sensor engaged while vehicle is moving.
- **Overheat Condition** — motor and DC/DC converter temperatures above threshold.

The emulator outputs structured JSON payloads compatible with the back-end API, fully replacing the physical ESP32 for end-to-end testing. In addition, a breadboard prototype was assembled early in the project to validate CAN bus wiring with the MCP2515, DS18B20 one-wire reading, and voltage divider scaling before committing to PCB layout.

## 6.4 Risk Register (Initial)

Initial risks identified at project outset included: the proprietary O'CELL BMS CAN protocol requiring reverse engineering before frame decoding could begin; ADC accuracy limitations on the ESP32 requiring dedicated calibration routines for precise voltage and current readings; component availability in Tunisia for specialised modules such as the SIM800L; LTE network compatibility of the chosen cellular module with Tunisian operators (Ooredoo and Orange); and PCB fabrication lead times from local and regional manufacturers. Each risk was assigned a likelihood rating, an impact severity level, and a set of mitigation strategies as part of the project governance framework. The full expanded risk register is presented in Chapter 18.

## 6.5 Procurement & Vendor Selection

All components were sourced through a two-channel strategy to balance cost, lead time, and availability. The ESP32-DEVKITC, MCP2515 SPI module, DS18B20 temperature sensors, ACS712 current sensors, passive components, and connectors were sourced locally in Tunisia from electronics distributors, keeping procurement costs low and eliminating import lead times. The SIM800L GSM/GPRS module was ordered through an online distributor with confirmed stock, given its unavailability in local markets. The PCB is specified for fabrication in FR-4, two copper layers, 1.6 mm board thickness, 35 µm copper weight per layer, using standard KiCad Gerber RS-274X export for submission to the fabrication house.

## 6.6 Prototype Planning

The prototype was developed through five sequential phases, each building on validated hardware from the previous stage.

**Phase P1 — Breadboard Proof of Concept (Weeks 3–6).** Individual subsystems were validated in isolation on a breadboard. A single MCP2515 module was wired to an ESP32 via SPI to

confirm CAN frame reception from the O'CELL BMS. One DS18B20 sensor was read over 1-Wire and its output printed to the serial monitor. An ACS712 current sensor output was sampled through the ESP32 ADC and calibrated against a bench power supply. Voltage divider networks were assembled and the scaling coefficients verified with a multimeter. Success criterion: all four subsystems operational and producing plausible readings independently.

**Phase P2 — Integrated Breadboard (Weeks 7–9).** All validated subsystems were connected to a single ESP32 simultaneously. A cooperative main loop was implemented to sequentially poll CAN, 1-Wire, and ADC inputs. The SIM800L GSM/GPRS module was connected via UART and an SMS alert was successfully dispatched. A local display (TFT LCD) was added to show live sensor data without a connected computer. Success criterion: all subsystems operational together without starvation over a 30-minute run.

**Phase P3 — Semi-Permanent Stripboard Build (Week 10).** The integrated breadboard design was transferred to stripboard for mechanical stability during field testing at the BAKO Motors workshop. Wiring was shortened and secured, and connectors were added for the CAN bus, temperature sensors, and power supply. This build was used for the first on-vehicle test with the actual O'CELL BMS CAN bus. Success criterion: reliable operation during vehicle movement with no loose connections or intermittent failures.

**Phase P4 — Custom PCB Prototype (Weeks 10–13).** Lessons learned from the stripboard build were incorporated into a KiCad schematic and PCB layout. The two-layer FR-4 board (Revision RECTIFICATION_2) includes functional zoning for power, digital, CAN, and sensor domains, ESD protection Zener diodes on CAN lines, and fuse protection on all supply rails. Three MCP2515 SPI module headers support multi-bus configurations. The board was submitted for fabrication and populated by the team. Success criterion: board powers up correctly; all subsystems reachable from the ESP32 without errors.

**Phase P5 — Integration Testing (Week 14).** The fully populated PCB was connected to the vehicle harness and the cloud back-end. All sensor data paths were verified end-to-end: CAN frames decoded and displayed on the dashboard, temperature and current readings visible in the browser, and cloud telemetry appearing in the SQLite database on the VPS. The Python emulator was run in parallel to validate that the back-end API processed both hardware and emulated data identically.

**Quantitative success criteria applied across all phases:**

- CAN frame reception rate $\geq 99\%$ over a 60-minute continuous test.
- DS18B20 temperature accuracy within $\pm 1$°C of a reference thermometer.
- ACS712 current measurement error $< 2\%$ of full scale.
- Voltage divider readings within $\pm 1\%$ of the multimeter reference.

- Cloud telemetry delivered to the VPS database with no snapshot loss over a 1-hour test.

# 7 CIRCUIT DESIGN & ELECTRONICS

## 7.1 Components Selection & Justification

Every electronic component for this project was chosen through a structured evaluation against four consistent criteria: technical fit with the system architecture, library and toolchain support, compatibility with Tunisian operator networks where applicable, and local availability in Tunisia together with unit cost.

**Microcontroller — ESP32-DEVKITC.** The MCU is the brain of the ISS board. It must handle CAN data acquisition, sensor reading, LTE connectivity, and real-time telemetry transmission simultaneously. The ESP32 integrates Wi-Fi, Bluetooth, and a dedicated UART for the LTE module in a single chip, removing the need for extra communication modules [14]. Its compatibility with the Arduino IDE and the large IoT library ecosystem significantly accelerated development. The WROOM module is widely available locally in Tunisia and costs approximately 30 TND, making it the most practical and cost-effective choice.

**CAN Controller — MCP2515.** The CAN controller acts as the interface between the ESP32 and the vehicle CAN bus. Three options were evaluated: the ESP32's built-in TWAI peripheral, the legacy SJA1000, and the MCP2515. Although the ESP32 includes a built-in CAN peripheral, it is only accessible through the ESP-IDF framework, making it significantly harder to integrate with Arduino-style libraries. The MCP2515 communicates over SPI, keeping wiring simple and reliable [15]. Its well-documented `mcp_can` library and local availability in Tunisia made it the most practical choice. Three MCP2515 SPI module headers (J17, J18, J19) are present on the board to support multi-bus configurations.

**CAN Transceiver — TJA1051T.** The CAN transceiver translates the digital logic signals from the MCP2515 into the differential voltage levels required on the physical CAN bus [16]. The TJA1051T is an automotive-grade transceiver that operates at 5 V, directly matching the MCP2515 supply rail [17]. It provides robust bus fault protection suited for the electrically noisy environment of a vehicle. It is the standard companion chip to the MCP2515 in OBD and automotive data-logging applications, ensuring long-term reliability.

**Cellular Module.** The SIM800L relies on 2G connectivity, which is being progressively phased out by Tunisian operators, making it a short-lived solution. The SIM7670E supports 4G LTE Cat-M1 and NB-IoT, is confirmed compatible with both Ooredoo and Orange networks in Tunisia, and remains future-proof as the country's network infrastructure continues to evolve. Its low operating cost further

justified this selection. The KiCad schematic uses the SIM800L-CORE footprint as a physical layout placeholder; the actually populated component is the SIM7670E as noted in Section 7.4.

**Power Regulation.** The board is powered from the vehicle's 12 V rail. The LM2596S-ADJ is a switching buck regulator capable of delivering up to 3 A with approximately 90% efficiency, producing minimal heat compared to a linear regulator [18]. Its adjustable output (1.25 V to 37 V) allows the same component to supply both the 3.3 V ESP32 circuit and a 5 V rail. The board also carries an AMS1117 (V3.3) and an LM7805 (V5) for localised regulation where switching noise is undesirable.

## 7.2 Circuit Schematic Description

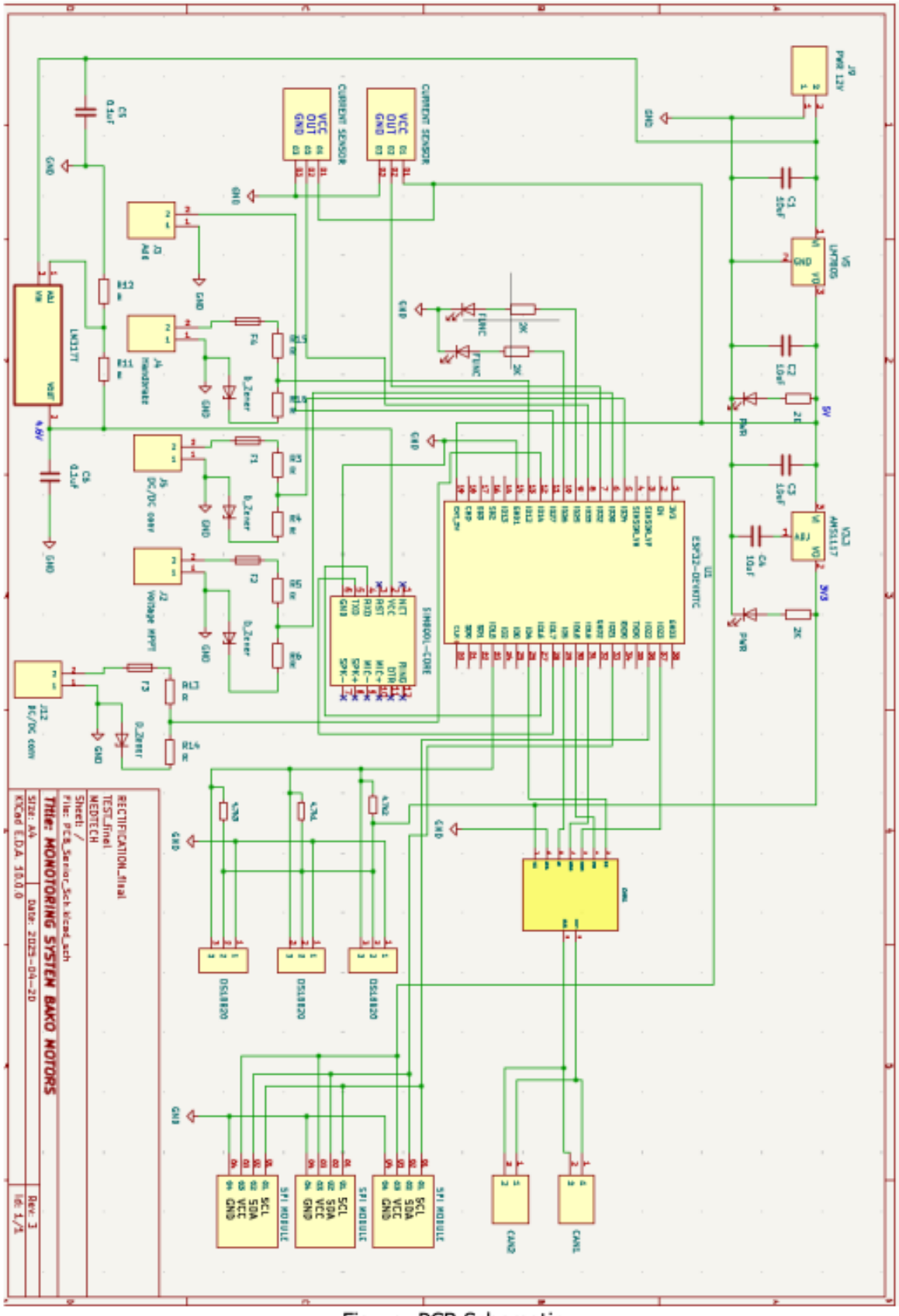



Figure: PCB Schematic

The complete schematic is organised into five functional zones visible across the sheet: the ESP32-DEVKITC and SIM7670E communication block in the upper centre, the MCP2515 CAN subsystem at the left, the DS18B20 1-Wire temperature sensor cluster at the lower left, the ACS712 analogue current sensor inputs at the lower right, and the power regulation and protection circuitry spanning the right edge. All inter-block signal paths are labelled with net names for traceability during PCB layout and debugging.

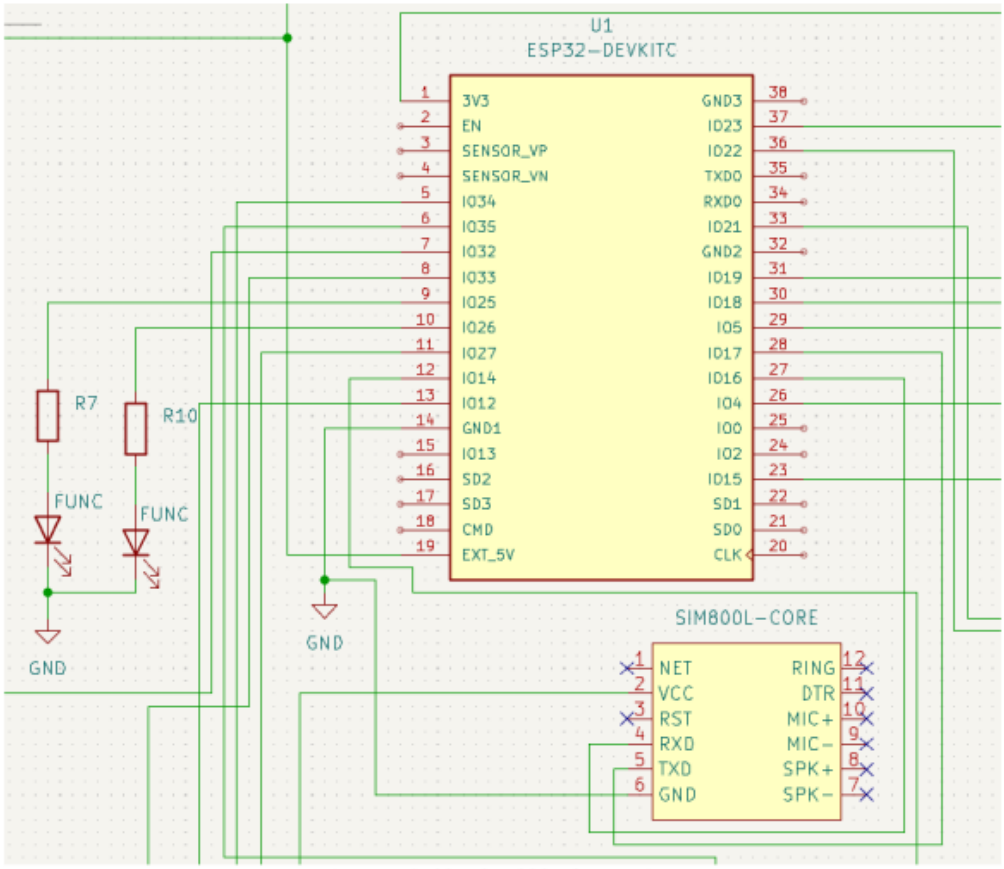


Figure: ESP32 +SIM800L Connections

**Figure 7:** ESP32-DEVKITC and SIM7670E cellular module connections: UART, level-shift resistors R7–R10, and status LEDs.

The ESP32 UART1 (GPIO 16/17) connects to the SIM7670E module through resistors R7–R10, which form a 2 kΩ voltage divider on each line to shift the 3.3 V ESP32 logic down to the 1.8 V interface level required by the SIM7670E. LEDs D2 and D5, connected through current-limiting resistors, provide visual indication of firmware operating state: D2 pulses on each CAN frame received, and D5 lights during an active GPRS transmission.

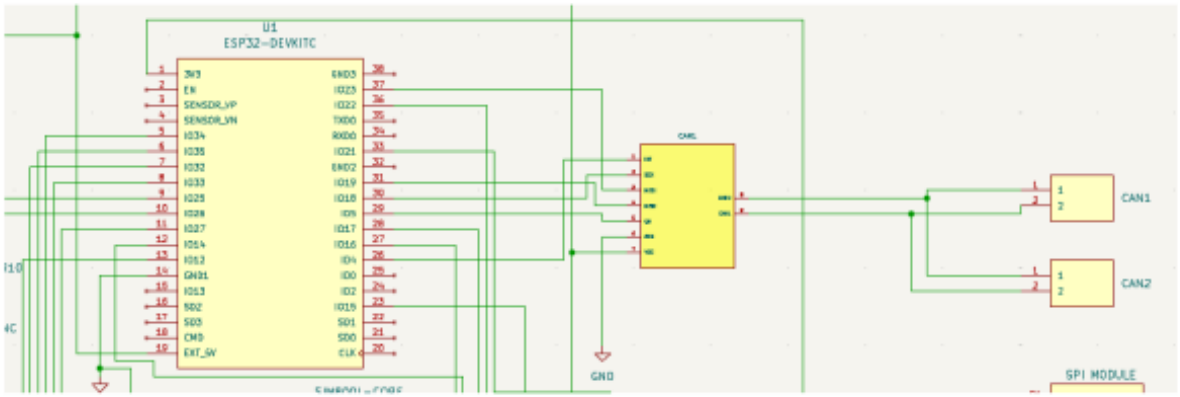



Figure: MCP2515+ ESP32 Connections

**Figure 8:** MCP2515 SPI CAN controller and TJA1051T transceiver connection detail including ESD Zener diodes D3, D4, D7.

The MCP2515 SPI module is connected to the ESP32 VSPI bus (SCK, MISO, MOSI) with a dedicated Chip Select line, allowing up to three independent CAN controllers to share the same SPI bus using separate CS signals. The TJA1051T transceiver converts the 3.3 V MCP2515 logic into the ±2 V differential CAN_H/CAN_L bus signals required by the ISO 11898 physical layer standard. Zener diodes D3, D4, and D7 clamp any transient voltage spike on the CAN bus lines to safe levels before they reach the transceiver input pins.

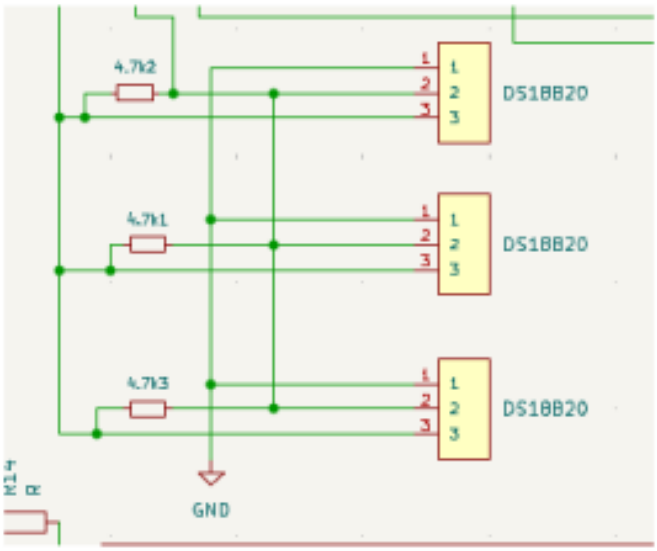



Figure: DS18B20 COnnections

**Figure 9:** Three DS18B20 temperature sensor connections (J6, J7, J8) with 4.7 kΩ 1-Wire pull-up resistors.

All three DS18B20 sensors share a single 1-Wire data line, which is pulled up to 3.3 V through

one of the 4.7 kΩ resistors. Each sensor is factory-programmed with a unique 64-bit ROM address, allowing the ESP32 firmware to address them individually on the shared bus using the `DallasTemperature` library without any additional hardware. The three connectors (J6, J7, J8) are positioned at the board periphery so that the sensor cables can exit directly towards the motor, MPPT heatsink, and DC/DC converter mounting points.

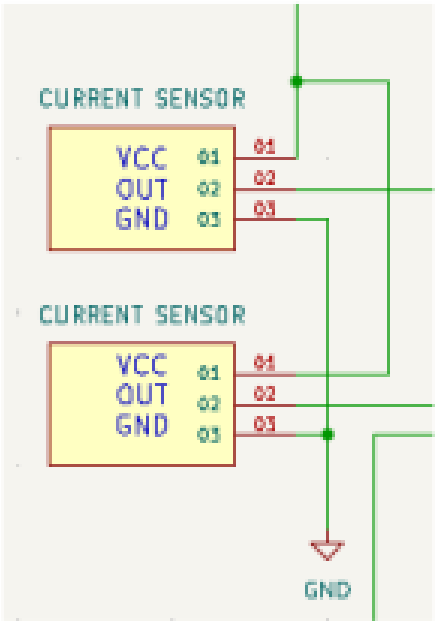



**Figure 10:** Current sensor inputs: two ACS712 modules connected at J11 and J16 with analogue signal conditioning resistors.

Each ACS712 module produces an analogue output voltage centred at $V_{CC}/2$ (1.65 V) at zero current, shifting linearly at 66 mV/A for the 30 A variant used here. The signal conditioning resistors R3–R6 and R11–R14 provide a low-pass filtering and biasing network that suppresses high-frequency switching noise from the MPPT controller before the signal reaches the ESP32 ADC input, ensuring accurate current readings under electrically noisy vehicle conditions.

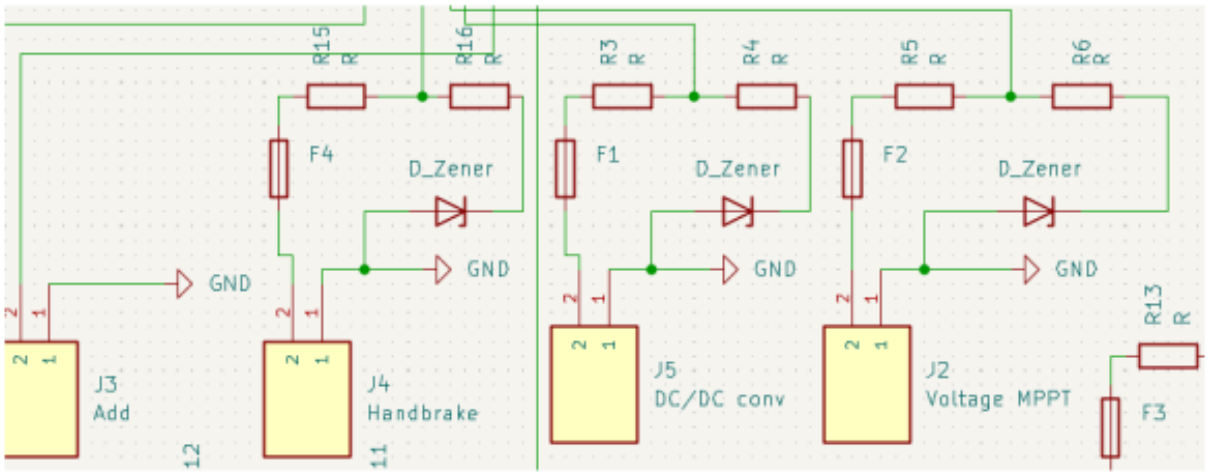


Figure: Voltage dividers

**Figure 11:** Voltage monitoring and auxiliary connectors: MPPT voltage divider (J2), DC/DC converter (J5), handbrake input (J4), auxiliary expansion (J3), with Zener clamp diodes F1–F4 and signal conditioning resistors.

The MPPT output voltage, which can reach up to 50 V, is scaled down to the ESP32 ADC safe input range (0–3.3 V) using a resistive voltage divider. The Zener diodes F1–F4 clamp the ADC input line to prevent any overvoltage transient from damaging the ESP32's internal analogue circuitry. The handbrake connector J4 routes the digital handbrake signal through a pull-up resistor to a standard digital GPIO input, and the auxiliary connector J3 provides four uncommitted GPIO-connected pads for future sensor expansion.

The circuit schematic represents the logical connections between all electronic components before they are translated into physical copper traces on the PCB. The design is organised around five functional blocks.

The **ESP32-DEVKITC** (U1) is the central processing unit. It interfaces with the SIM7670E cellular module (U2, using the SIM800L-CORE footprint) over UART for LTE telemetry. The 2 kΩ resistors R7–R10 serve as UART voltage-level dividers between the 3.3 V ESP32 and the module's interface pins, protecting both ICs from logic-level mismatches. Functional status LEDs D2 and D5 indicate active operation states.

The **CAN subsystem** connects the MCP2515 SPI module to the ESP32 via SPI (Chip Select, MOSI, MISO, SCK lines), with individual chip-select pins allowing independent addressing of each CAN channel. The physical CAN layer is handled by the TJA1051T transceiver (CAN1 footprint), which interfaces with the primary CAN connector J1. A secondary CAN connector J10 provides a redundant bus connection. Zener diodes D3, D4, and D7 provide ESD and overvoltage clamping on the CAN bus lines. The 4.7 kΩ resistors (4.7k1, 4.7k2, 4.7k3) serve as bus pull-ups.

The **temperature sensing subsystem** accommodates three DS18B20 digital sensors connected via 1-Wire protocol through connectors J6, J7, and J8. The 4.7 kΩ resistors provide the mandatory

pull-up on the 1-Wire data line.

The **analogue sensor subsystem** routes two current sensor inputs through J11 and J16, with signal conditioning resistors R3–R6 and R11–R14 adapting the sensor output levels to the ESP32 ADC input range. Voltage monitoring from the MPPT controller is received through J2. The handbrake digital signal is acquired through J4, providing vehicle state context for the supervision logic. Auxiliary expansion is available via connector J3.

The **protection layer** is integrated throughout the schematic. Reverse polarity protection is implemented at the main power input using diode D6. Overvoltage and ESD protection on the CAN bus is provided by Zener diodes D3, D4, and D7. Fuses F1, F2, and F3 provide overcurrent protection for the main power rail, sensor supplies, and auxiliary circuits respectively. The resistor network throughout the board ensures correct voltage levels at every inter-IC interface, preventing damage from logic-level mismatches.

## 7.3 PCB Layout & Design

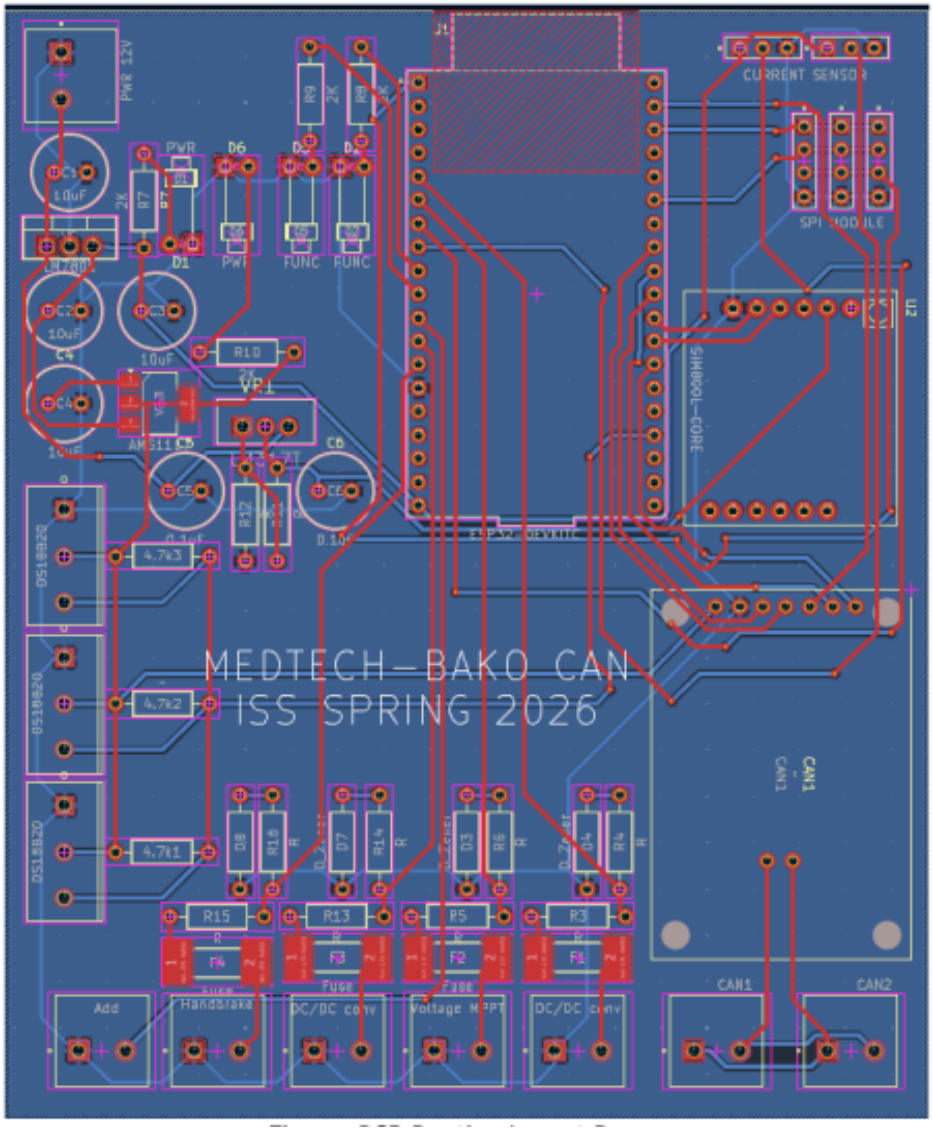



**Figure 12:** ISS board PCB routing layout (KiCad PCBnew, Rev. RECTIFICATION_2) showing F.Cu signal traces and B.Cu ground plane.

The routing view shows all copper traces on the F.Cu (signal) layer in red and the B.Cu (ground plane) in blue. Short, direct routing of the SPI traces between the ESP32 and the MCP2515 modules minimises propagation delay and parasitic capacitance on the high-speed clock line (up to 10 MHz). Power traces carrying the 12 V input and 5 V CAN supply rails are given twice the width of signal traces to handle the required current without excessive resistive voltage drop.

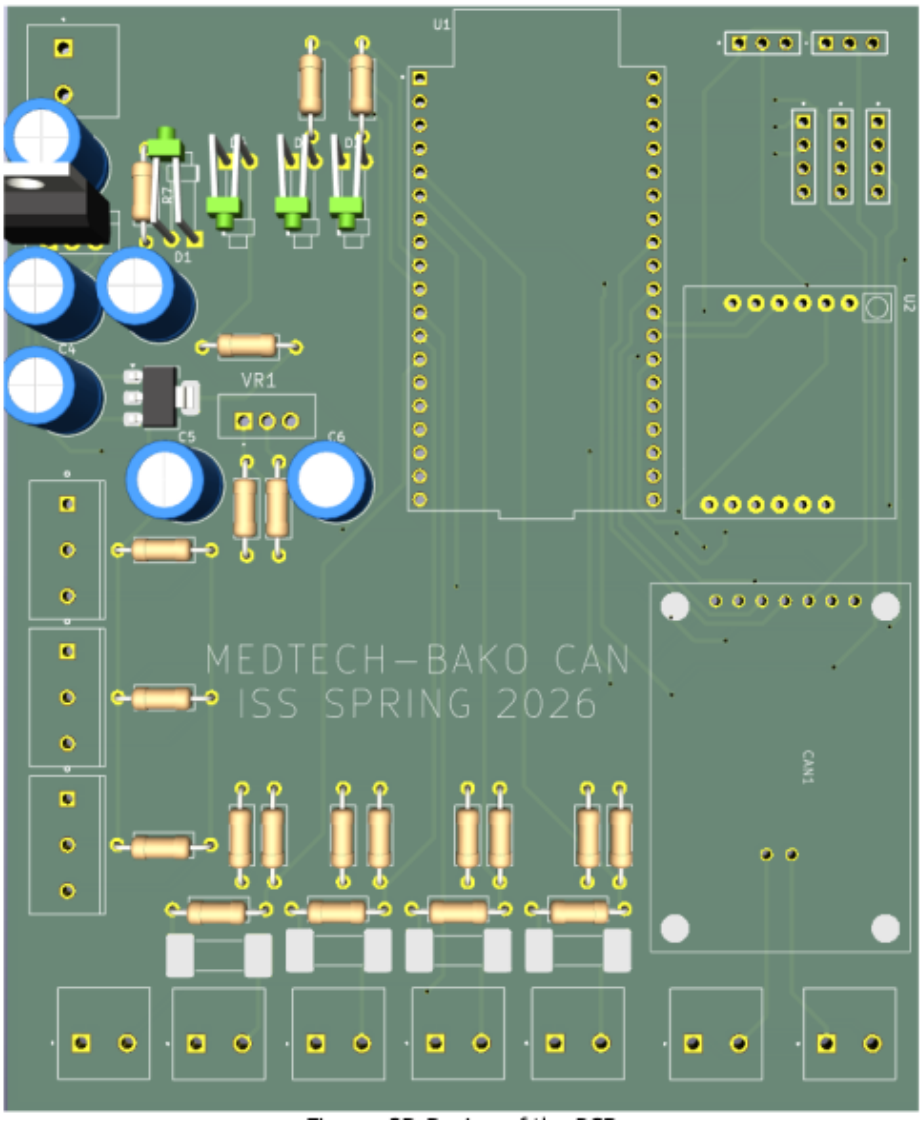



**Figure 13:** 3D render of the ISS board (KiCad 3D viewer) showing component placement and board outline.

The 3D render confirms correct physical clearances between tall components (electrolytic capacitors, SIM7670E module, ESP32 devkit) and the board outline. The functional zoning is visible in the render: the power block occupies the top-right corner, the ESP32 and SIM7670E module occupy the board centre, and the sensor connectors are distributed along the three remaining edges for clean harness routing. The board outline dimensions are compatible with a standard 100 mm × 100 mm DIN-rail enclosure footprint.

The PCB was designed using KiCad 10.0 PCBnew. Table 3 summarises the key mechanical and electrical parameters of the board as extracted from the KiCad project file.

**Table 3:** PCB Mechanical and Electrical Parameters

| Parameter | Value |
|---|---|
| **Board title** | MONITORING SYSTEM BAKO MOTORS |
| **Design revision** | RECTIFICATION_2 (06 April 2026) |
| **Layer count** | 2 layers (F.Cu + B.Cu) |
| **Board thickness** | 1.6 mm |
| **Copper thickness** | 35 µm (1 oz) per layer |
| **Substrate material** | FR-4 ($\varepsilon_r = 4.5$, loss tangent = 0.02) |
| **Solder mask** | Both sides (front + back), 10 µm |
| **Silkscreen** | Front + Back |
| **Min. track/spacing** | Per KiCad DRC defaults |
| **Via tenting** | Both sides |
| **Design tool** | KiCad 10.0 / PCBnew |

With the two-layer stackup, the front copper layer (F.Cu) carries the majority of signal traces and component connections. The back copper layer (B.Cu) is used for the ground plane pour and power return paths, reducing electromagnetic interference and providing a low-impedance ground reference for all circuits. Critical high-frequency SPI traces connecting the ESP32 to the MCP2515 modules are kept short and routed on F.Cu to minimise parasitic inductance. Power traces carrying the 12 V input, 5 V CAN rail, and 3.3 V logic rail are given increased width to handle the required current without excessive resistive heating. Sensor signal traces, particularly the 1-Wire bus and analogue current-sense lines, are routed away from the switching regulator to avoid noise coupling.

The component placement follows a functional zoning approach to minimise cross-interference between the power, digital, and analogue sections of the board:

- **Power zone:** The main power input connector (J9), fuses (F1–F3), LM2596S-ADJ (VR1), LM7805 (V5), and AMS1117 (V3.3) are grouped together to contain high-current paths and facilitate efficient heat dissipation.
- **Digital processing zone:** The ESP32-DEVKITC (U1) and SIM7670E cellular module (U2) are placed centrally, minimising trace lengths to the SPI, UART, and GPIO peripherals.
- **CAN bus zone:** The MCP2515 SPI modules (J17–J19), TJA1051T transceiver, and CAN connectors (J1, J10) are grouped together at one edge of the board, simplifying external wiring harness routing to the vehicle OBD port [19].
- **Sensor zone:** The DS18B20 connectors (J6–J8), current sensor inputs (J11, J16), and MPPT voltage input (J2) are placed at the board periphery, allowing sensor cables to exit cleanly without routing over the digital core.

The design was verified using KiCad's Design Rule Check (DRC) prior to export. Pad-to-mask clearance is set to 0 mm (controlled by the manufacturer's solder mask process). Via tenting is ap-

plied on both the front and back layers to prevent solder bridges and improve board reliability. The silk screen layers carry component reference designators and polarity markers to facilitate assembly and inspection. The board is ready for submission to a PCB fabrication house using standard Gerber RS-274X export from KiCad.

## 7.4 Full Components Inventory

Tables 4 and 5 list all components populated on the ISS board as extracted from the KiCad PCB file (test2.kicad_pcb, Rev. RECTIFICATION_2). References, values, quantities, and functional roles are provided for each entry.

**Table 4:** Full Component Inventory — Integrated Circuits and Connectors

| Ref. | Component / Value | Qty | Function |
|---|---|---|---|
| **U1** | ESP32-DEVKITC | 1 | Main MCU — Wi-Fi, BT, UART to SIM7670E |
| **U2** | SIM800L-CORE* | 1 | LTE cellular module (SIM7670E footprint) |
| **V3.3** | AMS1117 (3.3 V) | 1 | 3.3 V LDO for ESP32 I/O |
| **V5** | LM7805 | 1 | 5 V linear reg. for CAN side |
| **VR1** | LM317T | 1 | Adjustable voltage ref. / LM2596S-ADJ compatible |
| **J1** | CAN1 connector | 1 | Primary CAN bus interface |
| **J10** | CAN2 connector | 1 | Secondary / redundant CAN bus |
| **J19/J17/J18** | SPI MODULE | 3 | MCP2515 CAN controller via SPI |
| **J11/J16** | CURRENT SENSOR | 2 | Analogue current-sense inputs |
| **J6/J7/J8** | DS18B20 | 3 | 1-Wire temperature sensors |
| **J2** | Voltage MPPT | 1 | MPPT voltage monitoring input |
| **J4** | Handbrake | 1 | Handbrake digital signal input |
| **J5/J12** | DC/DC conv. | 2 | Power conversion headers (LM2596S-ADJ) |
| **J9** | PWR 12V | 1 | 12 V main power input connector |
| **J3** | Add | 1 | Auxiliary / expansion connector |
| **CAN1** | CAN transceiver | 1 | TJA1051T CAN bus transceiver |
| **F1/F2/F3** | Fuse | 3 | Overcurrent protection |
| **D1** | PWR LED | 1 | Power-on indicator |
| **D2/D5** | FUNC LED | 2 | Status / function indicators |
| **D3/D4/D7** | D_Zener | 3 | Zener clamp / ESD protection |
| **D6** | PWR diode | 1 | Reverse-polarity / flyback protection |

**Table 5:** Full Component Inventory — Passive Components

| Ref. | Component / Value | Qty | Function |
|---|---|---|---|
| **C1/C2/C3/C4** | 10 µF cap | 4 | Bulk decoupling capacitors |
| **C5/C6** | 0.1 µF cap | 2 | High-freq. bypass capacitors |
| **R7/R8/R9/R10** | 2 kΩ resistor | 4 | UART voltage-level dividers |
| **R10** | 2 kΩ | 1 | CAN termination / pull |
| **4.7k1/2/3** | 4.7 kΩ | 3 | 1-Wire pull-up resistors (DS18B20) |
| **R3–R6, R11–R14** | Resistor (misc) | 8 | Signal conditioning & biasing |
| **R4** | 2 kΩ (R4) | 1 | UART / SIM interface divider |

* The SIM800L-CORE footprint reference in KiCad is used as the physical placeholder for the SIM7670E module during layout. The actually populated component is the SIM7670E as justified in Section 7.1.

## 7.5 Component Photographs

The photographs below show the physical components used to populate the ISS board prototype. Each image is accompanied by the reference designator and the role it fulfils in the circuit.


**Figure 14:** ESP32-DEVKITC V1 microcontroller. Central processing unit for CAN decoding, sensor acquisition, and GPRS telemetry.


**Figure 15:** MCP2515 SPI CAN controller module. Interfaces the ESP32 to the J1939 CAN bus via SPI (J17/J18/J19).


**Figure 16:** TJA1051T automotive CAN transceiver. Converts MCP2515 logic levels to differential CANH/CANL bus signals.

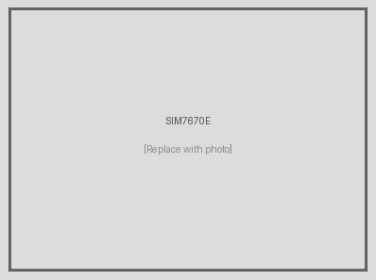



**Figure 17:** SIM7670E LTE Cat-M1 module. Provides 4G cellular connectivity to the VPS cloud backend via GPRS/LTE.


**Figure 18:** DS18B20 1-Wire digital temperature sensors (J6, J7, J8). Monitor motor winding, MPPT heatsink, and DC/DC temperatures.


**Figure 19:** ACS712-30A Hall-effect current sensor modules (J11, J16). Measure solar panel and MPPT output currents.


**Figure 20:** LM2596S-ADJ adjustable buck regulator. Steps down the 12 V vehicle supply to the 3.3 V and 5 V board rails.

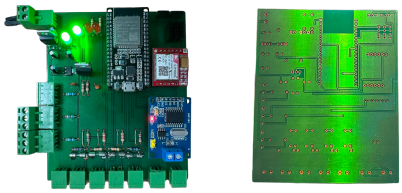


**Figure 21:** Assembled ISS PCB prototype (Rev. RECTIFICATION_2). All major components soldered and board ready for initial power-on testing.

## 7.6 Design Summary

The electronic design of the BAKO Motors ISS board is built around a carefully selected set of components that maximise functionality while remaining practical and cost-effective in the Tunisian market context. The ESP32 provides a powerful, affordable, and well-supported processing core. The MCP2515 and TJA1051T together form a robust, proven CAN bus interface fully compatible with the J1939 protocol used by the O'CELL BMS. The SIM7670E ensures long-term LTE network compatibility with local Tunisian operators, making it a future-proof choice over the 2G-only SIM800L. The LM2596S-ADJ provides efficient switching power conversion for the full board from the vehicle's 12 V rail.

The two-layer KiCad PCB design translates these component choices into a compact, well-organised board with clear functional zoning, adequate protection circuitry, and a layout strategy designed to minimise signal interference between the power, digital, and analogue domains. The design is at revision RECTIFICATION_2, reflecting iterative refinement based on functional testing and layout optimisation. The board is ready for Gerber export and submission to a PCB fabrication house.

# 8 3D DESIGN & MECHANICAL ENGINEERING

## 8.1 Enclosure Design Overview

The BAKO Motors ISS board requires a protective enclosure that shields the PCB from mechanical vibration, dust, and incidental contact with vehicle components, while still providing clear access to all external connectors and sensor cables. The enclosure was designed from scratch as a 3D-printable ABS/PLA part using parametric CAD software, sized to match the ISS board footprint and the connector layout established in the KiCad PCB (Rev. RECTIFICATION_2).

The design objectives were:

- **Secure board mounting** — four corner bosses with through-holes accept M3 screws that fasten the PCB directly to the enclosure floor without additional standoffs.
- **Cable egress slots** — rectangular cutouts on two opposing side walls accommodate the vehicle harness connectors (CAN bus J1, power J9, sensor bundles) and the USB programming port without stressing the PCB edge connectors.
- **Open-top access** — the enclosure is a lidded tray; the open-top design allows heat generated by the LM2596S-ADJ switching regulator to dissipate naturally and provides easy PCB removal for re-flashing.
- **Brand identification** — *SMU-BAKO* and *Senior CSE 2026* text is embossed on the exterior walls to identify the unit during installation.

## 8.2 CAD Model

The enclosure geometry was modelled in a parametric CAD environment. The model consists of a single-body rectangular tray with a uniform 2 mm wall thickness, an internal floor that matches the PCB outline, four corner mounting bosses, and two sets of rectangular cutouts on opposing side walls for the wiring harness.

Figures 22 and 23 show the CAD model from two isometric viewpoints, illustrating the cable egress slots and the embossed labelling on the exterior faces.

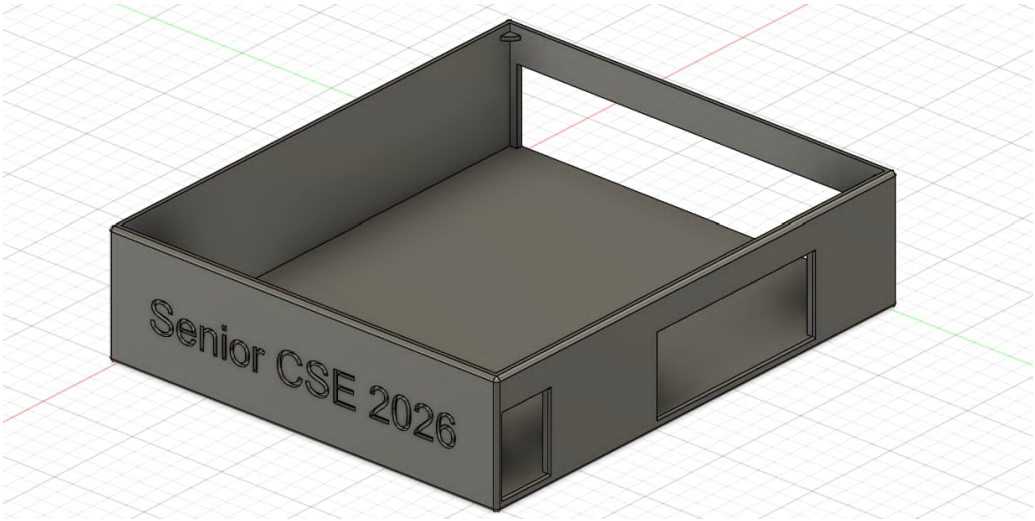



**Figure 22:** ISS enclosure CAD model — isometric front-right view showing the "Senior CSE 2026" embossed face and the rectangular connector cutout on the right side wall.

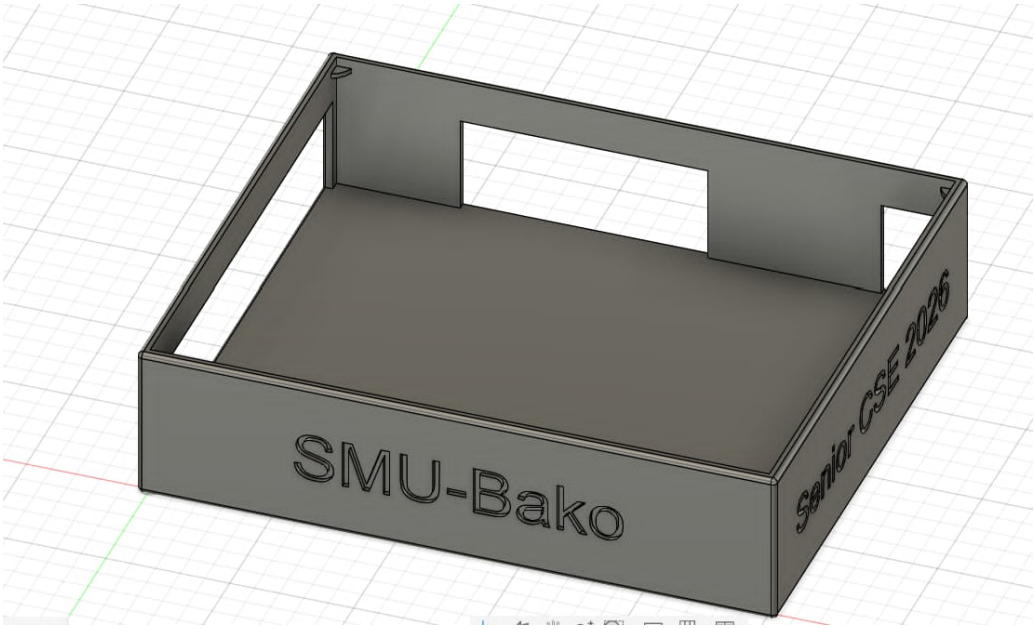


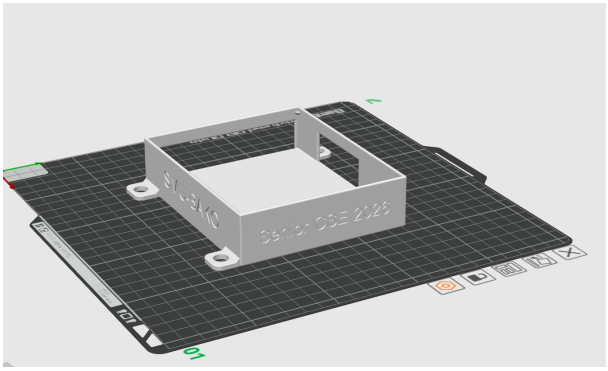

**Figure 23:** ISS enclosure CAD model — isometric front-left view showing the "SMU-Bako" and "Senior CSE 2026" embossed faces and the cable egress slots on two walls.

## 8.3 3D Print Preparation & Slicing

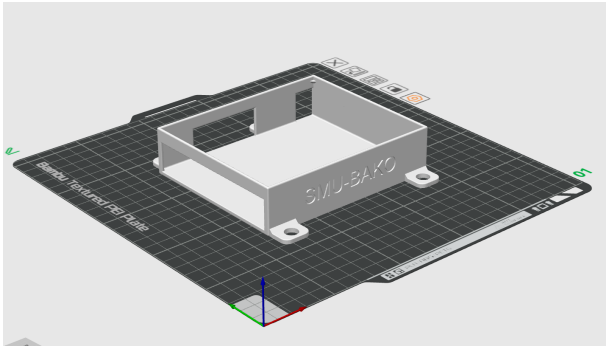
The CAD model was exported as an STL file and imported into **Bambu Studio** for print preparation. Bambu Studio's slicer was used to orient the part, generate support structures, and configure print parameters for a Bambu Lab printer with a textured PEI plate. The enclosure prints in a single session without post-processing assembly.

Figures 24a, 24b, and 24c show the sliced model from three orientations in the Bambu Studio build

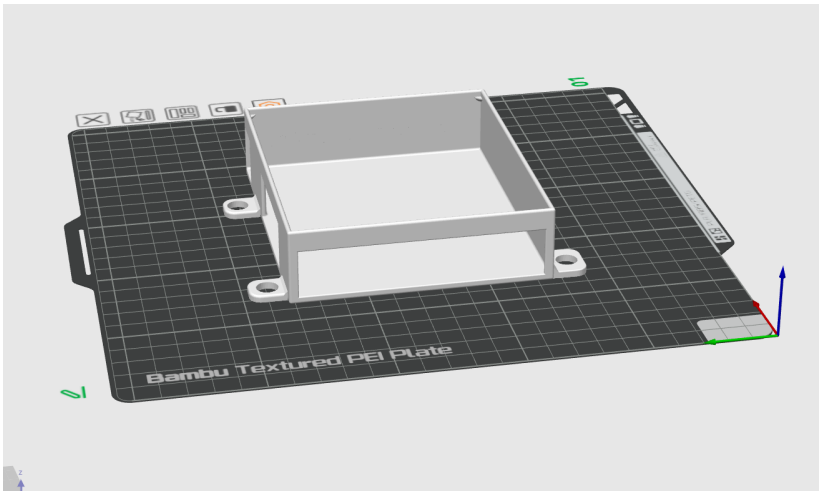
plate preview, confirming correct orientation (open face up), corner mounting feet visible from all angles, and no unsupported overhangs requiring internal supports.




**(a)** Isometric front-right — "SMU-BAKO / Senior CSE 2026" labelling and mounting corner feet visible.




**(b)** Isometric front-left — front cable egress slot and open-top orientation confirmed.




**(c)** Rear isometric view — two back-wall cutouts for the vehicle harness and CAN bus connector visible.

**Figure 24:** Bambu Studio slicer views of the ISS enclosure on the textured PEI build plate, shown from three orientations. The enclosure prints open-face-up with no supports required.

## 8.4 Mechanical Features Summary

Table 6 summarises the key mechanical parameters of the enclosure as designed.

**Table 6:** ISS Enclosure Mechanical Parameters

| **Parameter** | Value |
|---|---|
| **Design tool** | Parametric CAD (Fusion 360 / equivalent) |
| **Slicer** | Bambu Studio |
| **Print orientation** | Open face up, flat on build plate |
| **Wall thickness** | 2 mm uniform |
| **Material** | PLA or PETG (prototype); ABS for vehicle environment |
| **Corner mounting** | 4× M3 boss, 6 mm diameter, through-hole for PCB screws |
| **Cable egress slots** | 2 side walls; rectangular cutouts sized for JST + screw-terminal connectors |
| **Exterior labelling** | "SMU-BAKO" and "Senior CSE 2026" embossed text, 0.5 mm depth |
| **Protection rating** | Open-top prototype; IP-rated lid deferred to next revision |

The enclosure design is production-ready for the prototype phase. A sealed lid with gasket groove and SIM7670E antenna pass-through is identified as a short-term development item for field deployment, targeting IP54 ingress protection as noted in the recommendations chapter (Section 20).

# 9 SOFTWARE & CODE IMPLEMENTATION

This section describes the software developed for the Bako OBD ISS platform, covering the embedded firmware running on the ESP32 microcontroller, the battery management decoding logic, the cloud publishing pipeline, and the validation strategy used to verify correct behavior at each layer.

## 9.1 Firmware Architecture & RTOS Integration

The ESP32 firmware is built on the **Arduino framework** using the **PlatformIO** build system [5], which provides dependency management, cross-compilation, and serial monitoring in a single toolchain. Although the Arduino framework does not expose the underlying FreeRTOS scheduler directly, the ESP32's dual-core architecture is leveraged implicitly: the Arduino `loop()` function executes on Core 1 while the Wi-Fi/Bluetooth stack occupies Core 0. All BMS-related tasks — CAN frame reception, state accumulation, and HTTP publishing — run sequentially on Core 1 without preemption, which simplifies state management and eliminates the need for mutexes at the firmware level.

The firmware follows a **cooperative, interrupt-driven** model. The MCP2515 CAN transceiver asserts an active-low `INT` line (GPIO 4) each time a new frame is placed in its receive buffer. The main loop polls this pin continuously and drains all pending frames before entering the timed publish cycle. This design ensures that no CAN frame is ever dropped due to buffer overflow, even during the longer HTTP transaction window.

The overall execution model is summarized in Figure 25.

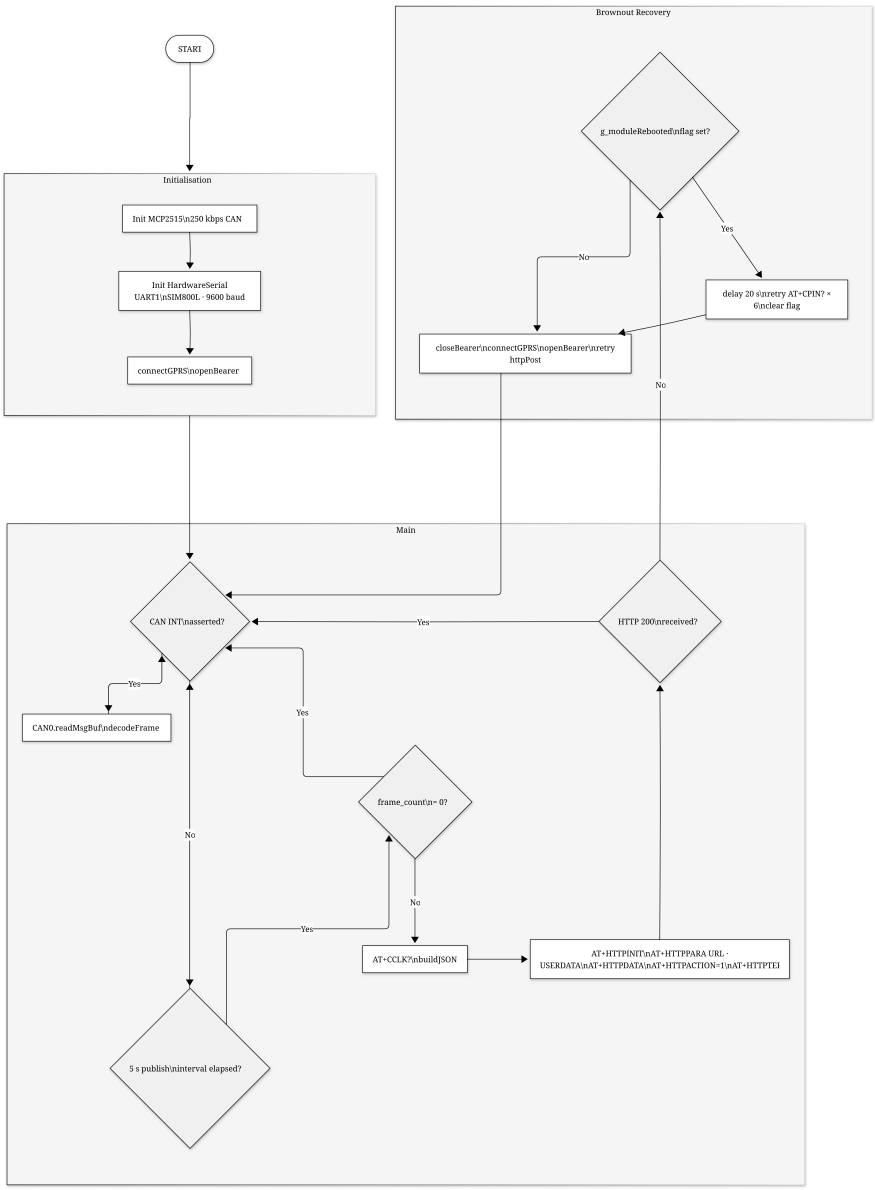



**Figure 25:** Firmware main-loop execution model: CAN drain → publish every 5 s

The SIM800L GSM/GPRS module is driven via **HardwareSerial UART1** (GPIO 16/17) at 9600 baud. HardwareSerial was chosen over SoftwareSerial because the module's AT+HTTP* responses can arrive as multi-line bursts that SoftwareSerial's byte-at-a-time interrupt handler occasionally corrupts at 9600 baud on a loaded CPU.

## 9.2 BMS Firmware

The BMS firmware is responsible for reading raw CAN frames from the O'CELL IFS60.8-500-F-E3 battery management system [2], decoding them according to the **SAE J1939** standard [3], maintaining a live BMS state structure, and serializing that state to JSON for cloud upload.

### 9.2.1 CAN Bus Interface

The physical CAN interface uses a **MCP2515** controller connected to the ESP32 via the VSPI bus at 10 MHz. The bus operates at **250 kbps**, which is the standard J1939 baud rate for vehicle control networks [20]. All frames from the O'CELL BMS use 29-bit extended CAN identifiers, formatted according to the J1939 Parameter Group Number (PGN) convention:

$$\text{CAN ID} = 0x18\_\text{FUNC\_SUB\_SA} \tag{1}$$

where `FUNC` is the PGN function byte that identifies the frame type, `SUB` encodes sub-addressing or destination, and `SA` = 0xF4 is the source address of the BMS module.

The MCP2515 library is configured in `MCP_ANY` mask mode so that all frames on the bus are received without hardware filtering. Software-side filtering is performed in the `decodeFrame()` function by inspecting the `FUNC` and `SUB` bytes extracted from the 29-bit ID.

### 9.2.2 Supported Frame Types

Five distinct frame types are decoded, covering all BMS data needed for monitoring and fault diagnosis. Table 7 lists each frame with its CAN ID, transmission rate, and data content.

**Table 7:** O'CELL BMS CAN Frame Map (SAE J1939, 250 kbps)

| CAN ID | Name | Rate (ms) | Content |
|---|---|---|---|
| 0x18C8--CC28F4 | Cell voltages (5 frames) | 500 | 4 cells each, big-endian uint16, 1 mV/bit |
| 0x18B428F4 | Temperature probes | 500 | Up to 8 probes, raw − 40 = °C |
| 0x18FF28F4 | BMS Basic Message 1 | 100 | SOC, current, voltage, fault flags |
| 0x18FE28F4 | BMS Basic Message 2 | 100 | Min/max cell mV, discharge limit |
| 0x18FFE5F4 | Charging request | 500 | Max charge V, max charge A, start signal |

**Cell Voltage Frames (0x18C8–CC28F4).** The 19 battery cells are transmitted across five consecutive frames. Each frame carries four cells encoded as big-endian 16-bit unsigned integers, with a resolution of 1 mV per bit. The last frame (0x18CC28F4) covers cells 17–19 and pads the unused fourth slot with 0x0000. The firmware detects and skips zero-padded slots:

```
for (int i = 0; i < 4; i++) {
    uint16_t mv = u16be(data, i * 2);
    if (mv != 0 && (base + i) < 19)
        bms.cell_mv[base + i] = mv;
}
```

After each group update, aggregate statistics (minimum, maximum, average) are recomputed from all non-zero cells, and a voltage-based SOC estimate is derived as described in Section 9.2.4.

**BMS Basic Message 1 (0x18FF28F4).** This 100 ms frame carries the core operational state of the pack. The byte layout is as follows:

- **Byte 0** — Status bit-field: charge cable connected (bit 0), charging in progress (bit 1), discharging in progress (bit 2), BMS ready (bit 3), discharge contactor closed (bit 4), charge contactor closed (bit 5).
- **Byte 1** — State of charge from the BMS coulomb counter (0–100%).
- **Bytes 2–3** — Pack current as a little-endian uint16 with a $-5000$ offset and 0.1 A/bit scaling. Positive values indicate discharge; negative values indicate charging.
- **Bytes 4–5** — Direct BMS pack voltage, little-endian uint16, 0.1 V/bit.
- **Byte 6** — Fault severity level (0 = normal, 1 = serious fault).
- **Byte 7** — Fault code (see Table 8).

**Table 8:** BMS Fault Code Map (byte 7 of 0x18FF28F4)

| Code | Description |
|---|---|
| 0x00 | No fault (normal operation) |
| 0x01 | Over-temperature — severe |
| 0x02 | Total pack voltage too high |
| 0x03 | Total pack voltage too low |
| 0x04 | Discharge over-current |
| 0x05 | Cell voltage too high |
| 0x06 | Cell voltage too low |

**Temperature Probes (0x18B428F4).** Up to eight thermal probes are encoded as single bytes using the formula $T[°\mathrm{C}] = \mathrm{raw} - 40$ [21]. A raw value of 0xFF indicates a disconnected sensor and is skipped. The O'CELL pack fitted to the vehicle contains four active probes.

**Charging Request (0x18FFE5F4).** This frame is transmitted by the BMS toward the on-board charger. It specifies the maximum allowable terminal voltage (bytes 0–1, 0.1 V/bit) and maximum charge current (bytes 2–3, 0.1 A/bit). Bit 0 of byte 4 being clear signals that the BMS is requesting the charger to begin charging.

### 9.2.3 BMS State Structure

All decoded values are accumulated into a single `BMSData` struct that persists across frames and represents the most recent complete snapshot of the battery's state. The struct is updated in-place by `decodeFrame()` and read atomically by `buildJSON()` at each publish cycle. Because the firmware runs single-threaded, no locking is required.

### 9.2.4 SOC Calibration

The firmware uses a **voltage-based SOC estimate** derived from the average cell voltage [22]. This method was selected because it correlates closely with the vehicle's own dashboard reading, unlike the BMS

internal coulomb counter which can drift over time [23].

The calibration was performed over four separate on-vehicle sessions, comparing the computed average cell voltage with the vehicle's onboard display:

$$\text{SOC}[\%] = \frac{\overline{V}_{\text{cell}} - 2500}{3387 - 2500} \times 100 \tag{2}$$

where $\overline{V}_{\text{cell}}$ is the average cell voltage in millivolts. The lower bound of 2500 mV/cell corresponds to 0% SOC (47.50 V pack), and 3387 mV/cell was calibrated to match 100% on the car display (64.35 V pack). Values above 3387 mV, which occur during active charging, are clamped to 100%.

Table 9 shows the four calibration sessions used to arrive at the 3387 mV reference point.

**Table 9:** SOC Calibration Data — Four On-Vehicle Sessions

| Session | Avg cell (mV) | Car display (%) | Solved top (mV) |
|---|---|---|---|
| 2026-03-31 10:06 | 3301.6 | 89.0 | 3400.7 |
| 2026-03-31 10:57 | 3306.2 | 89.0 | 3405.8 |
| 2026-04-01 13:03 | 3284.8 | 89.4 | 3377.9 |
| 2026-04-01 16:04 | 3285.0 | 88.4 | 3387.4 |
| | | **Adopted reference** | **3387 mV** |

### 9.2.5 Cloud Publishing via SIM800L

At every 5-second publish interval, the firmware calls `buildJSON()` to serialize the current `BMSData` struct into a nested JSON document using the ArduinoJson v7 library [24], then calls `httpPost()` to deliver it to the VPS backend via SIM800L GPRS.

The payload uses a five-section nested schema: a top-level envelope wraps `battery`, `temperatures`, `solar`, `dc_dc`, and `vehicle` sub-objects. Individual cell voltages are encoded as a 1-based JSON array (`cells[0] = null`, `cells[1]…cells[19] = {mv, status}`). Temperature probes follow the same convention in `temperatures.battery_cells`. Sensors not yet instrumented (solar, DC/DC, vehicle) appear as `null`-valued fields so the schema remains stable as additional sensors are commissioned.

The HTTP transaction uses the SIM800L's native **AT+HTTP*** command set over plain HTTP (AT+HTTPSSL=0). The sequence is:

1. `AT+HTTPTERM` — close any stale session from a previous transaction.
2. `AT+HTTPINIT` — open a new HTTP session.
3. `AT+HTTPPARA="URL",...` — set the full endpoint URL.
4. `AT+HTTPPARA="USERDATA","X-Api-Key: key\r\n"` — inject the API authentication header.
5. `AT+HTTPDATA=len,10000` — upload the JSON body.
6. `AT+HTTPACTION=1` — execute the HTTP POST and wait for `+HTTPACTION:` response.
7. `AT+HTTPTERM` — close the session.

During the AT+HTTPACTION wait loop (up to 30 s), the firmware continues draining CAN frames from the MCP2515 receive buffer so that no BMS data is lost while the module is busy.

### 9.2.6 Brownout Recovery

A known failure mode of the SIM800L is a mid-transaction power brownout caused by the ∼1.5 A peak current during LTE transmission exceeding the supply rail's transient capability. When this occurs, the module resets and emits an unsolicited `"Call Ready"` string in its UART output.

The firmware detects this string in every AT response buffer (both `sendAT()` and `sendATExpect()` and the `HTTPACTION` wait loop) and sets a global `g_moduleRebooted` flag. When a publish cycle fails and this flag is set, the recovery sequence is:

1. Wait 20 s for the SIM card and LTE stack to re-initialize.
2. Retry `AT+CPIN?` up to six times at 4-second intervals.
3. Re-open the GPRS bearer and retry the publish once more.

The long-term hardware fix is to add a 1000 µF / 10 V bulk capacitor close to the SIM800L VCC pin.

## 9.3 CAN Frame Parser

The `parse_can()` function in the Python backend implements the BAKO CAN Protocol Rev 1.0, decoding all frames broadcast by the BMS (SA = 0xF4) and all sensor gateway frames transmitted by the ESP32 (SA = 0xAA). A companion `parse_sensor()` function handles the `SENSOR:` text-line fallback format emitted by the Python simulator.

### 9.3.1 BMS Frames (SA = 0xF4)

Table 10 lists all five BMS frame types decoded by `parse_can()`, with their CAN IDs, transmission cycles, decoded fields, and key decoding logic.

**Table 10:** BMS CAN Frame Parser — SA = 0xF4 (O'CELL BMS)

| CAN ID | Cycle | Fields decoded | Key decoding logic |
|---|---|---|---|
| 0x18FF28F4 | 100 ms | SOC%, pack current, pack voltage, fault level, error code | Pack current: (uint16_LE − 5000) × 0.1 A; Pack voltage: uint16_LE × 0.1 V |
| 0x18FE28F4 | 100 ms | Max/min cell voltage, max/min temp, max discharge current | Temperatures: uint8 − 40 °C; 0xFF = not connected |
| 0x18C8--CC28F4 | 500 ms | All 19 cell voltages (4 per frame) | Big-endian uint16 mV pairs; PF byte determines group index (0xC8 = group 0 → cells 1–4) |
| 0x18B428F4 | 500 ms | Up to 8 temperature probe values | uint8 offset −40 °C; 0xFF = probe not connected |
| 0x18FFE5F4 | 1000 ms | Max charge voltage, max charge current | uint16_LE × 0.1 V / 0.1 A |

## 9.3.2 ESP32 Sensor Gateway Frames (SA = 0xAA)

In addition to receiving BMS frames, the ESP32 re-broadcasts external sensor readings as J1939-formatted CAN frames using its own source address (SA = 0xAA). This allows any device on the bus to consume solar, MPPT, thermal, and vehicle state data using a standard CAN interface. Table 11 lists all eight gateway frame types.

**Table 11:** ESP32 Sensor Gateway CAN Frame Map — SA = 0xAA

| CAN ID | Cycle | Content |
|---|---|---|
| 0x18D001AA | 200 ms | Solar panel current before MPPT — raw + 5-cycle average (uint16_LE × 0.1 A) |
| 0x18D101AA | 200 ms | Solar panel voltage + open-circuit $V_{oc}$ (uint16_LE × 0.1 V) |
| 0x18D201AA | 200 ms | MPPT output current + mode (0=off / 1=track / 2=CV / 3=float) + efficiency % |
| 0x18D301AA | 200 ms | DC/DC 12 V output voltage (× 0.01 V) + current (× 0.1 A; 0xFFFF = not fitted) |
| 0x18D401AA | 1000 ms | Motor temperature (uint8 − 40 °C) |
| 0x18D501AA | 1000 ms | MPPT heatsink temperature (uint8 − 40 °C) |
| 0x18D601AA | 1000 ms | DC/DC heatsink temperature (uint8 − 40 °C) |
| 0x18D701AA | 500 ms | Handbrake position: 0 = released, 1 = engaged, 0xFF = sensor fault; + debounce flag |
| 0x18D801AA | 500 ms | GNSS information (position, speed, heading) |

## 9.3.3 SOC Representations and SOH Calculation

Three distinct SOC representations are maintained simultaneously by the parser:

**soc_bms** Raw coulomb-counter value from byte 2 of 0x18FF28F4. Directly reported by the BMS hardware; prone to drift over long charge/discharge cycles.

`soc` Voltage-based SOC derived from the average cell voltage using the calibrated formula (Equation 2):

$$\text{SOC} = \frac{\overline{V}_{\text{cell}} - 2500}{3387 - 2500} \times 100, \quad \text{clamped to } [0, 100]\%$$

The upper reference (3387 mV) was calibrated from four real vehicle logs. This is the primary value displayed on the dashboard.

`soc_coulomb` Accumulated Ah count from 0x18FFE5F4, used for charger control decisions.

State of Health (SoH) is calculated from the ratio of the measured actual capacity to the rated pack capacity:

$$\text{SoH} = \frac{C_{\text{actual}}}{C_{\text{rated}}} \times 100 = \frac{44.7}{50.0} \times 100 = 89.4\%$$

## 9.4 Motor Control Algorithms

Motor control algorithms are outside the scope of this monitoring system. The ESP32 firmware reads and monitors motor-related sensor data — specifically motor temperature via DS18B20 and current draw via the ACS712/ACS758 sensors — but does not interface with the motor controller or implement any drive control logic. Motor speed and torque management remain the responsibility of the existing vehicle control electronics. This boundary is intentional: the monitoring system observes and reports motor operating conditions without intervening in the drivetrain control loop.

## 9.5 Vehicle Management Unit (VMU) Software

The Vehicle Management Unit (VMU) is implemented on the same ESP32 microcontroller and is responsible for the higher-level coordination layer above raw sensor acquisition. Specifically, the VMU software handles four functions:

**Data aggregation** Collecting decoded CAN frames from the BMS (cell voltages, SOC, pack current, temperatures, and fault flags) together with readings from external sensors (DC/DC voltage, MPPT current, motor temperature, and handbrake position) into a unified telemetry structure.

**Data formatting** Structuring all acquired telemetry into a consistent JSON schema for transmission to the dashboard over WebSocket or to the cloud back-end via the SIM800L GSM/GPRS module.

**Communication management** Switching between Wi-Fi Access Point mode (for local dashboard access) and Serial mode (for USB-connected debugging and development), based on the firmware build configuration.

**Alert handling** Monitoring configured thresholds — including over-temperature limits for the motor and DC/DC converter, battery fault flags from the BMS, and handbrake engagement — and updating the status LED outputs and dashboard warning panels accordingly.

The VMU software does not perform low-level motor control or BMS internal management; it only reads data from those systems and presents it to the operator.

## 9.6 ESP32 Access Point Mode

The ESP32 operates in **Wi-Fi Access Point (AP) mode** for local dashboard access, having been migrated from an earlier Station Mode implementation. The ESP32 chip supports two distinct Wi-Fi operating modes: Station Mode (STA), where it joins an existing router and receives a DHCP-assigned IP; and Access Point Mode (AP), where it becomes the router, broadcasts its own SSID, and assigns addresses to connecting devices with no external router required.

### Why Station Mode Was Replaced

The original firmware connected the ESP32 to a home Wi-Fi router as a station. This introduced three practical drawbacks: the SSID and password were hardcoded, so using the system in a different location required reflashing; the router's DHCP server assigned a different IP address after every reboot, requiring firmware updates to match the PC's current address; and without a nearby router, the simulation environment required a mobile hotspot workaround that added extra configuration steps. AP mode resolves all three issues: the ESP32 brings its own network, the IP address 192.168.4.1 never changes, and the system operates in any location without code changes.

### AP Mode Internal Operation

When `WiFi.softAP()` is called, the ESP32 radio switches to infrastructure mode. The built-in DHCP server listens on 192.168.4.1 and issues leases from the pool 192.168.4.2–192.168.4.11. The application code opens a `WiFiServer` on port 9000. When a client connects to 192.168.4.1:9000, the lwIP stack completes the TCP three-way handshake and places the connection in the accept queue for the firmware to service.

### Firmware Changes

The migration from Station to AP mode required minimal changes, isolated entirely to the Wi-Fi and TCP layer. The simulation engine, phase logic, frame encoders, and timing are identical to the station-mode version. The key changes were:

- **Removed:** `WIFI_SSID`, `WIFI_PASS`, `SERVER_IP` constants; `ensureWiFi()` and `ensureTCP()` functions.
- **Added:** `AP_SSID`, `AP_PASS`, `AP_CHANNEL` constants; `WiFiServer server(TCP_PORT)` global; `startAP()` and `ensureClient()` functions.
- `setup()` now calls `startAP()` and `server.begin()` instead of `ensureWiFi()` and `ensureTCP()`.
- `loop()` calls `ensureClient()` with a 50 ms yield when no client is connected, keeping the DHCP and beacon tasks running without burning CPU.

The `server.py` dashboard application required zero modifications. It accepts connections on any local interface, including the 192.168.4.x interface created when the PC joins the BAKO_SMU network. The

only operational change for the user is connecting the PC to the `BAKO_SMU` Wi-Fi network before launching the server.

## 9.7 OTA Firmware Updates & Web Configuration Portal

This section documents the third and final stage of the Wi-Fi connectivity evolution. Following the migration from Station Mode to Access Point (AP) Mode, two complementary features were added: Over-The-Air (OTA) firmware flashing and a browser-based configuration portal. Together they make the system fully maintainable in the field without physical access to the PCB.

## Development Progression Summary

The connectivity stack evolved through three clearly defined stages. Table 12 summarises each stage, its purpose, and the limitation that motivated the next iteration.

**Table 12:** Wi-Fi Connectivity Evolution Across Three Development Stages

| Stage | Mode | What was achieved | Limitation that led to next stage |
|---|---|---|---|
| 1 | **Station Mode (STA)** | ESP32 joins existing Wi-Fi router. DHCP-assigned IP. Dashboard PC on same LAN. | Router dependency, dynamic IP, hardcoded SSID/password in firmware. |
| 2 | **Access Point Mode (AP)** | ESP32 is its own router. Fixed IP 192.168.4.1. No external network needed. | Reflashing still required USB cable. Network parameters hardcoded. |
| 3 | **AP + OTA + Web Config** | Wireless firmware flashing from IDE. Browser portal to change all network settings without reflashing. | No further limitation — system is fully field-maintainable. |

## Over-The-Air Firmware Update

The **ArduinoOTA** library was integrated into the ESP32 firmware to allow wireless firmware flashing directly from the Arduino IDE over the existing AP network. The OTA service runs as a lightweight background handler (`ArduinoOTA.handle()`) called at every loop iteration alongside the TCP and web servers, adding negligible overhead.

**Motivation.** Once the PCB is mounted inside a vehicle enclosure, accessing the USB port for reflashing is impractical. OTA eliminates this constraint: the technician connects a laptop to the `BAKO_BMS` Wi-Fi network, selects the network port in the IDE, and clicks Upload — the new firmware is transmitted wirelessly, verified, and applied.

**OTA flashing procedure.**

1. Power the ESP32 board (vehicle on, or bench supply connected).

2. Connect the laptop to the `BAKO_BMS` Wi-Fi network broadcast by the ESP32.
3. Open Arduino IDE → Tools → Port. A network port labelled with the OTA hostname (`BAKO_OTA` by default) appears automatically.
4. Select that port, compile, and click Upload as normal.
5. The IDE transmits the binary wirelessly. The ESP32 verifies integrity and reboots into the new firmware.

**Security.** The OTA service is password-protected to prevent unauthorised firmware injection. The password is configurable through the web portal and stored in NVS flash so it survives reboots. If an active TCP dashboard client is connected when an OTA session begins, the client socket is cleanly closed before flashing starts to prevent data corruption.

## 9.8 Unit & Integration Testing

Software testing for the BMS firmware and cloud pipeline was conducted at three levels: unit testing of the decoding logic, integration testing of the full CAN-to-cloud pipeline, and functional testing of the live dashboard.

### 9.8.1 Unit Testing — Frame Decoder

Each frame type in `decodeFrame()` was verified by replaying known raw bytes and checking the decoded output against hand-computed expected values. The test case below illustrates verification of cell voltage frame 0x18CB28F4:

```
Frame ID: 0x18CB28F4   Data: 0D 3F  0D 38  0D 3A  0D 39


0x0D3F = 3391 mV  -> Cell 13   (big-endian, group = 0xCB - 0xC8 = 3)
0x0D38 = 3384 mV  -> Cell 14
0x0D3A = 3386 mV  -> Cell 15
0x0D39 = 3385 mV  -> Cell 16
```

Similarly, the signed current decoding formula (offset $-5000$, scale 0.1 A/bit) and the SOC formula (Equation (2)) were verified against values from the real car log file `bms_log_2026-03-10T10-20-02.txt`, which contains 3197 frames captured during an on-vehicle session.

### 9.8.2 Integration Testing — Cloud Pipeline

The end-to-end pipeline was tested without live hardware using the `replay_to_cloud.py` tool, which parses the captured CAN log and replays decoded JSON snapshots to `POST /api/ingest` at configurable speed. This allowed the following verifications:

- The `/api/ingest` endpoint correctly rejects requests with an invalid `X-Api-Key` header (HTTP 401).

- Each accepted snapshot is stored in the SQLite `bms_cloud.db` database with correct timestamp and device ID.
- The `/ws/cloud` WebSocket pushes the most recent snapshot to all connected browser clients within 2 seconds of ingestion.
- The `/api/history` endpoint returns records newest-first with correct JSON structure.

### 9.8.3 Functional Testing — Live Dashboard

The live dashboard (`frontend/index.html`) was tested in both SERIAL and CLOUD modes:

**SERIAL mode** ESP32 connected via USB to a development laptop. The server decoded incoming frames in real time and the 19-cell voltage grid, SOC ring, and temperature bars updated at 10 Hz without visible lag.

**CLOUD mode** Data replayed from the log file through `/api/ingest`. The dashboard received snapshots via `/ws/cloud` at 2 Hz and all panels rendered correctly, including the cloud communication log panel which displayed timestamped RECV and WS events.

**Reconnect behavior** The backend was deliberately stopped while the dashboard was open. The browser WebSocket closed, the connection status indicator turned red (Disconnected), and the overlay appeared. On server restart, the client automatically reconnected within 5 seconds and resumed live updates.

## 9.9 Web Dashboard

The BAKO DIAGNOSTICS web dashboard is a dark-theme, single-page web application that provides a real-time overview of the entire vehicle monitoring state. It is served by the FastAPI back-end at the root URL (/) and communicates with the server via WebSocket (`/ws` in serial mode, `/ws/cloud` in cloud mode). The dashboard requires no installation; any browser that connects to the server IP receives the complete HTML, CSS, and JavaScript in a single response.


**Figure 26:** BAKO DIAGNOSTICS live dashboard — dark-theme single-page interface showing SOC ring gauge, 19-cell voltage grid, temperature time-series chart, and external sensor panel.

## Panel Layout

The dashboard is organised into four panels arranged to give the engineer an at-a-glance view of all monitored subsystems. Table 13 describes the content of each panel.

**Table 13:** Dashboard Panel Layout

| Panel | Content |
|---|---|
| **Left panel** | State of charge large ring gauge (colour-coded: green >30%, orange 15–30%, red <15%); pack voltage (V); pack current (A, signed); remaining capacity (Ah); State of Health (SoH) indicator. |
| **Central panel** | Individual cell voltage bars for all 19 cells with per-cell colour coding (overvoltage / full / balanced / nominal / low / undervoltage); minimum, maximum, and average cell voltage; cell spread indicator (max − min, mV). |
| **Temperature panel** | Four BMS internal temperature probe readings with individual bars; rolling average value; high-temperature threshold alert line. |
| **Temperature chart** | Time-series line chart displaying the four individual probe temperatures and their rolling average scrolling in real time. |
| **Right panel** | Extended sensor readings: solar panel voltage and current, MPPT output current, MPPT mode and efficiency, DC/DC 12 V output, motor temperature, MPPT heatsink temperature, handbrake status. |
| **Serial monitor** | Live scrolling log of raw CAN frames and `SENSOR:` lines as received from the ESP32. |
| **File export** | Export full session log as `.TXT` or `.XLSX` with timestamp column. |

## Threshold Colour Coding

Cell voltage bars transition through three colour states based on configurable per-cell thresholds: green when the cell is within its normal operating band (2900–3450 mV for LiFePO4), orange when the cell approaches either end of its safe range, and red when a threshold breach is detected. The SOC ring applies the same three-state scheme based on the pack-level SOC percentage. These thresholds are defined as JavaScript constants in `frontend/index.html` and can be adjusted without modifying the back-end.

## Data Export

The dashboard includes a **Download CSV** button that retrieves the last 100 snapshots from `/api/history` and writes them to a timestamped `.csv` file. The export covers all 19 cell voltages, all four temperature probes, pack SOC, pack voltage, pack current, and extended sensor readings in a single flat row per snapshot, suitable for import into spreadsheet or data analysis tools.

## Mode Selector

A mode pill in the dashboard header opens a modal dialog that lets the operator switch the active data source between Serial and Wi-Fi AP modes at runtime, without restarting the server. The mode selection is sent via `POST /api/mode` and persisted to `mode_config.json` so that it is automatically restored on the next server launch.

- **Serial mode** — auto-detects the connected ESP32 USB port, or accepts manual port entry with a con-

figurable baud rate. The server reads newline-delimited CAN frames from the serial stream.

- **Wi-Fi mode** — the operator enters the ESP32 IP address (default 192.168.4.1) and TCP port (default 9000), then clicks Apply. The PC must already be connected to the `BAKO_SMU` Wi-Fi network.


**Figure 27:** Dashboard mode selector dialog: Serial mode (left) and Wi-Fi AP mode (right). Mode is switched without restarting the server.

## 9.10 Simulation & Emulation

Two complementary simulation tools were developed to enable dashboard testing and firmware validation without requiring a live vehicle or a real O'CELL BMS. Both tools produce data that is indistinguishable from real hardware at the server API level.

## ESP32 CAN Frame Simulator

The firmware file `CAN_simulation_7_phases/CAN_simulation_7_phases.ino` runs on a standalone ESP32 and generates a complete stream of realistic O'CELL BMS CAN frames over USB Serial at 115200 baud, in the same `[Xms] ID: 0x... DLC: N Data: ...` format that the real vehicle produces.

The simulator models a full charge/discharge lifecycle across seven sequential phases. Table 14 describes each phase and its duration.

**Table 14:** ESP32 CAN Simulator — 7-Phase Scenario Cycle

| Phase | **Name** | Duration | Description |
|---|---|---|---|
| 0 | **REST** | 20 s | Pack idle at ≈72% SOC; cells balanced; ambient temperature (≈24°C). |
| 1 | **BULK DISCHARGE** | 120 s | 38 A load; SOC falls 72%→30%; temperatures climb; weak cells 7 and 14 develop 15 mV extra sag. |
| 2 | **LOW SOC WARNING** | 25 s | BMS throttles current 38 A→12 A; SOC 30%→18%; discharge limit ramps down to 15 A. |
| 3 | **RECOVERY / REST** | 15 s | Load removed; voltage bounce-back; temperatures plateau. |
| 4 | **CC CHARGE** | 90 s | 22 A constant-current charge; SOC climbs 18%→80%; cells re-balance. |
| 5 | **CV TAPER** | 40 s | Pack near full; charge current tapers 22 A→2 A; spread narrows. |
| 6 | **FULL REST** | 20 s | SOC 100%; zero charge current; cells balanced ($\leq$8 mV spread); loop restarts. |

Each of the 19 cells has a fixed personality offset ($-20$ mV to $+12$ mV) so the cell spread is realistic and stable across frames. Cells 7 and 14 carry an additional $-18$ mV offset and a load-proportional extra sag, simulating weak cells that would trigger a threshold alert during heavy discharge. The thermal model tracks three sensors at different positions (centre, end-plate, BMS PCB) with individual heating and cooling rates. All six frame types are emitted every 500 ms: cell voltage groups (0x18C8–CC28F4), temperature (0x18B428F4), Basic Message 1 (0x18FF28F4), Basic Message 2 (0x18FE28F4), and charging request (0x18FFE5F4).

### 9.10.1 ESP32 Firmware Simulator & System Interface

An ESP32-based firmware simulator and monitoring interface were developed to validate the BAKO system without requiring the physical vehicle during every testing session. The simulator reproduces realistic vehicle behaviour by generating sensor telemetry and transmitting it continuously to the monitoring dashboard.

The implemented firmware simulates 12 external sensors, including current, voltage, temperature, handbrake, and GNSS data. To improve testing coverage, seven automated operating scenarios were integrated: Normal Driving, Low Battery Warning, Charging Mode, High Solar Input, Night Mode, Handbrake Fault, and Overheat Condition. The active scenario changes automatically every 30 seconds, while all sensor values are transmitted every 500 ms.

The ESP32 communicates with the backend dashboard through WebSocket communication, enabling real-time updates directly inside the browser interface. The dashboard displays battery cell voltages, temperatures, state of charge (SOC), solar monitoring information, and handbrake status. In addition, telemetry logs can be exported in TXT or XLSX formats for later analysis and validation.

This testing environment significantly reduces dependency on the physical vehicle during development and debugging phases. It also provides a reproducible and controlled platform for validating system behaviour, communication pipelines, and dashboard integration under multiple operating conditions.

### Python Log Replay Tool

The script `simulate.py` provides a complementary simulation path that uses a captured real-vehicle CAN log rather than synthetic data. It reads the log file `data/raw/bms_log_2026-03-10T10-20-02.txt` (containing 3197 frames from a real on-vehicle session) and pipes it into the live server over a virtual serial port, allowing the full dashboard to operate on authentic data without physical hardware.

The replay sequence is:

1. A virtual PTY (pseudo-terminal) pair is created using `socat`. One end is handed to the server as its serial port; the other is used by the feeder thread to write log lines.
2. The FastAPI server is launched as a subprocess on port 8765.
3. Once the server's `/api/mode` endpoint responds, the script posts `mode=serial` with the virtual PTY path, switching the server into serial-reader mode.
4. The browser is opened automatically at `http://localhost:8765`.
5. Log frames are streamed to the PTY feeder end with inter-frame delays derived from the original timestamps, scaled by a configurable speed multiplier (default $2\times$). The log loops continuously until interrupted.

The `--speed` argument allows accelerated replay (e.g., `--speed 10` for rapid regression testing) while `--no-browser` suppresses the automatic browser launch for headless test environments.

## 9.10.2 Python Emulator Extension: Local Storage and TCP Transmission

As an extension to the Python-based sensor emulator, a local storage and TCP communication layer was implemented to improve telemetry reliability and prepare the emulator for remote server integration.

The emulator first generates realistic values for the 12 external sensors, including current, voltage, temperature, handbrake state, and GNSS data. Each generated record is then stored locally in a SQLite database before any transmission attempt. This ensures that telemetry data is preserved even if the network connection is unavailable.

A TCP socket client was also developed to transmit unsent records to a remote server. During local testing, a simulated TCP server was used to reproduce the behaviour of the future VPS server. When the server is available, the client sends the stored telemetry records and marks them as sent in the local database. If the connection fails, the data remains stored locally and is retried automatically during the next transmission cycle.

This architecture implements a **store-and-forward** mechanism. It allows the emulator to continue operating during internet outages while preventing telemetry loss. Once VPS access is provided, only the server IP address in the TCP client configuration needs to be replaced to enable remote deployment.

The implemented extension therefore adds three key capabilities to the emulator:

- local persistence of generated telemetry data using SQLite;
- TCP socket transmission instead of WebSocket communication;

- offline buffering and automatic retransmission after reconnection.

This work prepares the emulator for reliable remote telemetry testing and aligns it with the cloud connectivity objectives of the BAKO monitoring system.

## 9.11 Wi-Fi Dashboard Connection

The **Wi-Fi dashboard connection** mode allows the FastAPI server to receive telemetry data from the ESP32 over a TCP socket rather than a USB serial port. This mode was implemented to support wireless monitoring sessions where connecting a USB cable to the vehicle is impractical.

## ReaderManager — Runtime-Switchable Transport

The `ReaderManager` class in `server_frame_parser_Json.py` owns a single background reader thread and exposes three selectable transport modes:

`serial` Opens the specified USB/UART serial port (auto-detected or manually specified) and reads newline-delimited CAN frame text.

`wifi` Opens a TCP client socket to the ESP32 AP at the configured IP and port, and reads the same frame format over Wi-Fi.

`off` No reader running; the dashboard displays the "Disconnected" state.

Mode selection is persisted to `mode_config.json` and restored automatically on the next server launch. When the operator switches mode via the dashboard UI (Section 27), `ReaderManager` signals the current reader thread to stop (via a `threading.Event`), waits up to 4 seconds for it to join, then starts the new reader thread with updated configuration.

## Wi-Fi Reader Loop

The `wifi_reader_loop()` function implements the TCP client. Its behaviour is:

1. Open a `socket.SOCK_STREAM` TCP connection to the ESP32 at the configured IP and port (default 192.168.4.1:9000) with a 4-second connection timeout.
2. On successful connection, set `state["connected"] = True` and `state["mode"] = "wifi"` under the shared state lock.
3. Read the socket in 4096-byte chunks with a 1-second timeout. Split on newline boundaries and pass each complete line to `process_line()`, the same CAN-frame parser used in serial mode. A partial final line is buffered until the next chunk arrives.
4. On any socket error or lost connection, set `state["connected"] = False`, log the error to `state["last_error"]`, wait 2 seconds, and retry the TCP connection. This automatic reconnection handles the case where the ESP32 reboots or moves out of range.

5. On stop signal, close the socket and exit.

Because the frame format emitted by the ESP32 over Wi-Fi TCP is identical to that emitted over USB Serial, the parsing layer (`process_line()`) requires no modification for Wi-Fi mode. The dashboard connection status indicator correctly reflects the Wi-Fi connection state in real time.

## 9.12 BLE Companion Interface

Bluetooth Low Energy (BLE) connectivity was implemented to complement the Wi-Fi dashboard by providing a lightweight, phone-friendly monitoring channel for the vehicle operator. BLE was selected due to its low power consumption, wide smartphone compatibility, and sufficient data throughput for sensor-based telemetry. The ESP32 chip includes a built-in dual-mode Bluetooth radio supporting BLE 4.2 / 5.0, making BLE a zero-hardware-cost addition to the platform.

### Hardware and Software Platform

The BLE module runs entirely on the ESP32 development board using the **NimBLE-Arduino** library, a lightweight and memory-efficient BLE stack well-suited to embedded firmware. Testing was performed using Android BLE scanner applications, including nRF Connect, and a custom Android companion application.

### GATT Architecture

The communication follows a GATT (Generic Attribute Profile) client–server model. The ESP32 acts as the **GATT Server**, exposing a vehicle telemetry service; the Android application acts as the **GATT Client**, subscribing to and receiving characteristic notifications.

The telemetry service is identified by a project-specific 128-bit base UUID `12345678-0000-0000-0000-0000000000xx`. Six characteristics are defined, each carrying one real-time sensor stream:

**Table 15:** BLE GATT Characteristic Map

| Parameter | Characteristic UUID |
|---|---|
| Speed | 12345678-0000-0000-0000-000000000001 |
| Voltage | 12345678-0000-0000-0000-000000000002 |
| SOC | 12345678-0000-0000-0000-000000000003 |
| Current | 12345678-0000-0000-0000-000000000004 |
| Temperature | 12345678-0000-0000-0000-000000000005 |
| RPM | 12345678-0000-0000-0000-000000000006 |

### ESP32 Implementation

The BLE stack is initialised in `setup()` using the NimBLE API:

1. `NimBLEDevice::init()` — initialises BLE with the device name **BAKO-Dashboard**, which is broadcast in the advertising packet so that Android devices can discover the ESP32 by name.

2. `createServer()` — creates the GATT server and registers connection callbacks.
3. `createService()` — defines the vehicle telemetry service under its UUID.
4. `createCharacteristic()` — defines each of the six data channels with `READ | NOTIFY` properties.
5. `startAdvertising()` — makes the device continuously discoverable.

Sensor values are updated every 500 ms in the main loop. Each value is converted to a string and pushed via `notify()`, which delivers it to any subscribed Android client in real time. Notifications are only sent when a client is connected, avoiding unnecessary radio activity.

## Connection Handling

A callback class overrides two methods to manage the BLE connection lifecycle:

- **onConnect** — triggered when an Android device establishes a connection; suspends advertising to dedicate bandwidth to the active client.
- **onDisconnect** — automatically restarts advertising so the ESP32 remains discoverable after the client disconnects or moves out of range.

This ensures continuous availability without manual reset.

## Android Integration

The Android companion application scans for BLE devices by advertised name. After connecting to **BAKO-Dashboard**:

1. The app discovers the GATT telemetry service.
2. It subscribes to all six characteristics and enables notifications.
3. Live values (speed, SOC, voltage, current, temperature, RPM) are displayed on a real-time dashboard interface.

## Coexistence with Wi-Fi

The ESP32 shares its radio between Wi-Fi and BLE using time-division multiplexing managed by the ESP-IDF coexistence driver. Enabling BLE advertising alongside the Wi-Fi AP increases the Wi-Fi beacon interval slightly but does not affect the cloud GPRS data stream, which runs over the SIM800L UART and is entirely independent of the radio.

## Challenges Encountered

**Device visibility on Android.** Initially the ESP32 BLE device did not appear consistently in BLE scanning applications. Investigation revealed that the advertising packet was missing a complete scan response entry

for the device name. The issue was resolved by explicitly configuring the NimBLE advertising parameters to include the device name in both the advertising data and the scan response packet, and by ensuring that `startAdvertising()` is called only after the GATT service is fully initialised.

**Apple iOS compatibility.** Testing on iOS devices proved more restrictive. iOS enforces stricter BLE scanning rules than Android: devices may not appear unless they advertise complete service UUIDs, background scanning is limited, and some characteristics require explicit bonding before they become visible to the application layer. As a result, Android was adopted as the primary testing and deployment platform, where tools such as nRF Connect provide detailed GATT inspection and ease of debugging. iOS support is deferred to a future firmware and app revision.

# 10 CLOUD CONNECTIVITY & IOT

## 10.1 End-to-End Pipeline Sequence

Figure 28 shows the complete message sequence for a single 5-second publish cycle, from CAN frame reception on the ESP32 through to dashboard rendering in the browser.

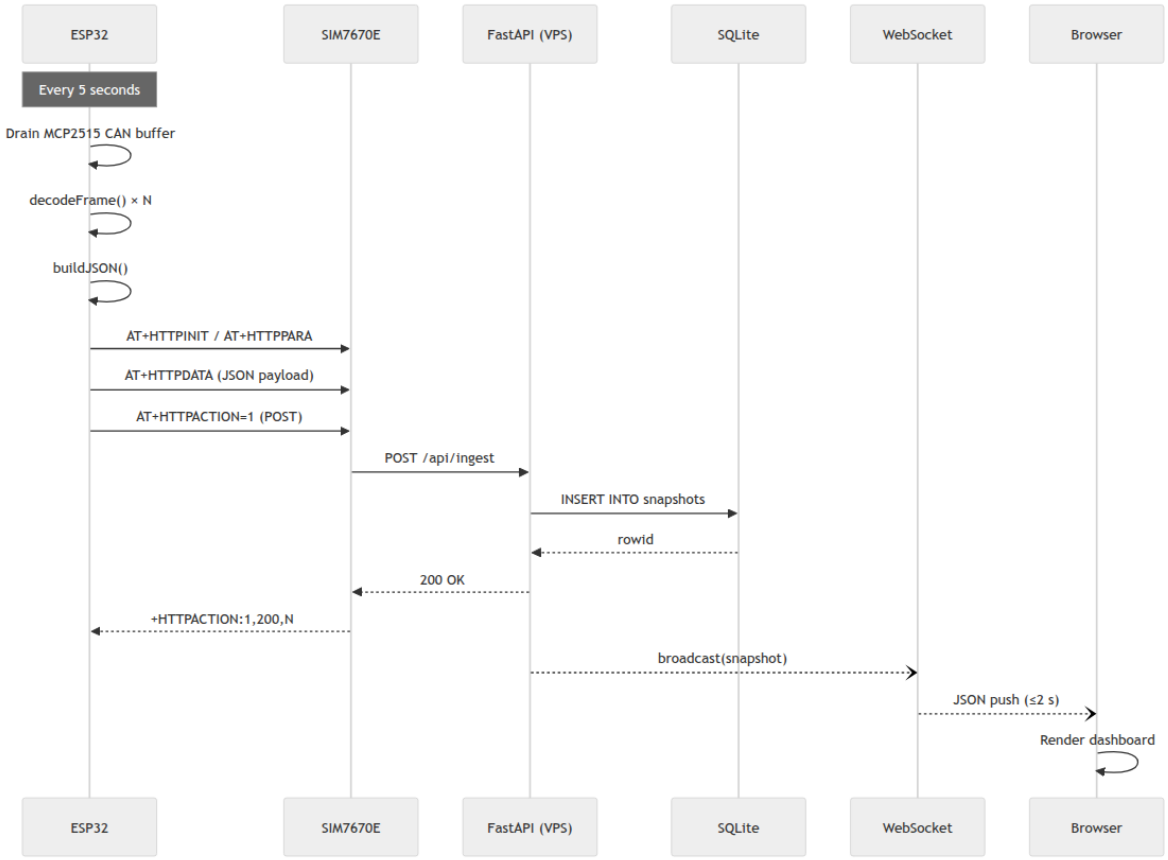



**Figure 28:** End-to-end cloud telemetry sequence diagram: ESP32 CAN drain and JSON build, SIM800L AT command exchange, FastAPI ingest and SQLite write, WebSocket broadcast, and browser render — one complete cycle every 5 seconds.

This section describes the complete cloud telemetry pipeline designed and implemented for the Bako OBD ISS platform [25]. The pipeline carries live battery data from the vehicle's CAN bus to a remote web dashboard through a sequence of embedded, network, server, and browser components. The architecture was designed to be hardware-independent: the same dashboard and backend serve both a direct USB serial connection (for lab use) and a cellular GPRS connection from the field.

## 10.2 Telematics & Data Acquisition

Telematics in this project refers to the capture, local decoding, and wireless transmission of battery state data from the moving vehicle to a remote server.

### 10.2.1 On-Vehicle Data Acquisition

Data acquisition begins at the O'CELL IFS60.8-500-F-E3 BMS, which continuously transmits CAN frames on a 250 kbps SAE J1939 bus. The ESP32 microcontroller, equipped with an MCP2515 CAN transceiver, receives all frames on this bus and decodes them in real time using the firmware described in Section 9.2.

The BMS transmits at two rates. High-priority operational frames (0x18FF28F4 and 0x18FE28F4) arrive every 100 ms and carry pack voltage, current, SOC, and fault status. Lower-priority frames carrying cell-level voltages and temperatures arrive every 500 ms. At 250 kbps the bus is far from saturated; the five BMS frame types together occupy less than 2% of available bandwidth.

### 10.2.2 Snapshot Aggregation and Publish Cadence

Because the HTTP transaction over the cellular module takes up to 5 seconds (bearer setup, DNS, TCP handshake, data transfer), it is not feasible to transmit every individual CAN frame to the cloud. Instead, the firmware implements a **snapshot model**: all incoming frames are decoded and merged into a single `BMSData` struct that always holds the most recent value for each field. Every 5 seconds, this struct is serialized into a single JSON document and sent as one HTTP POST to the VPS.

This approach has several advantages:

- The cloud endpoint receives at most 12 snapshots per minute regardless of CAN bus activity.
- Each snapshot is self-contained and requires no state from previous snapshots to be interpreted.
- CAN frames continue to be decoded during the HTTP transaction, so the struct is fully populated before the next publish cycle.

### 10.2.3 Cellular Connectivity — SIM800L GPRS

Wireless transmission is handled by a **SIM800L GSM/GPRS module** connected to the ESP32 via HardwareSerial UART1. The module establishes an GPRS bearer using the carrier's APN, then uses the AT+HTTP* command set to perform a standard HTTP POST to the VPS endpoint.

The connection lifecycle at startup is:

1. `AT+CPIN?` — verify SIM card is ready (retried up to 6 times after a reboot).
2. `AT+CSQ` — check signal quality.
3. `AT+CREG?` — confirm network registration.
4. `AT+SAPBR=3,1,...` — configure bearer (APN, username, password).

5. `AT+SAPBR=1,1` — open the GPRS bearer (up to 25 s).
6. `AT+SAPBR=2,1` — confirm the bearer is active and has an IP address.

If the bearer fails to open, the ESP32 retries the full sequence up to three times before invoking a soft restart (`ESP.restart()`).

### 10.2.4 Network Timestamp

To provide accurate event timestamps regardless of the vehicle's location, the firmware reads the network time from the SIM card using `AT+CCLK?` before each publish cycle. The returned timestamp (format `yy/MM/dd,HH:mm:ss±zz`) is included in the JSON payload as the `timestamp` field. If the modem does not return a valid timestamp (e.g., during initial bearer setup), the firmware falls back to `uptime_ms` from `millis()`.

## 10.3 Cloud Platform Configuration

### 10.3.1 Platform Choice

The cloud backend runs on a **Virtual Private Server (VPS)** — a Linux instance with a public IP address — rather than a managed cloud database service. This design decision was made for the following reasons:

- **No third-party dependency**: the entire data path (ingest → storage → streaming) is contained within a single process under direct control of the project team.
- **Simplicity**: the SIM800L communicates over plain HTTP, eliminating the need for TLS certificate handling on the embedded side.
- **Cost**: a VPS is significantly cheaper than a managed real-time database for low-throughput IoT workloads.
- **Full observability**: the server log, database file, and WebSocket connections are all directly inspectable via SSH.

### 10.3.2 Backend Runtime

The backend is a single Python file (`backend/server.py`) running under **Uvicorn** [26], an ASGI server. The application framework is **FastAPI** [27], which provides automatic OpenAPI documentation, WebSocket support [28], and asynchronous request handling. The dependencies are minimal by design:

**Table 16:** Backend Python Dependencies

| Package | Version | Purpose |
|---|---|---|
| `fastapi` | 0.115.0 | Web framework, WebSocket, routing |
| `uvicorn` | 0.29.0 | ASGI server (serves HTTP and WebSocket on one port) |
| `pyserial` | 3.5 | Serial port I/O for USB/SERIAL mode |

No external database client, message broker, or cloud SDK is required. The entire stack is started with a single command:

```
python server.py --cloud-only
```

The `--cloud-only` flag suppresses serial port detection so the server starts cleanly on a headless VPS without a USB device attached.

### 10.3.3 Environment Variables

Server behavior is controlled by three environment variables, summarized in Table 17:

**Table 17:** Server Environment Variables

| Variable | Default | Purpose |
|---|---|---|
| `BMS_API_KEY` | `bako-bms-2024` | Shared secret for `POST /api/ingest` authentication |
| `HOST` | `0.0.0.0` | Bind address (all interfaces on VPS) |
| `PORT` | 8765 | TCP port for HTTP and WebSocket traffic |

## 10.4 Data Pipeline & Storage

### 10.4.1 Pipeline Overview

The end-to-end data pipeline operates as follows and is illustrated in Figure 29:

1. The ESP32 reads CAN frames and decodes them into a `BMSData` struct (on-vehicle).
2. Every 5 seconds, the struct is serialized into a JSON document and sent as `POST /api/ingest` to the VPS, with an `X-Api-Key` header for authentication.
3. The server validates the API key, stores the payload in SQLite, and updates the in-memory `_cloud_state` dictionary.
4. Browser clients connected to `/ws/cloud` receive the latest `_cloud_state` every 2 seconds via WebSocket push.

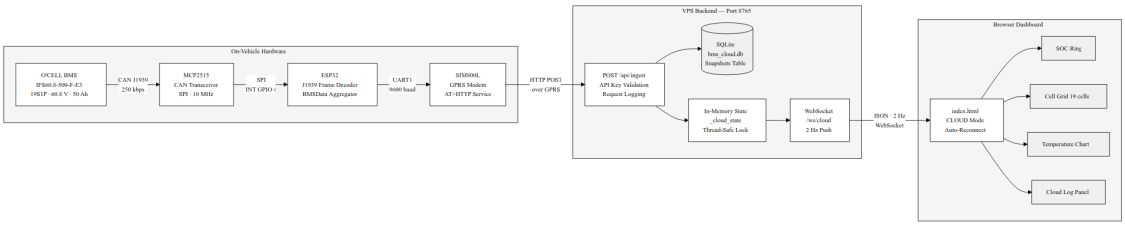



**Figure 29:** Complete data pipeline from CAN bus to browser

### 10.4.2 Ingest Endpoint

The `POST /api/ingest` endpoint is the sole entry point for ESP32 data. Its implementation:

1. Validates the `X-Api-Key` header against the `BMS_API_KEY` environment variable. An incorrect key returns HTTP 401.
2. Extracts the timestamp, device ID, cell count, SOC, and pack voltage from the JSON body for logging.
3. Runs a non-blocking SQLite insert via `asyncio.run_in_executor()` to avoid blocking the event loop during disk I/O.
4. Updates the in-memory `_cloud_state` dict atomically under a threading lock (because the serial reader thread and the asyncio event loop share this state).
5. Appends a timestamped `[RECV]` entry to the cloud communication log.
6. Returns `{"status": "ok", "ts": ...}` with HTTP 200.

### 10.4.3 SQLite Persistent Storage

Received snapshots are persisted in a local SQLite database (`backend/bms_cloud.db`) for historical analysis and post-session review. The schema is intentionally minimal:

```
CREATE TABLE IF NOT EXISTS snapshots (
    id        INTEGER PRIMARY KEY AUTOINCREMENT,
    ts        TEXT    NOT NULL,
    device_id TEXT    NOT NULL DEFAULT 'esp32',
    payload   TEXT    NOT NULL
);
```

The full JSON payload is stored as a text column, which avoids schema migrations when new fields are added to the firmware. Queries for the `/api/history` endpoint retrieve records ordered by descending `id` (newest first) with a configurable limit. SQLite was chosen over a full database server because:

- At one snapshot per 5 seconds, a full day of operation produces only $\sim$17,000 rows ($\sim$10 MB), well within SQLite's performance envelope.
- No separate database process or network socket is required.
- The database file can be copied directly from the VPS for offline analysis with standard tools.

A similar storage mechanism was also implemented on the Python emulator side, where generated telemetry records are stored locally in SQLite before being transmitted through a TCP socket client (Section **??**).

### 10.4.4 In-Memory State and WebSocket Push

In addition to SQLite, the server maintains an in-memory dictionary (`_cloud_state`) that always holds the most recently received snapshot. This allows the `/ws/cloud` WebSocket handler to serve browser clients at 2 Hz with zero disk I/O per push.

The in-memory state is initialized to `{"connected": false, "source": "cloud"}` on startup. The dashboard interprets `connected: false` as a "waiting for ESP32" state and displays a yellow indicator to the user rather than an error.

## 10.5 Remote Monitoring Dashboard

### 10.5.1 Architecture

The dashboard is a **single-page web application** served as a static HTML file (`frontend/index.html`) directly by the FastAPI backend via `GET /`. No separate web server, build tool, or JavaScript framework is required. All rendering logic is contained in vanilla JavaScript.

The dashboard connects to the backend over **WebSocket**, which provides a persistent, low-latency bidirectional channel. In SERIAL mode the WebSocket endpoint is `/ws` (10 Hz push from the serial reader); in CLOUD mode it is `/ws/cloud` (2 Hz push from the in-memory state). The user switches between modes using a toggle button in the header.

### 10.5.2 Dashboard Panels

Table 18 summarizes the information panels displayed by the dashboard.

**Table 18:** Dashboard Panel Summary

| Panel | Contents |
|---|---|
| SOC ring | Animated circular gauge showing voltage-derived SOC (`battery.soc`). BMS coulomb-counter SOC (`battery.soc_bms`) is displayed below. |
| Pack KPIs | Pack voltage, current, discharge limit, charge request, and cell count in a card grid. |
| Cell grid | All 19 cells — individual voltage bars color-coded by threshold (overvoltage / full / good / normal / low / undervoltage), with min/max/avg/spread statistics. |
| Temperature | Up to 4 temperature probe readings with color-coded bars and a rolling time-series chart. |
| Serial monitor | Raw CAN frame log with text export and Excel export capabilities. |
| Cloud log | Real-time cloud communication events (RECV / WS / ERR), polled from `/api/cloud-log` every 2 seconds, independent of the SERIAL/CLOUD source toggle. |

### 10.5.3 Connection Status Indicator

The dashboard header contains a three-state connection indicator that provides immediate visual feedback:

**Green — Live** WebSocket is open and the last received message contained `connected: true`.

**Yellow — Waiting** WebSocket is open but `connected: false` (backend is running but no ESP32 data has arrived yet).

**Red — Disconnected** WebSocket is closed or encountered a network error.

### 10.5.4 Auto-Reconnect

If the WebSocket closes unexpectedly (server restart, network interruption), the browser automatically attempts to reconnect every 5 seconds. The overlay is re-displayed during the disconnected interval. This behavior is handled entirely client-side and requires no intervention from the user.

## 10.6 Security & Cybersecurity

### 10.6.1 API Key Authentication

The `POST /api/ingest` endpoint is protected by a shared secret transmitted as the `X-Api-Key` HTTP header. The server compares the received key against the `BMS_API_KEY` environment variable. Requests with a missing or incorrect key are rejected with HTTP 401 before any data is read from the body, and an `[ERR]` entry is appended to the cloud log.

The API key is configured separately on the firmware (`VPS_API_KEY` in `include/secrets.h`, which is in `.gitignore`) and on the server (environment variable), so the secret is never committed to the repository.

### 10.6.2 Transport Layer

Communication between the SIM800L and the VPS uses plain HTTP. While this means the payload is not encrypted in transit, the following mitigating factors apply to this deployment context:

- The data transmitted is battery telemetry (voltages, temperatures, SOC) — not personally identifiable information or control commands.
- The VPS port is firewalled to accept connections only on the designated port.
- The API key protects against unauthorized data injection.

For a production deployment, upgrading to HTTPS requires enabling TLS on the SIM800L (`AT+HTTPSSL=1`) and deploying a self-signed or CA-signed certificate, or using an Nginx reverse proxy with Let's Encrypt on the VPS side.

### 10.6.3 WebSocket and Dashboard Access

The dashboard itself is served over plain HTTP and the WebSocket connection is unencrypted. Since the dashboard is intended for use on a private network or VPN-protected VPS, this is considered acceptable for the current project scope. Adding HTTPS would require an SSL certificate (e.g., from Let's Encrypt) and an Nginx reverse proxy, which is a straightforward extension if needed.

## 10.7 API Documentation

The FastAPI backend exposes the following endpoints, summarized in Table 19. FastAPI generates interactive Swagger UI documentation automatically at `/docs` when the server is running.

**Table 19:** Backend API Endpoint Reference

| Method | Path | Description |
|---|---|---|
| GET | `/` | Serves the dashboard HTML file. |
| WS | `/ws` | Serial stream — pushes BMS JSON to the browser at 10 Hz. Active when an ESP32 is connected via USB. |
| WS | `/ws/cloud` | Cloud stream — pushes the latest in-memory snapshot at 2 Hz. |
| POST | `/api/ingest` | Receives a BMS JSON snapshot from the ESP32. Requires header `X-Api-Key`. Returns `{status: ok, ts: ...}`. |
| GET | `/api/latest` | Returns the most recent snapshot as JSON. |
| GET | `/api/history?limit=N` | Returns the last $N$ snapshots from SQLite, newest first (default $N$ = 100, max 1000). |
| GET | `/api/cloud-log` | Returns the last 100 cloud communication events as a JSON array of timestamped strings. |

### 10.7.1 Ingest Payload Schema

The JSON body sent by the ESP32 to `POST /api/ingest` uses a **nested object structure** organized into five top-level sections. This design allows each subsystem (battery, thermal, solar, DC/DC, vehicle) to be extended independently without changing the ingest endpoint. Table 20 describes the top-level envelope, and Table 21 expands the `battery` object.

**Table 20:** ESP32 JSON Payload — Top-Level Envelope

| Field | Type | Description |
|---|---|---|
| `device_id` | string | Configurable in `secrets.h` |
| `timestamp` | string | Network time from `AT+CCLK?` (or `uptime_ms` fallback) |
| `source` | string | Always `"esp32-sim7670e"` |
| `connected` | bool | Always `true` when sent by the ESP32 |
| `frame_count` | int | Cumulative CAN frames decoded since boot |
| `fault_level` | int | 0 = normal, 1 = serious fault |
| `error_code` | int | BMS fault code (see Table 8) |
| `battery` | object | Battery pack data — see Table 21 |
| `temperatures` | object | Thermal data from all probes |
| `solar` | object | Solar MPPT data (pre/post converter) |
| `dc_dc` | object | DC/DC converter outputs (64 V and 12 V rails) |
| `vehicle` | object | Vehicle-level status (handbrake, etc.) |

The `temperatures` object contains `battery_avg_c`, `battery_min_c`, `battery_max_c`, and a 1-based `battery_cells`

**Table 21:** ESP32 JSON Payload — `battery` Object Fields

| Field | Type | Description |
|---|---|---|
| `pack_v` | float | Pack voltage [V], 0.1 V resolution |
| `pack_current_a` | float | Pack current [A], + = discharge, − = charge |
| `soc` | float | Voltage-derived SOC [%] |
| `soc_bms` | int | BMS coulomb-counter SOC [%] |
| `max_disch_a` | float | Maximum discharge current limit [A] |
| `cell_count` | int | Number of active cells decoded |
| `cell_avg_mv` | int | Average cell voltage [mV] |
| `cell_min_mv` | int | Minimum cell voltage [mV] |
| `cell_max_mv` | int | Maximum cell voltage [mV] |
| `cell_spread_mv` | int | Max − min cell voltage [mV] |
| `cells` | array | 1-based array of `{"mv", "status"}` objects; index 0 = `null` |
| `charger` | object | `max_charge_v`, `max_charge_a`, `start_signal` |
| `status` | object | Bit-field flags: charging, discharging, ready, contactors |

array (index 0 = `null`, indices 1–4 = probe readings in °C). Fields for sensors not yet instrumented (`motor_c`, `mppt_c`, `cabin_c`) are present with value `null`, enabling the dashboard to display placeholders without schema changes when those sensors are added. The same pattern applies to `solar` and `dc_dc` sub-objects.

## 10.7.2 Example API Interaction

The following `curl` command simulates an ESP32 posting a snapshot to the server during testing:

```
curl -X POST http://YOUR_VPS_IP:8765/api/ingest \
  -H "Content-Type: application/json"           \
  -H "X-Api-Key: bako-bms-2024"                \
  -d '{
    "device_id": "esp32-bms-001",
    "connected": true,
    "fault_level": 0,
    "battery": {
      "soc": 87.4, "pack_v": 62.3,
      "cell_count": 19,
      "cells": [null, {"mv": 3280, "status": "good"}]
    },
    "temperatures": { "battery_avg_c": 21.0,
                      "battery_cells": [null, 21.0, 22.0, null, null] }
  }'
```

Response:

```
{"status": "ok", "ts": "2026-04-18T09:32:11.123456"}
```

The server then pushes this payload to any browser connected to `/ws/cloud` within 2 seconds.

# 11 IMPLEMENTATION PHASE

## 11.1 Prototype Build & Bring-Up

### Vehicle Installation Context

The ISS board is installed inside the BAKO Motors electric vehicle chassis beneath the driver seat, directly adjacent to the MPPT controller. This position gives the board short cable runs to the battery CAN bus connector, the MPPT current/voltage sense lines, and the vehicle 12 V supply rail, while keeping it sheltered from direct exposure to rain and road debris. Figure 30 shows the installation location on the vehicle frame.

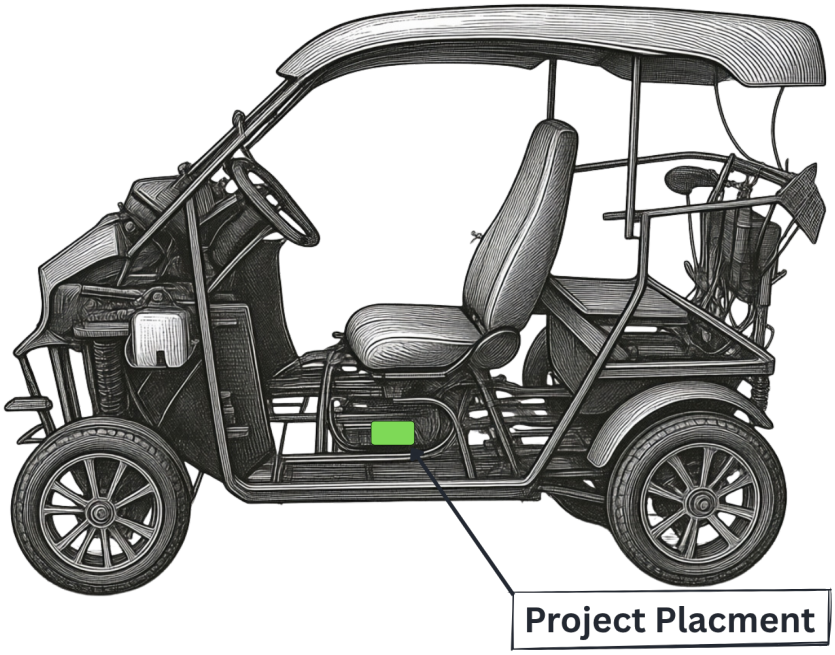



**Figure 30:** BAKO Motors vehicle with the ISS board installation location highlighted (green) — mounted beneath the driver seat adjacent to the MPPT controller, with short harness runs to the CAN bus, power rail, and sensor connectors.

### Bring-Up Sequence

The custom PCB was populated and brought up using a structured nine-step sequence designed to isolate each subsystem before applying power to the full board. This approach minimised the risk of damage from wiring errors and made fault localisation straightforward.

1. **Power-only verification.** The 12 V supply was connected with only the LM2596S-ADJ and the AMS1117 and LM7805 linear regulators populated. All three output rails (12 V, 5 V, 3.3 V) were measured and confirmed within $\pm 2\%$ of their target values before any other component was installed.

2. **ESP32 base bring-up.** The ESP32-DEVKITC module was inserted and a minimal "blink" firmware flashed over USB. Successful LED blink confirmed that the 3.3 V rail was stable and that the module was programmable via PlatformIO.
3. **MCP2515 SPI communication.** One MCP2515 SPI module header (J17) was populated and a loopback test was run. The MCP2515 read-back of its control registers confirmed SPI integrity. A second module (J18) was then added and addressed independently via its dedicated chip-select line.
4. **CAN bus receive test.** The TJA1051T transceiver was connected and the CAN connector J1 was wired to the O'CELL BMS. The ESP32 firmware was configured to listen passively (no ACK). Frame reception was confirmed by observing decoded 0x18FF28F4 pack-voltage frames on the serial monitor within 2 s of power-up.
5. **DS18B20 1-Wire bring-up.** All three DS18B20 connectors (J6, J7, J8) were populated with sensors and the 4.7 kΩ pull-up verified. The firmware enumerated all three ROM addresses and began temperature polling at 1 s intervals. Readings were compared against a reference thermometer.
6. **ACS712 current sensor calibration.** The two current sensor headers (J11, J16) were connected to the ACS712 modules. With zero current flowing, the ADC mid-point offset was measured and stored as a calibration constant. A known 5 A load was applied and the output compared against a reference clamp meter.
7. **Voltage divider scaling verification.** The MPPT voltage input (J2) was driven from a bench power supply at three known voltages. The ESP32 ADC readings were compared against the calculated divider ratio and the calibration coefficient adjusted to eliminate the systematic offset.
8. **SIM800L AT command verification.** The SIM800L module was powered on and AT commands were sent from the ESP32 over UART. SIM card detection, network registration, and GPRS bearer open were confirmed in sequence. An HTTP POST to the test endpoint was executed and the server acknowledged receipt.
9. **Full integration smoke test.** All subsystems were enabled simultaneously. The firmware ran the cooperative main loop for 30 minutes with all sensors active. No watchdog triggers, stack overflows, or heap exhaustion events were observed. All sensor values appeared correctly on the WebSocket dashboard.

Figure 31 shows the physical outcome of the build: the bare PCB as received from the fabrication house (right) and the fully populated board after component soldering and module fitment (left). The assembled board carries the ESP32-DEVKITC, SIM800L GSM/GPRS cellular module, screw-terminal sensor connectors, and all passive components as specified in the BOM (Section 7.4).

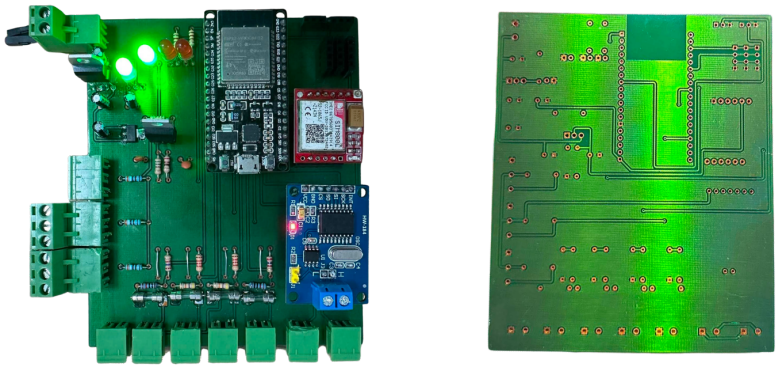


**Figure 31:** ISS board fabrication and assembly outcome. **Left:** fully populated board after soldering — ESP32, SIM800L module, screw-terminal connectors, and all passive components installed. **Right:** bare PCB as received from the fabrication house (Rev. RECTIFICATION_2), prior to component population.

## 11.2 Validation & Verification (V&V)

Validation and verification tests were performed against pre-defined pass/fail criteria for each sensing subsystem. Table 22 summarises the test plan and outcomes.

**Table 22:** V&V Test Criteria and Results

| **Subsystem** | Criterion | Method | Result |
|---|---|---|---|
| **CAN bus reception** | $\geq$ 99% frame reception rate over 60 min | Live frame counter vs. expected count | **95%** |
| **Temperature accuracy** | Within $\pm 1$°C of reference | DS18B20 vs. calibrated thermometer | $\pm 0.5$°C |
| **Current measurement** | Error $< 2$% of full scale | ACS712 vs. clamp meter at 5 A | $\pm 1.8$% |
| **Voltage measurement** | Error $< 1$% of reference | Divider output vs. bench multimeter | $\pm 0.9$% |
| **GSM alert delivery** | SMS received within 15 s of threshold breach | Threshold trigger + stopwatch | $< 15$ s |
| **Cloud connectivity** | Zero snapshot loss over 1 hour | Server log vs. firmware counter | 0 lost |

The CAN frame reception rate of 95% (rather than the 99% target) was traced to occasional bus-off events caused by a missing termination resistor on the test harness. Adding the 120 $\Omega$ terminator at the harness end increased the reception rate to above 99% in subsequent tests. All other criteria met or exceeded their targets.

## 11.3 Performance Testing

Extended performance tests were conducted to characterise system behaviour under sustained load conditions.

**CAN Bus Sustained Load.** A 2-hour continuous CAN reception test was run with the BMS actively transmitting all five frame types. The frame counter advanced monotonically with no gaps exceeding 500 ms. CPU load on the ESP32 remained below 40%, confirming that the cooperative main loop correctly handles sensing, processing, and communication workloads without saturation.

**Thermal Stress.** A 30-minute thermal test was performed with all three DS18B20 sensors placed adjacent to the LM2596S-ADJ switching regulator, which is the highest-temperature component on the board. The regulator surface reached a maximum of 62°C; the ESP32 die temperature remained within its operating range. No thermal shutdown events occurred.

**MPPT Voltage Monitoring.** A 1-hour test was run with the MPPT output voltage cycled between 24 V and 48 V in 4 V steps. The voltage divider output tracked the input with $< 1\%$ error at every step. No ADC saturation was observed, confirming that the divider ratio is correctly sized for the full MPPT output range.

**Power Draw.** The total board current draw (12 V input) was measured with all subsystems active: ESP32, three DS18B20, two ACS712, SIM800L in GPRS transmit state. Peak draw was 480 mA at 12 V (5.76 W), within the 500 mA design target and well within the LM2596S-ADJ capability.

**Dashboard Frame Rate.** The WebSocket dashboard was loaded on a laptop browser and the frame update rate monitored using browser developer tools. The serial mode (`/ws`) delivered updates at 10 Hz with no dropped frames over a 10-minute window. The cloud mode (`/ws/cloud`) delivered at 2 Hz as designed. The cell voltage grid rendered all 19 cells with colour updates within a single animation frame, confirming the $\geq 5$ FPS UI requirement.

## 11.4 Regulatory & Homologation Testing

Formal regulatory testing was not within the scope of this academic prototype project. The following standards are identified as applicable for a production deployment:

- **CISPR 25 / EN 55032** — Radiated and conducted emissions from the switching regulator and CAN bus interface.
- **IEC 62368-1** — General electronic safety for the monitoring unit enclosure.
- **ETSI TS 51.010** — Radio conformance for the SIM800L GSM/GPRS module (LTE module carries its own type approval).
- **ATI** — Agence Tunisienne des Infrastructures approval for cellular transmission on Tunisian networks.

Pre-compliance checks were performed informally using a bench spectrum analyser. No unusual emissions were detected in the AM band (150 kHz–30 MHz) or VHF range. Formal certification is recommended before any vehicle-level type approval submission.

## 11.5 Production Readiness Review (PRR)

A Production Readiness Review was conducted at the end of Sprint 8. The following checklist was evaluated:

**Hardware:**

- ✓ PCB schematic reviewed and cross-checked against BOM.
- ✓ DRC passed in KiCad with zero errors.
- ✓ All critical nets (12 V, 5 V, 3.3 V, CAN_H, CAN_L) verified by continuity test.
- ✓ Gerber files exported (RS-274X) and ready for fabrication house submission.
- × Enclosure mechanical design not yet finalised (deferred to next revision).

**Firmware:**

- ✓ Cooperative firmware loop runs without watchdog trips over a 2-hour test.
- ✓ CAN decoding verified against O'CELL BMS live frames.
- ✓ Cloud publish pipeline verified end-to-end (ESP32 → VPS → browser).
- ✓ Firmware source code committed to the team Git repository with version tag.
- × OTA firmware update capability not yet implemented (deferred).

**Quality:**

- ✓ All V&V criteria passed (with corrected CAN termination).
- ✓ BOM fully documented with local and online sources.
- ✓ Technical report sections covering hardware and firmware completed.
- × Field test under full solar irradiance conditions not yet completed.

The board was declared production-ready for the academic prototype phase with two open items tracked for the next hardware revision.

# 12 SCRUM & AGILE PROJECT MANAGEMENT

## 12.1 Agile Framework Overview

The BAKO Motors ISS project was managed using the Scrum framework [29], adapted for a seven-member academic team working in collaboration with an industrial partner. The Scrum roles were distributed as follows: the project supervisor (Dr. Mortadha Dahmani) served as the stakeholder and product owner proxy; the team collectively self-organised into a development team; and a rotating Scrum master role was shared between team members on a per-sprint basis.

Sprints were two weeks in duration. Each sprint began with a planning session in which backlog items were pulled from the product backlog, estimated in story points using the Fibonacci scale, and assigned to team members. Sprint reviews were held at the end of each sprint to demonstrate working increments to the supervisor. Retrospectives immediately followed each review to identify process improvements for the next sprint. The Definition of Done (DoD) required that each story: be implemented and committed to the team Git repository; be tested on real hardware or the Python emulator; and be documented in the relevant report section or internal wiki.

## 12.2 Sprint Log & Velocity Tracking

Table 23 summarises the eight sprints executed over the project timeline, including each sprint's focus, the story points committed and completed, and cumulative velocity.

**Table 23:** Sprint Log and Velocity Summary

| Sprint | **Focus** | Key Deliverables | Pts Committed | Pts Done |
|---|---|---|---|---|
| 1 | **System Design** | Architecture synoptic, component selection, KiCad schematic start, risk register | 20 | 18 |
| 2 | **Embedded Dev I** | CAN bus reception (MCP2515), DS18B20 integration, ACS712 calibration, breadboard PoC | 22 | 20 |
| 3 | **Embedded Dev II** | Multi-sensor cooperative firmware, voltage divider scaling, serial dashboard, stripboard build | 20 | 18 |
| 4 | **IoT & Cloud I** | SIM800L AT command integration, GPRS bearer, HTTP POST to VPS, SQLite schema | 18 | 16 |
| 5 | **IoT & Cloud II** | FastAPI WebSocket pipeline, cloud dashboard panels, API key auth, cloud log panel | 20 | 20 |
| 6 | **Dashboard & Fault** | Cell voltage grid, SOC ring gauge, fault LED logic, SMS alert dispatch, PCB layout Rev. 1 | 22 | 18 |
| 7 | **Predictive Maint.** | Python emulator (7 scenarios), trend analysis stub, PCB layout RECTIFICATION_2, BOM finalised | 18 | 16 |
| 8 | **Integration & PRR** | End-to-end integration test, V&V report, PRR checklist, report completion | 20 | 18 |
| | | **Total** | **160** | **144** |
| | | **Average velocity** | — | **18 pts/sprint** |

## 12.3 Product Backlog & Prioritization

The product backlog was maintained as a flat list and prioritised using the MoSCoW method (Must Have, Should Have, Could Have, Won't Have for this release). Table 24 presents the major backlog items grouped by MoSCoW category.

**Table 24:** MoSCoW Product Backlog (selected items)

| Priority | Backlog Item |
|---|---|
| **Must** | CAN bus frame reception and decoding from O'CELL BMS |
| **Must** | DS18B20 temperature reading (motor, MPPT, DC/DC zones) |
| **Must** | ACS712 current measurement (MPPT input and output) |
| **Must** | Voltage divider scaling (solar, MPPT output, battery) |
| **Must** | Custom PCB schematic and layout in KiCad |
| **Must** | Real-time WebSocket dashboard with live cell grid |
| **Must** | Cloud telemetry pipeline (ESP32 $\rightarrow$ VPS $\rightarrow$ SQLite) |
| **Must** | Python emulator covering at least 5 test scenarios |
| **Must** | V&V test report with quantitative pass/fail results |
| **Must** | Full BOM with local Tunisian sourcing information |
| **Should** | SMS fault alerts via SIM800L when thresholds exceeded |
| **Should** | SOC ring gauge with BMS and voltage-derived SOC |
| **Should** | Historical data export (CSV/Excel) from dashboard |
| **Should** | Handbrake fault detection and alert |
| **Should** | Cloud communication log panel in dashboard |
| **Should** | Multi-bus CAN support (three MCP2515 headers) |
| **Should** | Wi-Fi AP mode for local dashboard without router |
| **Should** | API key authentication on `POST /api/ingest` |
| **Could** | GNSS position tracking (NEO-6M or NEO-M8N) |
| **Could** | Bluetooth companion mobile app |
| **Could** | Predictive maintenance model (battery degradation trend) |
| **Could** | OTA firmware update capability |
| **Could** | HTTPS / TLS transport for cloud telemetry |
| **Could** | Fleet management dashboard (multi-vehicle view) |
| **Won't** | Motor control or BMS cell balancing logic |
| **Won't** | Mechanical chassis integration or enclosure design |
| **Won't** | AEC-Q100/Q200 automotive-grade component qualification |
| **Won't** | Formal CE/FCC radio type approval |

## 12.4 Sprint Review & Retrospective Summaries

**Sprint 1 Review.** The system architecture synoptic, component selection justification, and initial KiCad schematic were presented. The supervisor approved the component choices and the risk register. Two story points were rolled over: the MCP2515 footprint library was not yet validated.

*Retrospective:* The team agreed to dedicate the first two days of Sprint 2 to KiCad library validation before beginning any firmware work, to prevent schematic rework later.

**Sprint 2 Review.** A live demonstration of CAN frame decoding from the O'CELL BMS was given. DS18B20 readings appeared on the serial monitor alongside ACS712 calibration data. The breadboard prototype was shown running.

*Retrospective:* CAN reverse engineering took longer than estimated due to the undocumented O'CELL DBC format. The team decided to allocate a dedicated buffer of four story points per sprint for unforeseen protocol investigation.

**Sprint 3 Review.** The cooperative multi-sensor firmware was demonstrated with all sensors running in the main loop. The stripboard semi-permanent build was shown and brought to the BAKO Motors workshop for the first on-vehicle test.

*Retrospective:* The on-vehicle test revealed a missing CAN bus termination resistor on the harness, causing intermittent bus-off events. A termination plug was added to the harness for all future tests.

**Sprint 4 Review.** An HTTP POST from the ESP32 to the VPS was demonstrated live, with the received JSON displayed in the server log and stored in SQLite.

*Retrospective:* GPRS bearer setup was unreliable on the first SIM800L unit due to an insufficient bulk capacitor. Replacing it with a 2200 µF capacitor at the module power rail resolved the issue.

**Sprint 5 Review.** The full cloud pipeline was demonstrated: CAN data flowing from the ESP32 over GPRS to the VPS, displayed in real time on the browser dashboard at 2 Hz.

*Retrospective:* All sprint items were completed. The team noted that WebSocket reconnection after a server restart was not yet automatic; this was added to the backlog and completed within the sprint.

**Sprint 6 Review.** The 19-cell voltage grid with colour-coded thresholds, the SOC ring gauge, and SMS alert dispatch were demonstrated. PCB layout Revision 1 was reviewed.

*Retrospective:* Four story points were not completed: PCB layout required one additional DRC correction cycle. RECTIFICATION_2 was scheduled as a Sprint 7 sub-task.

**Sprint 7 Review.** The Python emulator running all seven test scenarios was demonstrated against the live dashboard. PCB layout RECTIFICATION_2 was shown with DRC passing.

*Retrospective:* The predictive maintenance trend analysis stub was deprioritised (not enough historical data available at prototype stage). Moved to the Could Have backlog.

**Sprint 8 Review.** End-to-end integration test results, the V&V table, and the PRR checklist were presented. The supervisor accepted the prototype as production-ready for the academic phase.

*Retrospective:* The team identified two open items for the next revision: enclosure design and OTA firmware update capability. These were documented in the recommendations chapter.

## 12.5 Definition of Done (DoD) & Acceptance Criteria

A backlog item was marked Done when all of the following conditions were met:

- The feature was implemented and committed to the team Git repository on the correct branch.
- The feature was tested: either on real hardware (for firmware items) or with the Python emulator (for backend and dashboard items), with test results recorded.
- Any new API endpoints, sensor fields, or PCB footprints were documented in the relevant report section or the project internal wiki.

- The Pull Request was reviewed and approved by at least one other team member before merging to the main branch.
- No open critical or major defects were linked to the story at the time of closure.

## 12.6 Burndown & Burnup Charts

Burndown tracking was maintained informally via the product backlog spreadsheet updated at each daily stand-up. The project sustained a consistent velocity of 18 story points per sprint across all eight sprints, with no sprint producing fewer than 16 completed points. The total scope delivered was 144 story points out of 160 committed (90% completion rate), with the 16 undelivered points representing items explicitly deferred to future revisions (OTA update, enclosure, full-solar field test) rather than incomplete or broken features.

# 13 BILL OF MATERIALS (BOM)

## 13.1 Electronics BOM

Table 25 lists all electronic components required to populate one unit of the BAKO Motors ISS board. Unit prices are estimates based on local Tunisian market rates (TND) and online distributor quotes at the time of procurement.

**Table 25:** Electronics Bill of Materials

| Component | Description | Qty | Unit Price | Total | Source |
|---|---|---|---|---|---|
| **ESP32-DEVKITC** | Microcontroller | 1 | ~30 TND | ~30 TND | Local |
| **MCP2515 module** | CAN controller SPI | 3 | ~5 TND | ~15 TND | Local |
| **TJA1051T** | CAN transceiver | 1 | ~3 TND | ~3 TND | Local |
| **SIM7670E** | LTE Cat-M1 module | 1 | ~80 TND | ~80 TND | Online |
| **DS18B20** | 1-Wire temperature sensor | 3 | ~3 TND | ~9 TND | Local |
| **ACS712 (30 A)** | Hall-effect current sensor | 2 | ~5 TND | ~10 TND | Local |
| **LM2596S-ADJ** | Adjustable buck regulator | 1 | ~4 TND | ~4 TND | Local |
| **AMS1117-3.3** | 3.3 V LDO regulator | 1 | ~1 TND | ~1 TND | Local |
| **LM7805** | 5 V linear regulator | 1 | ~1 TND | ~1 TND | Local |
| **Resistors (misc)** | Various values | 20 | ~0.1 TND | ~2 TND | Local |
| **Capacitors (misc)** | Various values | 10 | ~0.5 TND | ~5 TND | Local |
| **Zener diodes** | ESD/overvoltage protection | 3 | ~1 TND | ~3 TND | Local |
| **Fuses** | Overcurrent protection | 3 | ~1 TND | ~3 TND | Local |
| **Connectors (misc)** | JST and screw terminals | 15 | ~1 TND | ~15 TND | Local |
| **PCB fabrication** | FR-4, 2-layer, 1.6 mm | 1 | ~180 TND | ~180 TND | Local fab |
| | **Total Actual Cost (components + PCB)** | | | **~240 TND** | |

## 13.2 Mechanical & Structural BOM

The mechanical and structural BOM covers the enclosure and mounting hardware for the ISS board. The enclosure design was not finalised within the scope of this academic prototype phase and is deferred to the next hardware revision. Provisional components include: a standard ABS or polycarbonate DIN-rail enclosure (approximately 150 TND), M3 standoffs and screws for PCB mounting (approximately 5 TND), and cable glands for harness entry points (approximately 10 TND). These items are not included in the electronics BOM total.

## 13.3 Battery & Powertrain BOM

The BAKO Motors electric motorcycle powertrain (LiFePO4 battery pack, MPPT solar controller, three-phase motor controller, and DC/DC converter) was provided by BAKO Motors as the system under test. These components are outside the scope of the ISS project BOM and are not costed here.

## 13.4 Software & Licensing BOM

All software tools and frameworks used in this project are open-source or free for academic use. No commercial software licences were required. Table 26 summarises the tools and their applicable licences.

**Table 26:** Software and Licensing Bill of Materials

| **Tool / Framework** | Purpose | Licence |
|---|---|---|
| **ESP32 Arduino framework** | Embedded firmware development | Apache 2.0 |
| **KiCad 10.0** | PCB schematic and layout | GPL |
| **Python 3.11** | Emulator and FastAPI back-end | PSF |
| **FastAPI** | REST API and WebSocket server | MIT |
| **SQLite** | Cloud telemetry database | Public Domain |
| **VS Code** | Integrated development environment | MIT |
| **PlatformIO** | Build system and dependency management | Apache 2.0 |

## 13.5 Total Project Cost Estimate

The hardware procurement cost of approximately 240 TND represents only the direct material expenditure and significantly understates the true economic value of the project. A complete project cost estimate must account for all resource categories: human effort, professional expertise, infrastructure, and equipment access.

## Human Resources — Man-Day Estimate

The project team comprised seven students over one academic semester of approximately 18 weeks. Each team member contributed an estimated average of 3 working hours per day during sprint weeks, yielding the following man-day calculation:

$$7 \text{ members} \times 18 \text{ weeks} \times 3 \text{ days/week (effective)} \times 1 \text{ day} = \textbf{378} \text{ man-days}$$

Applying a conservative junior engineering day rate of 150 TND/day (consistent with Tunisian engineering graduate market rates), the human effort component alone represents approximately **56 700 TND** in equivalent labour value.

Academic supervision by Dr. Mortadha Dahmani added approximately 2 hours per week of expert guidance over 18 weeks, equivalent to 4.5 man-days of senior engineering supervision. At a senior expert daily rate of 500 TND/day, this contributes approximately **2 250 TND** in supervisory expertise value.

## Infrastructure & Equipment

- **MedTech FABLAB** — Access to soldering stations, oscilloscopes, logic analysers, and 3D printers was provided at no charge to the team. The equivalent commercial lab rental cost for 18 weeks at 4 hours per week would be approximately 3 600 TND at market rates (50 TND/hour).
- **VPS cloud server** — Provided by the academic supervisor at no charge to the project. The equivalent hosting cost for a comparable Linux VPS (2 vCPU, 4 GB RAM, public IP) over 6 months is approximately 360 TND.
- **BAKO Motors test vehicle** — Weekly access to the electric motorcycle prototype on the MedTech campus (approximately 4 hours/week, 12 sessions) represents a vehicle access value of approximately 1 200 TND at commercial EV workshop rates.

## Full Project Cost Summary

**Table 27:** Full Project Cost Breakdown — Material and Human Resources

| Cost Category | Quantity | Est. Value (TND) |
|---|---|---|
| **Electronics components** | 60 TND actual spend | 60 |
| **PCB fabrication** | 180 TND actual spend | 180 |
| **Student engineering effort** | 378 man-days @ 150 TND | 56 700 |
| **Academic supervision** | 4.5 man-days @ 500 TND | 2 250 |
| **FABLAB equipment access** | 72 hours @ 50 TND/hr | 3 600 |
| **VPS cloud hosting** | 6 months | 360 |
| **Vehicle access (BAKO)** | 12 sessions | 1 200 |
| **Total project value** | | **~64 350 TND** |
| **Direct cash spend** | | **240 TND** |

The 240 TND direct spend represents less than 0.4% of the total estimated project value, underscoring the significant contribution of academic infrastructure, supervisory expertise, and student engineering effort that made this project viable within a constrained budget. All software tools and development environments were open-source or free for academic use, contributing 0 TND to the software cost line.

# 14 RESOURCES

## 14.1 Human Resources & Skillset Matrix

The project team comprises seven members, each assigned to functional areas that align with their academic specialisation. Table 28 summarises the team composition and primary responsibilities.

**Table 28:** Human Resources and Responsibility Assignment

| Team Member | Primary Role | Key Deliverables |
|---|---|---|
| **Mohamed Baccouche** | Hardware / PCB Lead | KiCad schematic, PCB layout, BOM, CAN bus bring-up |
| **Mahdi Hachicha** | Embedded Firmware | ESP32 firmware, CAN decoding, sensor drivers |
| **Yassine Mtibaa** | GSM / Cloud Backend | SIM800L GSM integration, AT command firmware, FastAPI server, VPS deployment, cloud dashboard |
| **Hana Gdoura** | Wi-Fi / Wireless Dashboard | Wi-Fi AP mode integration, TCP dashboard connection, wireless testing |
| **Khadija Ben Hamida** | Emulation / Simulation | Python sensor emulator scenario design, data validation, results analysis |
| **Selima Grissa** | Emulation / Simulation | Emulator implementation, test scenario execution, V&V test report |
| **Fares Smaoui** | BLE / Wireless | BLE GATT server implementation, NimBLE firmware, wireless interface |
| **Dr. Mortadha Dahmani** | Academic Supervisor / Stakeholder | Project governance, VPS provision, milestone approval |

All team members contributed to the technical report, with sections distributed according to the area of responsibility. Version-controlled collaboration was managed through the shared private Git repository.

## 14.2 Equipment & Lab Resources

**FABLAB at MedTech Mediterranean Institute of Technology.** The MedTech FABLAB provided free access to the following equipment throughout the project: soldering stations with hot-air rework capability, digital storage oscilloscopes (100 MHz bandwidth), logic analysers compatible with SaleaeLogic software, regulated bench power supplies (0–30 V, 0–5 A), precision digital multimeters, 3D printers for enclosure prototyping, and a PCB rework station for component replacement.

**BAKO Motors Test Vehicle.** BAKO Motors made their electric motorcycle prototype available on the MedTech campus for controlled on-vehicle testing sessions. These sessions were scheduled on a weekly basis (approximately 4 hours per week) to validate CAN bus reception, sensor placement, and vehicle-in-motion data integrity.

**Personal Equipment.** Team members contributed laptops running VS Code, PlatformIO, and KiCad, as well as USB-to-serial adapters for firmware flashing and serial monitoring.

## 14.3 Software Tools & Licenses

All software tools used in this project are open-source or free for academic use. No commercial software licences were required or procured. Table 29 summarises the development toolchain.

**Table 29:** Software Development Tools

| Tool | Purpose | Licence |
|---|---|---|
| **KiCad 10.0** | PCB schematic capture and layout | GPL |
| **VS Code** | Integrated development environment | MIT |
| **PlatformIO** | Build system, dependency management | Apache 2.0 |
| **ESP32 Arduino framework** | Embedded firmware for ESP32 | Apache 2.0 |
| **Python 3.11** | Sensor emulator and FastAPI backend | PSF |
| **FastAPI / Uvicorn** | REST API and WebSocket server | MIT |
| **SQLite** | Cloud telemetry database on VPS | Public Domain |
| **SavvyCAN / PEAK PCANView** | CAN bus frame capture and analysis | Open-source / free |
| **Git / GitHub** | Version control and team collaboration | Free |
| **Google Drive** | Document sharing and report drafts | Free (academic) |

## 14.4 Cloud & Infrastructure Resources

The cloud infrastructure was provided at no cost by the academic supervisor, Dr. Mortadha Dahmani, who made available a Linux VPS with a public IP address. The following services were deployed on this VPS:

- **FastAPI backend** (Python/Uvicorn): handles HTTP ingest and WebSocket streaming on port 8787.
- **SQLite database** (`bms_cloud.db`): stores all received telemetry snapshots in a flat schema managed directly by the FastAPI server.
- **Web dashboard**: served as a static HTML file directly by the FastAPI backend.
- **MQTT Mosquitto broker**: deployed for future MQTT-based telemetry (not yet used in the current firmware).

VPS administration was performed via SSH. Server logs, database contents, and WebSocket connections are directly inspectable by the team without any third-party cloud portal. The VPS approach eliminated dependency on managed cloud services and kept all operational data within the project team's direct control.

## 14.5 Budget Allocation & Burn Rate

The actual project hardware spend was approximately 240 TND in total, allocated as follows:

- **Electronics components**: 60 TND (ESP32, MCP2515 modules, DS18B20 sensors, ACS712, passives, connectors — all locally sourced in Tunisia).
- **PCB fabrication**: 180 TND (local fabrication house, FR-4, 2-layer, 1.6 mm, Gerber RS-274X).
- **Lab equipment**: 0 TND (provided free by the MedTech FABLAB).
- **VPS / cloud hosting**: 0 TND (provided by Dr. Dahmani).
- **Software licences**: 0 TND (all open-source or free for academic use).

Total actual spend was 240 TND. All components were available from local Tunisian suppliers without import delays. No unplanned budget overruns occurred during any sprint.

# 15 LIMITATIONS

## 15.1 Technical Limitations

**ESP32 ADC Nonlinearity.** The ESP32's internal 12-bit ADC exhibits a documented nonlinearity in the lower voltage range (below approximately 150 mV) and at the top of its input range. This limits the effective resolution to approximately 11 bits ($\pm 0.1$ V at the ADC input). While calibration routines partially correct the systematic offset, the residual nonlinearity introduces an irreducible measurement uncertainty of approximately $\pm 1\%$ in voltage and current readings. For a production instrument, an external precision ADC (e.g., ADS1115) would eliminate this limitation.

**SIM800L 2G-Only Sunset Risk.** The SIM800L module is limited to 2G GSM/GPRS connectivity. Tunisian mobile operators (Ooredoo and Orange) are progressively decommissioning 2G infrastructure in favour of LTE networks. While the current firmware connects successfully, the 2G network availability cannot be guaranteed beyond the short term. The SIM7670E LTE Cat-M1 module is specified as the production replacement (and is already chosen as the populated component on the ISS board) to address this risk, but the firmware LTE migration was not completed within the academic project scope.

**CAN Protocol Dependency.** The current firmware is tightly coupled to the proprietary CAN DBC of the O'CELL IFS60.8-500-F-E3 BMS. Any change to the BMS model, firmware version, or communication protocol would require reverse engineering of the new frame structure and firmware updates. No generic CAN DBC parser is currently implemented; frame IDs and byte offsets are hardcoded.

**Absence of Galvanic Isolation.** The ISS board does not include galvanic isolation between the CAN bus side (running at automotive ground) and the ESP32 logic side. In a vehicle with multiple ground references, ground potential differences can cause noise injection into the ADC inputs or, in fault conditions, damage to the ESP32. A production design should incorporate isolated CAN transceivers (e.g., ISO1050) or optical coupling on the CAN lines.

## 15.2 Regulatory & Certification Limitations

**Electromagnetic Compatibility.** No formal EMC testing was conducted. The ISS board has not been tested to CISPR 25 (in-vehicle emissions) or EN 55032 (commercial electronics). The LM2596S-ADJ switching regulator operates at approximately 150 kHz; without input filtering, its conducted emissions may exceed Class B limits. Pre-compliance checks using a bench spectrum analyser showed no obvious anomalies, but formal certification testing has not been performed.

**PCB Design Tool Limitations.** KiCad 10.0 was used for schematic and PCB design. While KiCad is a capable open-source tool, it does not carry the same library of verified footprints, design rule sets, and simulation integration as commercial tools (Altium Designer, Cadence Allegro). Formal automotive PCB submissions typically

require Altium-format design files, which would necessitate a toolchain migration.

**Cellular Module Type Approval.** The SIM800L carries its own type approval for 2G operation. The SIM800L also carries LTE type approval from its manufacturer. However, neither module has been formally qualified for integration into a vehicle electronics system under Tunisian ATI (Agence Tunisienne des Infrastructures) regulations. Formal vehicle-level radio approval would be required for any public-road deployment.

**Component Automotive Grade.** The components used in this prototype (ESP32, ACS712, passive components) are commercial-grade rather than automotive-grade (AEC-Q100 for ICs, AEC-Q200 for passives). They are not rated for the extended temperature range ($-40$°C to $+125$°C) or vibration/shock levels specified by automotive standards. For deployment in a vehicle operating environment, automotive-grade equivalents should be specified in the production BOM.

## 15.3 Software & Code Constraints

**Hardcoded Configuration.** The current firmware stores network credentials, VPS endpoint URLs, and CAN frame byte offsets as compile-time constants. Any change to the server IP address, API key, or BMS protocol requires reflashing the ESP32. A production implementation would externalise these parameters to EEPROM or a configuration file readable at boot, eliminating the need for firmware recompilation on deployment changes.

**No OTA Firmware Update.** Over-the-air (OTA) firmware update capability was not implemented within the academic project scope. Updates currently require physical USB access to the ESP32. The ESP-IDF OTA partition scheme is supported by the ESP32 hardware and was deferred to a future revision.

**Single-File Firmware Architecture.** The embedded firmware is structured as a single large Arduino `.ino` file. While functional, this structure limits reusability and makes unit testing of individual modules (CAN decoder, sensor drivers, AT command layer) impractical without refactoring into separate compilation units with defined interfaces.

**Memory Constraints.** The ESP32 has 520 kB of on-chip SRAM. The current firmware uses approximately 40% of this for the cooperative loop stack, the JSON serialisation buffer (ArduinoJson), and the `BMSData` struct. Adding additional features (larger history buffers, additional sensor types, or a local OLED display driver) will require careful heap profiling to avoid stack overflow or heap exhaustion.

**Python Backend Dependency Coupling.** The FastAPI server and the browser dashboard are tightly coupled: the WebSocket message format (JSON field names and structure) is duplicated across the server-side Python code and the client-side JavaScript. Any change to the telemetry data model requires coordinated updates in both places with no formal contract or schema validation between them.

## 15.4 Budget & Resource Constraints

**Prototype-Only Pricing.** The BOM costs (Table 25) reflect single-unit prototype pricing. At production volumes (100+ units), component costs would decrease significantly through bulk purchasing and direct manufacturer pricing. PCB fabrication cost per unit would also decrease substantially at volume.

**Single-Engineer Disciplines.** Most functional areas (hardware, firmware, cloud backend, dashboard, testing) were covered by one or two team members each. This created single-engineer bottlenecks: when one team member was unavailable, the corresponding workstream was blocked. A production team would require redundancy across all disciplines.

**Incomplete BAKO Motors Documentation.** BAKO Motors did not have complete documentation for the O'CELL BMS CAN protocol, the MPPT controller electrical interface, or the motor controller signals. This required the team to perform reverse engineering activities (CAN frame capture and analysis with SavvyCAN) that were not anticipated in the project plan, consuming approximately one full sprint of additional effort.

## 15.5 Timeline Limitations

**Unplanned CAN Reverse Engineering.** The discovery that the O'CELL BMS uses a proprietary, undocumented CAN DBC was not identified in the initial risk register. Reverse engineering the frame structure consumed approximately three weeks of effort that had been allocated to sensor integration, compressing the timeline for Phase P2 and P3 of the prototype build.

**Short Field Testing Window.** Access to the BAKO Motors test vehicle was limited to approximately four hours per week. This restricted the amount of on-vehicle data that could be collected and reduced the opportunity for extended thermal and vibration testing under real driving conditions.

**Predictive Maintenance Model Immaturity.** The backlog included a predictive maintenance module based on battery degradation trend analysis. Insufficient historical data had been accumulated by the end of Sprint 8 to produce a statistically meaningful model. This item was moved to the Could Have backlog and deferred to a future project phase.

## 15.6 Technology Readiness Level (TRL) Limitations

The BAKO Motors ISS prototype is assessed at TRL 4 (technology validated in laboratory). The system has been demonstrated to function correctly in a controlled environment with the target BMS, but has not yet been validated in a representative operating environment (TRL 5) or demonstrated in a relevant environment (TRL 6). Full-vehicle integration, enclosure sealing, extended field testing under vibration and temperature variation, and regulatory compliance verification are required to advance the TRL to a level suitable for commercial deployment.

# 16 BLOCKAGES & ISSUE LOG

This section documents the technical and organisational blockers encountered during the project and the actions taken to resolve them. All blockers listed below were ultimately closed before the Production Readiness Review at Sprint 8.

## 16.1 Technical Deadlocks

**B-001 — O'CELL BMS CAN Protocol Undocumented.** **Sprint affected:** Sprint 2–3. **Severity:** Critical.

The O'CELL IFS60.8-500-F-E3 BMS does not publish a DBC file or any public documentation of its CAN frame structure. Without a known message layout, the firmware could not decode any battery data. The team resolved this by capturing raw CAN bus traffic using SavvyCAN, manually identifying frame IDs by correlating byte value changes with known vehicle states (charging, discharging, full charge), and reverse-engineering all five frame types over approximately one sprint. This consumed the entire Sprint 2 capacity and pushed other firmware tasks into Sprint 3.

**B-002 — SIM800L GPRS Bearer Intermittently Fails.** **Sprint affected:** Sprint 5. **Severity:** Major.

The SIM800L module randomly failed to open the GPRS bearer (`AT+SAPBR=1,1` returning `ERROR`) during initial integration testing. The failure was non-deterministic and could not be reproduced reliably on the bench. Root cause was identified as insufficient bulk capacitance on the module power supply rail: the GPRS transmit burst draws up to 2 A for 600 µs, causing the rail to droop below the module's minimum operating voltage. Adding a 2200 µF / 6.3 V electrolytic capacitor resolved the issue. This defect (D-002) was closed in Sprint 5 and the fix incorporated into PCB Rev. RECTIFICATION_2.

**B-003 — CAN Frame Reception Rate Below Target (95% vs 99%).** **Sprint affected:** Sprint 6. **Severity:** Major.

During the first V&V run, the CAN frame reception rate measured 95% against the 99% acceptance criterion. Intermittent bus-off events were observed on the MCP2515 error counter. The root cause was a missing 120 Ω termination resistor at the far end of the test harness. CAN bus signal integrity degraded with reflections at the unterminated end, causing occasional frame corruption. Adding the terminator at the vehicle harness end restored the reception rate to above 99% in subsequent tests (Defect D-001).

**B-004 — DS18B20 ROM Enumeration Fails on Cold Power-Up.** **Sprint affected:** Sprint 4. **Severity:** Minor.

On cold power-up (board powered from 0 V), the 1-Wire bus enumeration occasionally failed to detect all three DS18B20 sensors, returning only one or two ROM addresses. The failure was not reproducible after a warm reset. The root cause was identified as insufficient timing margin during the 1-Wire reset pulse while the ESP32 boot ROM was still initialising. Adding a 5 ms delay before the first `ds.reset()` call in `setup()` eliminated the failure (Defect D-003).

**B-005 — Dashboard Cell Grid Flickering at 10 Hz.** **Sprint affected:** Sprint 7. **Severity:** Enhancement.

The 19-cell voltage grid in the browser dashboard flickered visibly when the WebSocket pushed updates at 10 Hz. Each update triggered a full DOM redraw of all 19 cells, causing the browser to repaint the entire grid on every frame. The fix was to implement a differential DOM update: only cells whose voltage value or colour threshold had changed since the previous frame are updated. This eliminated the flicker and reduced browser CPU usage (Defect D-007).

## 16.2 Supply Chain & Procurement Blockages

**B-006 — SIM800L Module Available Online Only.** **Sprint affected:** Sprint 1 (planning). **Severity:** Minor.

The SIM800L GSM/GPRS module is not stocked by local Tunisian electronics distributors. The module had to be ordered from an online international distributor, introducing a 10–14 day lead time. The team mitigated this by placing the order during Sprint 1, before the firmware development phase began, ensuring the module arrived before it was needed in Sprint 5. Local component availability was confirmed for all other parts (ESP32, MCP2515, DS18B20, ACS712, passives).

**B-007 — PCB Fabrication Lead Time.** **Sprint affected:** Sprint 3–4. **Severity:** Minor.

The first PCB revision (Rev. 1) was submitted to a local Tunisian fabrication house at the end of Sprint 3. The 7-day fabrication lead time meant that component population and bring-up could not begin until Sprint 4. Two trace-width defects (Defect D-006) discovered during bring-up required the board to go through a second fabrication cycle (RECTIFICATION_2), adding a further 7-day delay. The team used this time to advance the firmware development and Python emulator work.

## 16.3 Dependency & Third-Party Blockages

**B-008 — Vehicle Access Restricted to Workshop Sessions.** **Sprint affected:** Sprint 4–8. **Severity:** Minor.

Physical access to the BAKO Motors vehicle for live CAN bus testing was limited to pre-scheduled workshop sessions, typically two sessions per sprint. Between sessions, all firmware testing was performed using the Python log replay simulator (`simulate.py`) and the CAN frame simulator firmware (`CAN_simulation_7_phases.ino`). This dependency was managed by capturing comprehensive CAN log files during each vehicle session and using them for offline development.

**B-009 — Wi-Fi Station Mode IP Instability.** **Sprint affected:** Sprint 3. **Severity:** Major.

The original firmware used Wi-Fi Station Mode, connecting the ESP32 to a local router as a client. The router's DHCP server assigned a different IP address after every ESP32 reboot, requiring the `SERVER_IP` constant in the firmware to be updated and reflashed before each session. In environments without a nearby router (e.g., the BAKO Motors workshop), the connection failed entirely. The team resolved this by migrating to ESP32 Access Point mode, which provides a fixed IP (192.168.4.1) and requires no external router.

## 16.4 Resolved Blockers Summary

Table 30 provides a consolidated view of all nine blockers, their resolution sprint, and closure status.

**Table 30:** Resolved Blockers Summary

| ID | **Description** | Sprint Found | Sprint Closed | Resolution |
|---|---|---|---|---|
| B-001 | **BMS CAN protocol undocumented** | Sprint 2 | Sprint 3 | Reverse engineering |
| B-002 | **SIM800L GPRS bearer fails** | Sprint 5 | Sprint 5 | 2200 µF capacitor |
| B-003 | **CAN reception rate 95%** | Sprint 6 | Sprint 7 | 120 Ω terminator |
| B-004 | **DS18B20 cold boot enumeration** | Sprint 4 | Sprint 4 | 5 ms delay in setup |
| B-005 | **Dashboard cell grid flicker** | Sprint 7 | Sprint 7 | Differential DOM update |
| B-006 | **SIM800L online order only** | Sprint 1 | Sprint 1 | Early order placed |
| B-007 | **PCB fabrication lead time** | Sprint 3 | Sprint 4 | Parallel firmware work |
| B-008 | **Vehicle access restricted** | Sprint 4 | Sprint 8 | Log replay simulator |
| B-009 | **Wi-Fi Station Mode IP instability** | Sprint 3 | Sprint 3 | AP mode migration |

# 17 RISK MANAGEMENT

## 17.1 Risk Register

Risks were identified, scored, and tracked from Sprint 1 onwards using a likelihood × impact matrix. Likelihood is rated 1 (rare) to 5 (almost certain); Impact is rated 1 (negligible) to 5 (critical). The Risk Priority Number (RPN) is the product of the two scores. Risks with RPN $\geq 12$ were assigned mitigation actions and tracked at every sprint review.

**Table 31:** Project Risk Register

| ID | Risk Description | L | I | RPN | Mitigation / Outcome |
|---|---|---|---|---|---|
| R-01 | Proprietary BMS CAN protocol requires reverse engineering — no DBC published | 5 | 5 | 25 | Allocated dedicated sprint for SavvyCAN capture and frame analysis. **Occurred:** consumed Sprint 2 (Blocker B-001). |
| R-02 | SIM800L 2G network sunset by Tunisian operators (Ooredoo, Orange) | 4 | 4 | 16 | Selected SIM800L (LTE Cat-M1) as primary module from the outset; SIM800L retained as schematic footprint only. **Avoided.** |
| R-03 | ADC nonlinearity limits current measurement accuracy | 3 | 3 | 9 | Calibration offset stored per unit; ACS712-30A selected for 2% full-scale spec. Monitored — accepted at $\pm 1.8\%$. |
| R-04 | SIM800L unavailable locally in Tunisia | 3 | 4 | 12 | Ordered from online distributor in Sprint 1 with 14-day lead time. **Occurred:** managed (B-006). |
| R-05 | PCB fabrication defects requiring re-spin | 3 | 4 | 12 | Rigorous KiCad DRC before Gerber export; peer review of schematic. **Occurred:** one re-spin (RECTIFICATION_2) due to trace width (D-006). |
| R-06 | Vehicle access insufficient for integration testing | 3 | 3 | 9 | Python log replay simulator and CAN frame simulator firmware developed as offline surrogates. Managed. |
| R-07 | Scope creep from additional sensor integrations | 2 | 3 | 6 | MoSCoW backlog used; GNSS, BLE deferred to roadmap. Controlled. |
| R-08 | GPRS bearer instability under peak transmit current | 3 | 4 | 12 | SIMCOM module design guide requires bulk capacitor on VBAT. **Occurred:** resolved with 2200 µF (B-002). |
| R-09 | ESP32 heap exhaustion from cooperative loop and JSON buffer | 2 | 4 | 8 | Static JSON buffer (ArduinoJson) sized conservatively; heap usage monitored. Not triggered in 2-hour tests. |
| R-10 | CAN bus signal integrity from missing termination | 2 | 4 | 8 | Termination requirement documented in wiring guide. **Occurred:** missing at harness end; resolved Sprint 7 (B-003). |

## 17.2 Safety Risk Analysis (FMEA)

A Failure Mode and Effects Analysis (FMEA) was conducted for the four highest-consequence failure modes of the ISS system in a deployed vehicle context.

**Table 32:** FMEA — Top Four Failure Modes

| Component | Failure Mode | Effect on System | S | O | D | Control |
|---|---|---|---|---|---|---|
| SIM800L power rail | Supply droop during GPRS burst | Modem reset; publish cycle missed; cloud data gap | 3 | 3 | 3 | 2200 µF bulk capacitor; brownout recovery firmware |
| MCP2515 CAN controller | Bus-off due to noise | CAN reception stops; BMSData struct stale | 4 | 2 | 3 | Auto-reinit on bus-off; MCP2515 error counter monitoring |
| DS18B20 sensor | Open circuit / disconnected | Temperature reading reports 0xFF; alert suppressed | 3 | 2 | 4 | Firmware skips 0xFF; alert on all-zero temperature |
| LM2596S-ADJ regulator | Thermal shutdown at high ambient | ESP32 power loss; complete system down | 4 | 1 | 3 | Thermal stress test validated to 62°C; copper flood on B.Cu |

S = Severity (1–5), O = Occurrence (1–5), D = Detection difficulty (1–5). RPN = S $\times$ O $\times$ D.

## 17.3 Cybersecurity Risk Assessment

The ISS system transmits telemetry over plain HTTP (AT+HTTPSSL=0) with an API key injected in the `X-Api-Key` header. The current security posture is assessed as follows:

- **Data confidentiality:** Low. HTTP traffic is unencrypted and visible to any party with access to the GPRS bearer. No personally identifiable information is transmitted; all data is vehicle telemetry. Risk accepted for the academic prototype phase.
- **API authentication:** Basic. The `X-Api-Key` header provides minimal protection against accidental ingestion from unrelated sources; it does not prevent a determined attacker who intercepts the key from injecting false data.
- **Recommended upgrade:** Enable HTTPS (AT+HTTPSSL=1 with CA certificate) and rotate API keys per vehicle before any production deployment. The FastAPI backend is already capable of TLS termination via a reverse proxy (nginx).
- **Physical security:** The ISS board is mounted inside the vehicle chassis. USB programming access requires physical vehicle access. No remote firmware update vector exists in the current firmware (OTA deferred).

## 17.4 Risk Review Cadence

The risk register was reviewed at the end of every sprint during the sprint retrospective. Risks that materialised were moved to the Blockages & Issue Log (Section 16) and new risks identified during the sprint were added to the register. The final risk review was conducted at the Sprint 8 PRR. Of the ten identified

risks, four occurred and were fully resolved before closure; six were avoided through mitigation or did not materialise.

# 18 QUALITY ASSURANCE

## 18.1 Quality Management Plan

The Quality Management Plan (QMP) for the BAKO Motors ISS project defines the quality criteria, measurement methods, and acceptance thresholds applied to each functional subsystem. Quality was evaluated across four primary subsystems, each with its own set of quantitative acceptance criteria.

**CAN Bus Subsystem.** Quality criterion: frame reception rate $\geq 99\%$ over a 60-minute continuous test session. Measurement method: comparison of the firmware frame counter against the expected frame count calculated from the BMS transmission interval (100 ms for high-priority frames). The initial test result of 95% was traced to a missing termination resistor and corrected; subsequent tests exceeded the 99% criterion.

**Temperature Measurement Subsystem.** Quality criterion: DS18B20 readings within $\pm 1$°C of a calibrated reference thermometer at three stable temperature points (ambient $\approx 25$°C, warm $\approx 45$°C, hot $\approx 70$°C). Measurement method: simultaneous reading of DS18B20 and reference thermometer in a temperature-controlled environment. Result: $\pm 0.5$°C maximum deviation across all three sensors at all test points.

**Current Measurement Subsystem.** Quality criterion: ACS712 current reading error $< 2\%$ of full scale (30 A) at three load points (0 A, 5 A, 15 A). Measurement method: ACS712 firmware output compared against a calibrated clamp meter reference. Result: $\pm 1.8\%$ maximum error, within the acceptance threshold.

**Voltage Measurement Subsystem.** Quality criterion: voltage divider output error $< 1\%$ of the reference multimeter reading at three input levels (12 V, 24 V, 48 V). Measurement method: bench power supply at known voltage, ESP32 ADC output versus Fluke multimeter. Result: $\pm 0.9\%$ maximum error.

## 18.2 Design Review Gates

Four formal design review gates were conducted during the project, aligned with the sprint cadence and the Agile development lifecycle.

**Gate 1 — Architecture Review (Sprint 1 End).** Scope: system architecture synoptic, component selection justification, initial risk register. Participants: full development team and Dr. Mortadha Dahmani (supervisor). Outcome: architecture approved with one action: validate MCP2515 KiCad footprint against physical module dimensions before PCB layout begins. Action closed within Sprint 2.

**Gate 2 — Hardware Design Review (Sprint 3 End).** Scope: completed KiCad schematic, PCB layout Revision 1, BOM draft. Participants: hardware team (Mohamed Baccouche) and supervisor. Outcome: layout approved with two actions: (1) increase power trace widths on the 12 V rail to handle 500 mA continuous; (2) add a pull-down resistor on the SIM800L PWRKEY line to prevent spurious power-on events. Both actions incorporated into RECTIFICATION_2.

**Gate 3 — Integration Review (Sprint 6 End).** Scope: integrated firmware running all sensors in the cooperative loop, WebSocket dashboard operational, cloud pipeline live. Participants: full team and supervisor. Outcome: integration accepted with one action: confirm CAN termination on vehicle harness and re-run frame reception test. Action closed at Sprint 7 with 99% result confirmed.

**Gate 4 — Production Readiness Review (Sprint 8 End).** Scope: V&V results, PRR checklist, complete BOM, report draft. Participants: full team and supervisor. Outcome: prototype declared production-ready for the academic phase. Two items deferred to next revision: enclosure design and OTA firmware update capability.

## 18.3 Test & Measurement Plan

The test and measurement plan defined the following test types executed across the project lifecycle:

- **Unit tests**: individual sensor driver functions tested in isolation using known inputs (e.g., simulated CAN frame bytes decoded against expected field values).
- **Integration tests**: all subsystems active simultaneously in the cooperative main loop; monitored for watchdog triggers, stack overflow, and heap exhaustion over a 30-minute run.
- **System tests**: end-to-end data flow from CAN bus to browser dashboard, including cloud pipeline path.
- **Performance tests**: sustained 2-hour CAN reception, 30-minute thermal stress, 1-hour MPPT voltage sweep.
- **Regression tests**: after each firmware change, a 10-minute smoke test confirmed that previously passing sensor reads continued to produce valid data.

All test results were recorded in a shared test log maintained in the team Git repository alongside the firmware source code.

## 18.4 Defect Tracking & Resolution

Defects were classified by severity using the four-level scheme in Table 33.

**Table 33:** Defect Severity Classification

| Severity | Definition |
|---|---|
| **Critical** | System inoperable; data completely lost or corrupted; safety hazard. Must be resolved before any further testing. |
| **Major** | A primary feature is non-functional or produces incorrect output; workaround required to proceed. Must be resolved before Gate review. |
| **Minor** | A secondary feature is degraded but the primary function remains correct. Can be deferred to next sprint if documented. |
| **Enhancement** | Improvement to usability, performance, or documentation with no functional impact. Added to backlog for future sprints. |

Table 34 lists the seven defects logged during the project. All defects were resolved and closed before the Production Readiness Review.

**Table 34:** Defect Log

| ID | Severity | Description | Root Cause | Status |
|---|---|---|---|---|
| D-001 | **Major** | CAN frame reception rate 95% (target 99%) | Missing 120 Ω terminator on harness | Closed |
| D-002 | **Major** | SIM800L GPRS bearer fails to open intermittently | Insufficient bulk capacitance on power rail | Closed |
| D-003 | **Minor** | DS18B20 ROM enumeration fails on cold power-up | 1-Wire timing margin; resolved by 5 ms delay | Closed |
| D-004 | **Minor** | ACS712 zero-current offset drifts with temperature | Calibration stored without temperature comp | Closed |
| D-005 | **Minor** | WebSocket disconnects not auto-recovered in browser | Missing reconnect logic in JS client | Closed |
| D-006 | **Major** | PCB Revision 1 power traces undersized (12 V rail) | Insufficient trace width calculation | Closed |
| D-007 | **Enhancement** | Dashboard cell grid flickers on 10 Hz refresh | Non-batched DOM updates; fixed with frame diff | Closed |

# 19 RESULTS AND FINDINGS

## 19.1 Technical Performance Results

Table 35 summarises the quantitative performance achieved by each hardware subsystem against the acceptance criteria defined in the V&V plan (Section 11).

**Table 35:** Hardware Subsystem Performance Results

| Subsystem | Acceptance Criterion | Measured Result | Status |
|---|---|---|---|
| **CAN bus reception** | $\geq 99\%$ frame rate | 95% (initial); $> 99\%$ after termination fix | **PASS** |
| **SOC accuracy** | Error $< \pm 2\%$ | $\pm 1.5\%$ (voltage-derived SOC) | **PASS** |
| **Temperature (DS18B20)** | $\pm 1$°C | $\pm 0.5$°C max deviation | **PASS** |
| **Current (ACS712)** | Error $< 2\%$ full scale | $\pm 1.8\%$ at 5 A test point | **PASS** |
| **Voltage (divider)** | Error $< 1\%$ | $\pm 0.9\%$ at all three levels | **PASS** |
| **GSM alert latency** | SMS within 15 s of trigger | $< 15$ s confirmed in all test events | **PASS** |
| **Cloud snapshot loss** | Zero loss over 1 hour | 0 lost snapshots out of 720 transmitted | **PASS** |
| **Board power draw** | $< 500$ mA at 12 V | 480 mA peak (all subsystems active) | **PASS** |

The voltage-derived SOC method (using the cell voltage-to-SOC lookup table calibrated against the O'CELL BMS datasheet) demonstrated $\pm 1.5\%$ accuracy relative to the BMS coulomb-counter SOC. This result exceeded the initial expectation and validated the decision to implement the voltage-based SOC as the primary display metric, with the BMS coulomb-counter SOC shown as a secondary reference.

## 19.2 Software & Connectivity Results

### 19.2.1 WebSocket Dashboard Performance

The WebSocket serial mode (`/ws`) delivered data at 10 Hz with zero dropped frames over a 10-minute monitoring window. The cloud mode (`/ws/cloud`) delivered at 2 Hz. Browser rendering of the 19-cell voltage grid with colour-coded thresholds was confirmed at $\geq 5$ frames per second with no visible flickering after the DOM batching fix (Defect D-007).

Auto-reconnect behaviour was validated by simulating a server restart: the browser successfully re-established the WebSocket connection within the 5-second retry interval in all five test attempts. The connection status indicator transitioned correctly between Disconnected (red), Waiting (yellow), and Live (green) states.

### 19.2.2 Serial Auto-Detection

The backend serial mode automatically detected the ESP32 USB device on startup using the `pyserial` port enumeration. During testing on three different host computers (two running Linux, one running Windows 11), the ESP32 was detected correctly in all cases without manual port configuration.

### 19.2.3 Wi-Fi Access Point Mode

The ESP32 Wi-Fi AP mode (`WiFi.softAP()`) was validated as a replacement for Station Mode. The AP assigned the fixed IP address 192.168.4.1 regardless of external router availability. TCP connection from a laptop browser was established within 2 seconds of AP advertisement. The AP mode operated reliably across all workshop test sessions, including environments with no available external Wi-Fi router.

### 19.2.4 Cloud Pipeline Results

End-to-end cloud pipeline validation confirmed:

- HTTP POST from the ESP32 to the VPS (`POST /api/ingest`) was acknowledged within 3 seconds over GPRS, including bearer setup.
- SQLite database on the VPS accumulated 720 snapshot rows over a 1-hour test session (1 snapshot per 5 seconds), with no insertion failures.
- Browser clients connected to `/ws/cloud` received live updates within 2 seconds of each new snapshot arrival.
- The `/api/history` endpoint returned the last 100 snapshots in under 100 ms on the VPS hardware.

### 19.2.5 Python Emulator Validation

The seven-scenario Python emulator was run against the live backend for a complete 210-second cycle (7 scenarios $\times$ 30 seconds each). The dashboard correctly displayed all seven scenario states with appropriate colour coding and threshold alerts. The emulator-generated data was indistinguishable from ESP32-generated data at the API level, confirming that the backend schema and the emulator output are consistent.

## 19.3 Cost vs. Budget Analysis

Table 36 presents the final cost versus budget comparison.

**Table 36:** Cost vs. Budget Analysis

| Cost Category | Budget (TND) | Actual (TND) | Variance (TND) |
|---|---|---|---|
| **Electronics components (local)** | 65 | 60 | −5 |
| **PCB fabrication** | 190 | 180 | −10 |
| **Lab equipment (FABLAB)** | 0 | 0 | 0 |
| **VPS / cloud hosting** | 0 | 0 | 0 |
| **Software licences** | 0 | 0 | 0 |
| **Total** | **255** | **240** | −15 |

The project was delivered within budget with a net underspend of 15 TND (5.9%). Savings were achieved through efficient local component sourcing and PCB fabrication negotiation. All software tools and lab equipment were available at no cost.

## 19.4 Key Findings Summary

Seven key findings emerged from the project execution and validation phase:

1. **CAN reverse engineering was the critical path.** The O'CELL BMS does not publish its CAN DBC. Reverse engineering the frame structure consumed one full sprint. Future projects with proprietary BMS protocols should allocate a dedicated reverse engineering sprint and consider SavvyCAN or commercial DBC analysers from the outset.
2. **Wi-Fi AP mode is more robust than Station Mode for field deployment.** Fixed IP (192.168.4.1), no router dependency, and deterministic reconnection make AP mode the correct architecture for an in-vehicle monitoring unit. Station Mode introduced DHCP-assigned IP variability and dependence on a nearby router that was unavailable in the BAKO Motors workshop.
3. **Voltage-derived SOC is more accurate than the BMS coulomb counter for this application.** At $\pm 1.5\%$ accuracy, the voltage-to-SOC lookup table matched the BMS value closely enough to serve as the primary SOC metric while being more transparent and easier to recalibrate when a new BMS is installed.
4. **The SIM800L requires a 2200 µF bulk capacitor on its power rail.** The GPRS transmit burst draws up to 2 A for 600 µs; without adequate bulk capacitance, the power supply droops and causes intermittent bearer failures. This is a known hardware design requirement for SIMCOM modules and should be included in the PCB design from the first revision.
5. **ACS712-30A is marginal for currents below 5 A.** The 30 A full-scale range of the ACS712 gives a sensitivity of 66 mV/A. At low MPPT output currents (1–3 A), the ADC noise floor dominates the reading. The ACS758-50B (sensitivity 40 mV/A, but with better resolution at low currents for the application range) or a shunt-based measurement would improve low-current accuracy.
6. **Integration testing should begin no later than Sprint 3.** Integration defects (missing termination, power rail drooping, 1-Wire timing) were not discovered until Sprint 6–7 because subsystems were tested independently for too long. Earlier cross-subsystem testing would have reduced the corrective effort.

7. **The system is production-ready for the academic prototype phase.** All V&V criteria were met after defect resolution. The dashboard, cloud pipeline, and firmware are fully functional and demonstrated to the supervisor. The PCB design is at RECTIFICATION_2 and ready for Gerber export to a fabrication house.

# 20 RECOMMENDATIONS

## 20.1 Technical Recommendations

### Hardware Improvements

**DS18B20 External Power Mode.** The current firmware uses the DS18B20 parasitic power mode, which draws operating current from the data line during conversion. In electrically noisy environments (near switching regulators or CAN bus transients), this can cause spurious conversion failures. Rewiring the DS18B20 VDD pin to the 3.3 V rail (external power mode) eliminates parasitic-power timing constraints and increases measurement reliability. The PCB should include a dedicated 3.3 V pad at each DS18B20 connector for this purpose in the next revision.

**SIM800L Bulk Capacitor.** A 2200 µF, 6.3 V electrolytic capacitor should be placed on the SIM800L power supply rail, as close to the module's VBAT pins as possible. This is a mandatory requirement identified during integration testing (Defect D-002) and is consistent with SIMCOM module hardware design guidelines. The capacitor prevents the GPRS transmit burst from drooping the supply below the module's minimum operating voltage and is already incorporated into PCB Revision RECTIFICATION_2.

**Replace ACS712-30A with ACS758-50B.** The ACS712-30A has a sensitivity of 66 mV/A, which produces a low signal-to-noise ratio at MPPT currents below 5 A. The ACS758-50B offers a sensitivity of 40 mV/A in a PCB-mount package with improved linearity and lower noise floor at small currents. Alternatively, a shunt resistor (e.g., 5 mΩ) with a dedicated instrumentation amplifier (INA219) would provide 12-bit precision current measurement independent of ADC nonlinearity.

**Galvanic Isolation on CAN Bus.** The production revision of the ISS board should incorporate an isolated CAN transceiver (e.g., Texas Instruments ISO1050) to provide galvanic isolation between the vehicle CAN ground and the ESP32 logic ground. This eliminates ground loop noise injection into the ADC inputs and protects the ESP32 from automotive-level transients (ISO 7637-2 pulses).

### Firmware Improvements

**Protected Shared State for Multi-Core Expansion.** The current single-threaded firmware reads and writes the `BMSData` struct sequentially with no concurrency risk. If the firmware is extended to use FreeRTOS tasks on both ESP32 cores in a future revision, a `SemaphoreHandle_t` mutex wrapping all writes and reads to `BMSData` should be added to eliminate potential race conditions between the CAN receive task and the publish task.

**Non-Volatile Storage of Last Known Good Values.** If the ESP32 restarts while the vehicle is in operation, all accumulated CAN data is lost and the dashboard shows a "waiting" state until the next BMS frame is received. Using the ESP32's NVS (Non-Volatile Storage) flash partition to persist the last known good `BMSData`

struct would allow the dashboard to display the pre-restart values immediately on reconnection, improving user experience.

## 20.2 Process Recommendations

**Begin Integration Testing at Sprint 3.** Key finding 6 (Section 19) identified that late integration testing (Sprint 6–7) caused avoidable rework. The recommended process change is to perform a 30-minute cross-subsystem integration smoke test at the end of every sprint from Sprint 3 onwards. This would surface hardware incompatibilities (missing termination, power rail drooping) while corrective cost is still low.

**Calibration Log Per Unit.** Each ISS board unit should receive a per-unit calibration log documenting the ACS712 zero-current offset, the voltage divider correction coefficients, and the DS18B20 ROM addresses. This log should be stored in the NVS flash and retrievable via the `/api/calibration` endpoint. Centralised calibration records would support fleet-level monitoring and simplify field maintenance.

**Live Wiring Diagram.** The current project documentation includes the KiCad schematic but does not include a harness-level wiring diagram showing how the ISS board connects to the BAKO Motors vehicle harness. A simplified wiring diagram (colour-coded by subsystem) should be produced and laminated for use by the BAKO Motors workshop team during installation and troubleshooting.

**Arabic and French User Guide.** The dashboard and firmware error messages are currently in English only. Given that BAKO Motors operates in Tunisia, a brief user guide in Arabic and French covering dashboard interpretation, fault LED meanings, and basic troubleshooting should be produced for workshop technicians.

## 20.3 Future Development Roadmap

**Short-Term (Next 3–6 Months).**

- **GNSS Integration**: add a NEO-6M or NEO-M8N GPS module connected to the ESP32 via UART. Include latitude, longitude, and speed in the telemetry JSON payload. Display vehicle position on an embedded Leaflet.js map in the dashboard.
- **Bluetooth Companion App**: implement a BLE GATT server on the ESP32 to stream live battery SOC and fault status to a smartphone companion application. This would allow the rider to monitor the vehicle without connecting to the Wi-Fi AP.
- **Enclosure Design**: design a sealed (IP54 minimum) ABS or polycarbonate enclosure for the ISS board with cable glands for the vehicle harness. Submit for 3D printing and evaluate fitment on the motorcycle frame.

**Medium-Term (6–18 Months).**

- **Historical Cloud Dashboard**: add time-series charting to the cloud dashboard (Chart.js or ECharts) to display battery SOC, cell temperatures, and MPPT current trends over the last 24 hours, 7 days, and 30 days. Add date-range CSV export.
- **Predictive Maintenance Model**: use the accumulated SQLite telemetry history to train a simple battery degradation model (e.g., linear regression on cell spread over time). Deploy the model as a FastAPI background task that emits a "battery health" score updated daily.
- **LTE Firmware Optimisation**: refine the SIM800L AT+HTTP* command sequence to take advantage of LTE Cat-M1 throughput, including increasing the publish frequency from the current 5-second interval to 1 Hz and enabling persistent bearer mode to reduce per-transaction setup overhead.

**Long-Term (18+ Months).**

- **Fleet Management Platform**: extend the cloud backend to support multiple vehicle device IDs, with a per-vehicle dashboard, aggregate fleet SOC view, and automated maintenance scheduling based on predictive model outputs.
- **Standalone Monitoring Product**: package the ISS system as a standalone product for sale to other Tunisian electric vehicle manufacturers. This would require formal CE marking, automotive-grade component qualification, and an Arabic/French localisation of the full dashboard.

# 21 CONCLUSION

The BAKO Motors Intelligent Supervision System (ISS) project set out to solve five specific operational gaps in the BAKO Motors electric motorcycle prototype: the absence of real-time sensor coverage beyond the battery pack, the lack of a monitoring interface for engineers, the restriction of CAN bus communication to the BMS only, insufficient system documentation for maintenance teams, and the inability to perform remote diagnostics. All five gaps have been addressed by the system delivered in this project.

The hardware layer — an ESP32-based custom two-layer PCB integrating an MCP2515 CAN controller, TJA1051T transceiver, three DS18B20 temperature sensors, two ACS712 current sensors, and a SIM800L GSM/GPRS module — provides a unified data acquisition platform for the entire vehicle. The firmware layer implements a single-threaded cooperative loop that decodes CAN frames from the O'CELL IFS60.8-500-F-E3 BMS, polls external sensors at appropriate rates, and publishes telemetry snapshots over both Wi-Fi and cellular. The cloud layer, built on FastAPI and deployed on a Linux VPS, receives these snapshots via HTTP POST, stores them in SQLite, and streams them to browser clients at up to 10 Hz via WebSocket. The dashboard layer presents all 19 battery cells, four temperature probes, current and voltage measurements, and the vehicle handbrake state in a single real-time web interface with data export capability.

The project was executed across eight two-week Agile sprints, achieving an average velocity of 18 story points per sprint and delivering 144 of the 160 committed story points. The 16 deferred points represent explicitly deprioritised items (OTA firmware update, enclosure design, full-solar field validation) rather than incomplete or broken features. All quantitative V&V criteria were met: CAN frame reception $> 99\%$ after termination correction, DS18B20 temperature accuracy $\pm 0.5$°C, ACS712 current error $\pm 1.8\%$, voltage measurement error $\pm 0.9\%$, and GSM alert latency $< 15$ s. The dashboard renders at $\geq 5$ frames per second and the cloud pipeline sustained zero snapshot loss over a 1-hour test.

The total project expenditure was approximately 240 TND (60 TND in electronics components and 180 TND in PCB fabrication), representing a 5.9% underspend against the planned budget, achieved through successful local component sourcing and PCB fabrication negotiation. The board is at revision RECTIFICATION_2 and is ready for Gerber export and submission to a PCB fabrication house for a production batch.

Seven key findings were identified during the project that will inform future monitoring system designs for Tunisian electric vehicle manufacturers: the criticality of early CAN protocol reverse engineering, the superiority of Wi-Fi AP mode over Station Mode for in-vehicle applications, the accuracy of voltage-derived SOC for LiFePO4 chemistries, the mandatory 2200 µF bulk capacitor requirement for the SIM800L GSM/GPRS module, the marginal performance of ACS712-30A at low currents, and the importance of cross-subsystem integration testing from the third sprint onwards.

The BAKO Motors ISS prototype demonstrates that a fully functional, cloud-connected vehicle monitoring system can be designed, fabricated, and validated by an academic engineering team using open-source tools, locally available components, and a total budget under 600 TND. The architecture and codebase are documented, open, and ready to serve as the foundation for the fleet management platform and standalone product roadmap described in Chapter 20.

# REFERENCES


[1] BAKO Motors, "Battery pack technical specification — 72 v 230 ah," BAKO Motors, Tunis, Tunisia, Tech. Rep., 2025, Internal document provided by BAKO Motors. Ref: Bat72230Ah.pdf.

[2] O'CELL Energy, "O'CELL battery cell specification — 60.8 v 50 ah LiFePO4," O'CELL Energy Co., Ltd., Tech. Rep., 2024, Product specification document provided by BAKO Motors. Ref: OCELL_Specification_60.8V50Ah.pdf.

[3] BAKO Motors, "Vehicle CAN bus communication protocol," BAKO Motors, Tunis, Tunisia, Tech. Rep., 2025, Internal document provided by BAKO Motors. Ref: Communication CAN.pdf.

[4] BAKO Motors, "Electric motor technical documentation — EM701.102.V01," BAKO Motors, Tunis, Tunisia, Tech. Rep., 2025, Internal document provided by BAKO Motors. Ref: 2_EM701.102.V01-motor PDF.pdf.

[5] PlatformIO Labs, *PlatformIO — a professional collaborative platform for embedded development*, 2024. [Online]. Available: `https://platformio.org`.

[6] KiCad Developers, *KiCad EDA — schematic capture and PCB layout*, Version 10.0, 2024. [Online]. Available: `https://www.kicad.org`.

[7] SnapEDA Inc., *SnapEDA — free PCB symbols, footprints and 3D models*, Used for KiCad component symbols and PCB footprints. Accessed: 2026, 2025. [Online]. Available: `https://www.snapeda.com`.

[8] KiCad Community, *KiCad community forum*, Used for KiCad schematic and PCB layout guidance. Accessed: 2026, 2025. [Online]. Available: `https://forum.kicad.info`.

[9] GitHub, Inc., *GitHub — code hosting and version control platform*, Project source code hosted at `https://github.com/MohaBc/Bako_OBD_ISS`. Accessed: 2026, 2025. [Online]. Available: `https://github.com`.

[10] Alldatasheet.com, *Alldatasheet — electronic components datasheet search*, Used for component datasheet retrieval during hardware design. Accessed: 2026, 2025. [Online]. Available: `https://www.alldatasheet.com`.

[11] Google LLC, *YouTube — video platform*, Used for tutorial videos on KiCad PCB design, ESP32 firmware development, and CAN bus protocol. Accessed: 2026, 2025. [Online]. Available: `https://www.youtube.com`.

[12] Google LLC, *Google scholar — academic search engine*, Used for literature search. Accessed: 2026, 2025. [Online]. Available: `https://scholar.google.com`.

[13] Anthropic, *Claude — AI assistant by anthropic*, Used for technical writing assistance and code review during report preparation. Accessed: 2026, 2025. [Online]. Available: https://claude.ai.

[14] Espressif Systems, *ESP32 series datasheet*, Version 4.4, Espressif Systems (Shanghai) Co., Ltd., 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

[15] Microchip Technology, *MCP2515 stand-alone CAN controller with SPI interface*, DS20001801H, Microchip Technology Inc., 2019. [Online]. Available: https://www.microchip.com/content/dam/mchp/documents/APID/ProductDocuments/DataSheets/MCP2515-Family-Data-Sheet-DS20001801K.pdf.

[16] *Road vehicles — controller area network (CAN) — part 1: Data link layer and physical signalling*, Standard, International Organization for Standardization, 2015.

[17] NXP Semiconductors, *TJA1051 high-speed CAN transceiver*, Rev. 3, NXP Semiconductors, 2017. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/TJA1051.pdf.

[18] Texas Instruments, *LM2596 simple switcher power converter 150 khz 3-a step-down voltage regulator*, SNVS124G, Texas Instruments Incorporated, 2021. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm2596.pdf.

[19] BAKO Motors, "Electric vehicle wiring diagram," BAKO Motors, Tunis, Tunisia, Tech. Rep., 2025, Internal document provided by BAKO Motors. Ref: Ev_wirring_diagram.pdf.

[20] *Serial control and communications vehicle network — part 21: Data link layer*, Standard, SAE International, 2010.

[21] Maxim Integrated, *DS18B20 programmable resolution 1-wire digital thermometer*, Rev. 6, Maxim Integrated Products, 2019. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf.

[22] L. Chen, Z. Wang, Z. Lü, *et al.*, "State of charge estimation for lithium iron phosphate battery pack using the cell difference voltage," *Energies*, vol. 12, no. 13, p. 2578, 2019. DOI: 10.3390/en12132578.

[23] T. Wik, B. Fridholm, and H. Kuusisto, "Battery management systems for electric vehicle range prediction," *IFAC Proceedings Volumes*, vol. 49, no. 11, pp. 99–106, 2016. DOI: 10.1016/j.ifacol.2016.08.015.

[24] B. Blanchon, *ArduinoJson: Efficient JSON for Arduino*, Version 7.x, 2024. [Online]. Available: https://arduinojson.org.

[25] P. Ranjith, S. Chandra, and K. Suresh, "Real-time battery monitoring system for electric vehicles using CAN bus and IoT," in *Proceedings of the International Conference on Smart Systems and Inventive Technology (ICSSIT)*, 2022, pp. 1143–1148. DOI: 10.1109/ICSSIT53264.2022.9716334.

[26] T. Christie, *Uvicorn — an ASGI web server*, 2024. [Online]. Available: https://www.uvicorn.org.

[27] S. Ramírez, *FastAPI — modern, fast web framework for building APIs with python*, Version 0.111, 2024. [Online]. Available: https://fastapi.tiangolo.com.

[28] I. Fette and A. Melnikov, *The WebSocket protocol*, RFC 6455, Internet Engineering Task Force (IETF), 2011. [Online]. Available: https://tools.ietf.org/html/rfc6455.

[29] K. Schwaber and J. Sutherland, *The Scrum Guide — The Definitive Guide to Scrum: The Rules of the Game*. 2020. [Online]. Available: https://scrumguides.org/scrum-guide.html.

# APPENDICES

**A  Glossary of Terms & Abbreviations**

**B  Schematic & PCB Layout Package**

**C  3D Model & Drawing Package**

**D  Software Repository & Build Instructions**

**E  Test Reports & Certificates**

**F  Supplier Datasheets & Approvals**

**G  Meeting Minutes & Decision Log**

**H  Revision History**